Skip List

0.4.5

Generated by Doxygen 1.9.1

1 Main Page 1

1.1 General . . . 1

1.2 Examples of Usage . . . 2

1.2.1 C++ . . . 2

1.2.2 Python . . . 2

1.3 Design . . . 2

1.3.1 Recursive Search for the Node Position . . . 2

1.3.2 Node Creation . . . 3

1.3.3 Recursive Unwinding . . . 3

1.4 Algorithms . . . 3

1.4.1 has(T &val) const; . . . 3

1.4.2 at(size_t index) const; . . . 3

1.4.3 insert(T &val) . . . 4

1.4.4 remove(T &val) . . . 4

1.5 Code Layout . . . 4

1.5.1 Other Files of Significance . . . 5

1.6 Code Idioms . . . 5

1.6.1 Prevent Copying . . . 5

1.6.2 Reverse Loop of Unsigned int . . . 5

1.7 Roads not Travelled . . . 5

1.7.1 Key/Value Implementation . . . 5

1.7.2 Adversarial Users . . . 5

1.7.3 Probability != 0.5 . . . 6

1.7.4 Optimisation: Re-index Nodes on Complete Traversal . . . 6

1.7.5 Optimisation: Reuse removed nodes for insert() . . . 6

1.7.6 Reference Counting . . . 6

1.7.7 Use and Array of <tt>{skip_width, Node<T>*}</tt> rather than a vector . . . 6

1.8 Performance . . . 6

1.8.1 Performance of at() and has() . . . 6

1.8.2 Performance of insert() and remove() . . . 7

1.8.3 Performance of a rolling median . . . 7

1.8.4 Space Complexity . . . 8

2 Todo List 9

3 Namespace Index 11

3.1 Namespace List . . . 11

4 Hierarchical Index 13

4.1 Class Hierarchy . . . 13

5 Class Index 15

5.1 Class List . . . 15

6 File Index 17
6.1 File List . . . 17

7 Namespace Documentation 19
7.1 OrderedStructs Namespace Reference . . . 19
7.1.1 Detailed Description . . . 19
7.2 OrderedStructs::RollingMedian Namespace Reference . . . 19
7.2.1 Detailed Description . . . 20
7.2.2 Enumeration Type Documentation . . . 20
7.2.2.1 RollingMedianResult . . . 20
7.2.3 Function Documentation . . . 20
7.2.3.1 dest_count() . . . 21
7.2.3.2 dest_size() . . . 21
7.2.3.3 even_index() . . . 21
7.2.3.4 even_odd_index() . . . 22
7.2.3.5 odd_index() . . . 23
7.2.3.6 rolling_median() . . . 24
7.2.3.7 rolling_median_lower_bound() . . . 24
7.3 OrderedStructs::SkipList Namespace Reference . . . 25
7.3.1 Detailed Description . . . 26
7.3.2 Enumeration Type Documentation . . . 26
7.3.2.1 IntegrityCheck . . . 26
7.3.3 Function Documentation . . . 27
7.3.3.1 _throw_exceeds_size() . . . 27
7.3.3.2 seedRand() . . . 27
7.3.3.3 tossCoin() . . . 27
7.3.4 Variable Documentation . . . 28
7.3.4.1 gSkipListMutex . . . 28

8 Class Documentation 29
8.1 AcquireLock Class Reference . . . 29
8.1.1 Detailed Description . . . 29
8.1.2 Constructor & Destructor Documentation . . . 29
8.1.2.1 AcquireLock() . . . 29
8.1.2.2 ~AcquireLock() . . . 30
8.1.3 Member Data Documentation . . . 30
8.1.3.1 _pSL . . . 30
8.2 cmpPyObject Class Reference . . . 30
8.2.1 Detailed Description . . . 31
8.2.2 Constructor & Destructor Documentation . . . 31
8.2.2.1 cmpPyObject() [1/2] . . . 31
8.2.2.2 ~cmpPyObject() . . . 31
8.2.2.3 cmpPyObject() [2/2] . . . 32

8.2.3 Member Function Documentation . . . 32
8.2.3.1 copy_cmpPyObject() . . . 32
8.2.3.2 operator()() . . . 32
8.2.3.3 operator=() . . . 33
8.2.4 Member Data Documentation . . . 33
8.2.4.1 cmp_func . . . 33
8.3 OrderedStructs::SkipList::Exception Class Reference . . . 33
8.3.1 Detailed Description . . . 34
8.3.2 Constructor & Destructor Documentation . . . 34
8.3.2.1 Exception() . . . 34
8.3.2.2 ∼Exception() . . . 34
8.3.3 Member Function Documentation . . . 34
8.3.3.1 message() . . . 34
8.3.4 Member Data Documentation . . . 34
8.3.4.1 msg . . . 35
8.4 ExecClock Class Reference . . . 35
8.4.1 Detailed Description . . . 35
8.4.2 Member Typedef Documentation . . . 35
8.4.2.1 tHiResDouble . . . 35
8.4.3 Constructor & Destructor Documentation . . . 36
8.4.3.1 ExecClock() . . . 36
8.4.4 Member Function Documentation . . . 36
8.4.4.1 seconds() . . . 36
8.4.5 Member Data Documentation . . . 36
8.4.5.1 _start . . . 36
8.5 OrderedStructs::SkipList::FailedComparison Class Reference . . . 36
8.5.1 Detailed Description . . . 37
8.5.2 Constructor & Destructor Documentation . . . 37
8.5.2.1 FailedComparison() . . . 37
8.6 OrderedStructs::SkipList::HeadNode< T, Compare > Class Template Reference . . . 37
8.6.1 Detailed Description . . . 38
8.6.2 Constructor & Destructor Documentation . . . 39
8.6.2.1 HeadNode() [1/2] . . . 39
8.6.2.2 ∼HeadNode() . . . 39
8.6.2.3 HeadNode() [2/2] . . . 39
8.6.3 Member Function Documentation . . . 40
8.6.3.1 _adjRemoveRefs() . . . 40
8.6.3.2 _lacksIntegrityCyclicReferences() . . . 40
8.6.3.3 _lacksIntegrityNodeReferencesNotInList() . . . 41
8.6.3.4 _lacksIntegrityOrder() . . . 41
8.6.3.5 _lacksIntegrityWidthAccumulation() . . . 41
8.6.3.6 _nodeAt() . . . 42

8.6.3.7 _throwIfValueDoesNotCompare() . . . 42
8.6.3.8 _throwValueErrorNotFound() . . . 43
8.6.3.9 at() [1/2] . . . 43
8.6.3.10 at() [2/2] . . . 44
8.6.3.11 dotFile() . . . 44
8.6.3.12 dotFileFinalise() . . . 45
8.6.3.13 has() . . . 45
8.6.3.14 height() [1/2] . . . 46
8.6.3.15 height() [2/2] . . . 46
8.6.3.16 index() . . . 47
8.6.3.17 insert() . . . 47
8.6.3.18 lacksIntegrity() . . . 48
8.6.3.19 operator=() . . . 48
8.6.3.20 remove() . . . 48
8.6.3.21 size() . . . 49
8.6.3.22 size_of() . . . 49
8.6.3.23 width() . . . 49
8.6.4 Member Data Documentation . . . 51
8.6.4.1 _compare . . . 51
8.6.4.2 _count . . . 51
8.6.4.3 _dot_file_subgraph . . . 51
8.6.4.4 _nodeRefs . . . 52
8.7 OrderedStructs::SkipList::IndexError Class Reference . . . 52
8.7.1 Detailed Description . . . 52
8.7.2 Constructor & Destructor Documentation . . . 52
8.7.2.1 IndexError() . . . 53
8.8 module_state Struct Reference . . . 53
8.8.1 Detailed Description . . . 53
8.8.2 Member Data Documentation . . . 53
8.8.2.1 error . . . 53
8.9 OrderedStructs::SkipList::Node< T, Compare > Class Template Reference . . . 53
8.9.1 Detailed Description . . . 54
8.9.2 Constructor & Destructor Documentation . . . 55
8.9.2.1 Node() [1/2] . . . 55
8.9.2.2 Node() [2/2] . . . 55
8.9.3 Member Function Documentation . . . 55
8.9.3.1 _adjRemoveRefs() . . . 56
8.9.3.2 at() . . . 56
8.9.3.3 dotFile() . . . 57
8.9.3.4 has() . . . 57
8.9.3.5 height() . . . 58
8.9.3.6 index() . . . 58

8.9.3.7 insert() . . . 58
8.9.3.8 lacksIntegrity() . . . 59
8.9.3.9 lacksIntegrityRefsInSet() . . . 60
8.9.3.10 next() . . . 60
8.9.3.11 nodeRefs() [1/2] . . . 60
8.9.3.12 nodeRefs() [2/2] . . . 61
8.9.3.13 operator=() . . . 61
8.9.3.14 pNode() . . . 61
8.9.3.15 remove() . . . 62
8.9.3.16 size_of() . . . 62
8.9.3.17 value() . . . 63
8.9.3.18 width() . . . 63
8.9.3.19 writeNode() . . . 63
8.9.4 Member Data Documentation . . . 64
8.9.4.1 _compare . . . 64
8.9.4.2 _nodeRefs . . . 64
8.9.4.3 _value . . . 64
8.10 OrderedStructs::SkipList::NodeRef< T, Compare > Struct Template Reference . . . 64
8.10.1 Detailed Description . . . 65
8.10.2 Member Data Documentation . . . 65
8.10.2.1 pNode . . . 65
8.10.2.2 width . . . 65
8.11 reversed< T > Struct Template Reference . . . 65
8.11.1 Detailed Description . . . 66
8.11.2 Member Function Documentation . . . 66
8.11.2.1 operator()() . . . 66
8.12 SkipList Struct Reference . . . 66
8.12.1 Detailed Description . . . 67
8.12.2 Member Data Documentation . . . 67
8.12.2.1 . . . 67
8.12.2.2 _data_type . . . 67
8.12.2.3 lock . . . 67
8.12.2.4 pSl_bytes . . . 67
8.12.2.5 pSl_double . . . 68
8.12.2.6 pSl_long . . . 68
8.12.2.7 pSl_object . . . 68
8.12.2.8 pSl_void . . . 68
8.13 StreamFormatState Class Reference . . . 68
8.13.1 Detailed Description . . . 69
8.13.2 Constructor & Destructor Documentation . . . 69
8.13.2.1 StreamFormatState() . . . 69
8.13.2.2 ~StreamFormatState() . . . 69

8.13.3 Member Data Documentation . . . 70
8.13.3.1 _ff . . . 70
8.13.3.2 _fill . . . 70
8.13.3.3 _pStream . . . 70
8.14 SubTestCount Class Reference . . . 70
8.14.1 Detailed Description . . . 71
8.14.2 Constructor & Destructor Documentation . . . 71
8.14.2.1 SubTestCount() . . . 71
8.14.3 Member Function Documentation . . . 71
8.14.3.1 failure() . . . 71
8.14.3.2 test() . . . 71
8.14.3.3 test_count() . . . 71
8.14.3.4 test_failures() . . . 72
8.14.4 Member Data Documentation . . . 72
8.14.4.1 m_failure . . . 72
8.14.4.2 m_test_count . . . 72
8.15 OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare > Class Template Reference . . . 72
8.15.1 Detailed Description . . . 73
8.15.2 Constructor & Destructor Documentation . . . 74
8.15.2.1 SwappableNodeRefStack() [1/2] . . . 74
8.15.2.2 SwappableNodeRefStack() [2/2] . . . 74
8.15.3 Member Function Documentation . . . 74
8.15.3.1 allNodePointerMatch() . . . 74
8.15.3.2 canSwap() . . . 75
8.15.3.3 height() . . . 75
8.15.3.4 incSwapLevel() . . . 75
8.15.3.5 lacksIntegrity() . . . 75
8.15.3.6 noNodePointerMatches() . . . 76
8.15.3.7 operator=() . . . 76
8.15.3.8 operator[]() [1/2] . . . 77
8.15.3.9 operator[]() [2/2] . . . 77
8.15.3.10 pop_back() . . . 78
8.15.3.11 push_back() . . . 78
8.15.3.12 resetSwapLevel() . . . 78
8.15.3.13 size_of() . . . 78
8.15.3.14 swap() . . . 79
8.15.3.15 swapLevel() . . . 79
8.15.4 Member Data Documentation . . . 79
8.15.4.1 _nodes . . . 79
8.15.4.2 _swapLevel . . . 80
8.16 TestResult::test_result Struct Reference . . . 80
8.16.1 Detailed Description . . . 80

8.16.2 Member Data Documentation . . . 80
8.16.2.1 _execTimeS . . . 80
8.16.2.2 count . . . 80
8.16.2.3 failed . . . 81
8.17 TestResult Class Reference . . . 81
8.17.1 Detailed Description . . . 82
8.17.1.1 Note on gnuplot . . . 82
8.17.1.2 Note on Using the Rate Column . . . 84
8.17.2 Constructor & Destructor Documentation . . . 84
8.17.2.1 TestResult() [1/4] . . . 84
8.17.2.2 TestResult() [2/4] . . . 84
8.17.2.3 TestResult() [3/4] . . . 84
8.17.2.4 TestResult() [4/4] . . . 84
8.17.3 Member Function Documentation . . . 85
8.17.3.1 atomicTestMeanExecTime() . . . 85
8.17.3.2 execTime() . . . 85
8.17.3.3 execTimeAdd() . . . 85
8.17.3.4 execTimeMax() . . . 85
8.17.3.5 execTimeMin() . . . 86
8.17.3.6 execTimeStdDev() . . . 86
8.17.3.7 failed() [1/2] . . . 86
8.17.3.8 failed() [2/2] . . . 86
8.17.3.9 hasExecTimeStdDev() . . . 86
8.17.3.10 name() . . . 86
8.17.3.11 numScaleValues() . . . 87
8.17.3.12 numTests() [1/2] . . . 87
8.17.3.13 numTests() [2/2] . . . 87
8.17.3.14 operator=() . . . 87
8.17.3.15 scaleValues() . . . 87
8.17.3.16 setFailed() . . . 87
8.17.3.17 testCount() . . . 88
8.17.3.18 totalTime() . . . 88
8.17.4 Member Data Documentation . . . 88
8.17.4.1 _name . . . 88
8.17.4.2 _results . . . 88
8.18 TestResultS Class Reference . . . 88
8.18.1 Detailed Description . . . 89
8.18.2 Member Typedef Documentation . . . 89
8.18.2.1 tResults . . . 89
8.18.3 Constructor & Destructor Documentation . . . 89
8.18.3.1 TestResultS() . . . 89
8.18.4 Member Function Documentation . . . 89

8.18.4.1 dump_header() . . . 90
8.18.4.2 dump_tail() . . . 90
8.18.4.3 dump_tests() . . . 90
8.18.4.4 failed() . . . 90
8.18.4.5 push_back() . . . 90
8.18.4.6 results() . . . 90
8.18.5 Member Data Documentation . . . 91
8.18.5.1 _failed . . . 91
8.18.5.2 _results . . . 91
8.19 OrderedStructs::SkipList::ValueError Class Reference . . . 91
8.19.1 Detailed Description . . . 92
8.19.2 Constructor & Destructor Documentation . . . 92
8.19.2.1 ValueError() . . . 92

9 File Documentation **93**
9.1 /Users/paulross/CLionProjects/skiplist/src/cpp/HeadNode.h File Reference . . . 93
9.1.1 Detailed Description . . . 93
9.2 /Users/paulross/CLionProjects/skiplist/src/cpp/IntegrityEnums.h File Reference . . . 94
9.2.1 Detailed Description . . . 94
9.3 /Users/paulross/CLionProjects/skiplist/src/cpp/main.cpp File Reference . . . 95
9.3.1 Function Documentation . . . 95
9.3.1.1 main() . . . 95
9.3.1.2 test_all() . . . 95
9.3.1.3 test_clock_resolution() . . . 96
9.3.1.4 test_example_introduction_A() . . . 96
9.4 /Users/paulross/CLionProjects/skiplist/src/cpp/Node.h File Reference . . . 96
9.4.1 Detailed Description . . . 97
9.4.2 Macro Definition Documentation . . . 97
9.4.2.1 nullptr . . . 97
9.5 /Users/paulross/CLionProjects/skiplist/src/cpp/NodeRefs.h File Reference . . . 97
9.6 /Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.cpp File Reference . . . 98
9.7 /Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.h File Reference . . . 98
9.7.1 Detailed Description . . . 99
9.7.2 Macro Definition Documentation . . . 99
9.7.2.1 ROLLING_MEDIAN_ERROR_CHECK . . . 100
9.8 /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.cpp File Reference . . . 100
9.9 /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h File Reference . . . 101
9.9.1 Detailed Description . . . 102
9.9.2 Macro Definition Documentation . . . 102
9.9.2.1 INCLUDE_METHODS_THAT_USE_STREAMS . . . 102
9.10 /Users/paulross/CLionProjects/skiplist/src/cpp/test/save_stream_state.h File Reference . . . 102
9.11 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.cpp File Reference . . . 103

9.11.1 Detailed Description . . . 104
9.11.2 Function Documentation . . . 104
9.11.2.1 _test_perf_insert_count_has_remove_count_multi_threads() . . . 104
9.11.2.2 _test_perf_insert_count_has_remove_count_single_thread() . . . 105
9.11.2.3 insert_count_has_remove_count() . . . 105
9.11.2.4 insert_has_remove() . . . 106
9.11.2.5 insert_has_remove_count() . . . 106
9.11.2.6 insert_value() . . . 107
9.11.2.7 test_concurrent_all() . . . 107
9.11.2.8 test_perf_insert_count_has_remove_count_multi_threads_fixed_length() . . . 108
9.11.2.9 test_perf_sim_rolling_median_multi_thread_multi_no_thread() . . . 108
9.11.2.10 test_perf_sim_rolling_median_single_thread() . . . 108
9.11.2.11 test_perf_single_thread_fixed_length() . . . 108
9.11.2.12 test_single_thread_insert() . . . 109
9.11.2.13 test_two_thread_insert_count_has_remove_count() . . . 109
9.11.2.14 test_two_thread_insert_has_remove() . . . 109
9.11.3 Variable Documentation . . . 109
9.11.3.1 SKIPLIST_FIXED_LENGTH . . . 110
9.11.3.2 SKIPLIST_MAX_THREADS . . . 110
9.12 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.h File Reference . . . 110
9.12.1 Detailed Description . . . 110
9.12.2 Function Documentation . . . 111
9.12.2.1 test_concurrent_all() . . . 111
9.13 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.cpp File Reference . . . 111
9.13.1 Detailed Description . . . 111
9.13.2 Function Documentation . . . 112
9.13.2.1 test_doc_height_trend() . . . 112
9.13.2.2 test_doc_insert() . . . 112
9.13.2.3 test_doc_insert_remove() . . . 112
9.13.2.4 test_doc_insert_remove_duplicates() . . . 113
9.13.2.5 test_doc_insert_remove_repeat() . . . 113
9.13.2.6 test_documentation_all() . . . 113
9.13.2.7 tests_doc_simple_dot() . . . 113
9.14 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.h File Reference . . . 114
9.14.1 Function Documentation . . . 114
9.14.1.1 test_documentation_all() . . . 114
9.15 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.cpp File Reference . . . 114
9.15.1 Detailed Description . . . 116
9.15.2 Function Documentation . . . 116
9.15.2.1 test_at() . . . 116
9.15.2.2 test_at_dest() . . . 116
9.15.2.3 test_at_dest_fails() . . . 117

9.15.2.4 test_at_fails() . . . 117
9.15.2.5 test_at_large() . . . 117
9.15.2.6 test_functional_all() . . . 118
9.15.2.7 test_has() . . . 118
9.15.2.8 test_has_not() . . . 118
9.15.2.9 test_index_basic_7_node() . . . 118
9.15.2.10 test_index_large() . . . 119
9.15.2.11 test_index_throws() . . . 119
9.15.2.12 test_ins_at_rem_with_srand() . . . 119
9.15.2.13 test_ins_rem_rand() . . . 120
9.15.2.14 test_insert_and_remove_same() . . . 120
9.15.2.15 test_insert_n_numbers_same() . . . 120
9.15.2.16 test_insert_nan_throws() . . . 121
9.15.2.17 test_insert_one_million() . . . 121
9.15.2.18 test_insert_remove_multiple() . . . 121
9.15.2.19 test_remove_fails() . . . 122
9.15.2.20 test_reversed_simple_insert() . . . 122
9.15.2.21 test_simple_insert() . . . 122
9.15.2.22 test_single_ins_rem_middle() . . . 123
9.15.2.23 test_single_insert_remove() . . . 123
9.15.2.24 test_very_simple_insert() . . . 123
9.16 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.h File Reference . . . 123
9.16.1 Function Documentation . . . 124
9.16.1.1 test_functional_all() . . . 124
9.17 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.cpp File Reference . . . 124
9.17.1 Detailed Description . . . 126
9.17.2 Function Documentation . . . 126
9.17.2.1 perf_1m_median_values() . . . 126
9.17.2.2 perf_1m_medians_1000_vectors() . . . 127
9.17.2.3 perf_at_in_one_million() . . . 127
9.17.2.4 perf_has_in_one_million_vary_length() . . . 127
9.17.2.5 perf_index() . . . 128
9.17.2.6 perf_index_vary_length() . . . 128
9.17.2.7 perf_large_skiplist_ins_only() . . . 128
9.17.2.8 perf_large_skiplist_ins_rem() . . . 129
9.17.2.9 perf_median_sliding_window() . . . 129
9.17.2.10 perf_roll_med_by_win_size() . . . 129
9.17.2.11 perf_roll_med_odd_index_wins() . . . 130
9.17.2.12 perf_roll_med_vector_style_even_win_length() . . . 130
9.17.2.13 perf_roll_med_vector_style_even_win_length_string() . . . 130
9.17.2.14 perf_roll_med_vector_style_odd_win_length() . . . 131
9.17.2.15 perf_simulate_real_use() . . . 131

9.17.2.16 perf_single_at_middle() . . . 131
9.17.2.17 perf_single_has_middle() . . . 132
9.17.2.18 perf_single_ins_at_rem_middle() . . . 132
9.17.2.19 perf_single_ins_rem_middle() . . . 132
9.17.2.20 perf_single_ins_rem_middle_vary_length() . . . 133
9.17.2.21 perf_single_insert_remove() . . . 133
9.17.2.22 perf_size() . . . 133
9.17.2.23 perf_size_of_1m() . . . 134
9.17.2.24 perf_size_of_char_vary_length() . . . 134
9.17.2.25 perf_size_of_double_vary_length() . . . 134
9.17.2.26 perf_skiplist() . . . 135
9.17.2.27 perf_skiplist_unfair_coin() . . . 135
9.17.2.28 perf_test_double_at_1m() . . . 135
9.17.2.29 perf_test_double_at_1m_all() . . . 135
9.17.2.30 perf_test_double_has_1m() . . . 136
9.17.2.31 perf_test_double_has_1m_all() . . . 136
9.17.2.32 perf_test_double_index_1m() . . . 136
9.17.2.33 perf_test_double_index_1m_all() . . . 136
9.17.2.34 perf_test_double_insert_remove_value() . . . 137
9.17.2.35 perf_test_double_insert_remove_value_begin() . . . 137
9.17.2.36 perf_test_double_insert_remove_value_end() . . . 137
9.17.2.37 perf_test_double_insert_remove_value_mid() . . . 137
9.17.2.38 perf_test_node_height_growth() . . . 138
9.17.2.39 perf_test_string_insert_remove_value() . . . 138
9.17.2.40 perf_test_string_insert_remove_value_begin() . . . 138
9.17.2.41 perf_test_string_insert_remove_value_end() . . . 138
9.17.2.42 perf_test_string_insert_remove_value_mid() . . . 139
9.17.2.43 test_performance_all() . . . 139
9.17.3 Variable Documentation . . . 139
9.17.3.1 GLOBAL_REPEAT_COUNT . . . 139
9.18 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.h File Reference . . . 139
9.18.1 Function Documentation . . . 139
9.18.1.1 test_performance_all() . . . 140
9.19 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.cpp File Reference . . . 140
9.19.1 Detailed Description . . . 140
9.19.2 Function Documentation . . . 141
9.19.2.1 print_result() . . . 141
9.19.3 Variable Documentation . . . 141
9.19.3.1 FUNCTION_WIDTH . . . 141
9.20 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.h File Reference . . . 141
9.20.1 Function Documentation . . . 142
9.20.1.1 print_result() . . . 142

9.20.2 Variable Documentation . . . 142
9.20.2.1 FUNCTION_WIDTH . . . 142
9.21 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.cpp File Reference . . . 142
9.21.1 Function Documentation . . . 143
9.21.1.1 perf_roll_med_odd_index() . . . 143
9.21.1.2 test_roll_med_even_mean() . . . 143
9.21.1.3 test_roll_med_even_win() . . . 144
9.21.1.4 test_roll_med_simple() . . . 144
9.21.1.5 test_rolling_median_all() . . . 144
9.22 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.h File Reference . . . 144
9.22.1 Function Documentation . . . 145
9.22.1.1 test_rolling_median_all() . . . 145
9.23 /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.cpp File Reference . . . 145
9.23.1 Macro Definition Documentation . . . 146
9.23.1.1 REGEX_SPACE_ANYTHING . . . 146
9.23.1.2 REGEX_SPACE_FLOAT . . . 146
9.23.1.3 REGEX_SPACE_INTEGER . . . 146
9.23.1.4 REGEX_SPACE_STRING_NO_SPACE . . . 146
9.23.2 Function Documentation . . . 146
9.23.2.1 count_of_unique_string() . . . 147
9.23.2.2 operator<<() [1/2] . . . 147
9.23.2.3 Note on The Output . . . 147
9.23.2.4 operator<<() [2/2] . . . 148
9.23.2.5 random_string() . . . 148
9.23.2.6 reset_count_of_unique_string() . . . 148
9.23.2.7 unique_string() . . . 148
9.23.2.8 unique_u16string() . . . 149
9.23.2.9 unique_u32string() . . . 149
9.23.2.10 unique_vector_char() . . . 149
9.23.3 Variable Documentation . . . 150
9.23.3.1 str_count . . . 150
9.23.3.2 TIME_PRECISION . . . 150
9.23.3.3 TIME_WIDTH . . . 150
9.24 /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h File Reference . . . 150
9.24.1 Function Documentation . . . 151
9.24.1.1 count_of_unique_string() . . . 151
9.24.1.2 operator<<() [1/2] . . . 151
9.24.1.3 Note on The Output . . . 151
9.24.1.4 operator<<() [2/2] . . . 152
9.24.1.5 random_string() . . . 152
9.24.1.6 reset_count_of_unique_string() . . . 152
9.24.1.7 unique_string() . . . 153

9.24.1.8 unique_u16string() . . . 153
9.24.1.9 unique_u32string() . . . 153
9.24.1.10 unique_vector_char() . . . 154
9.25 /Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.cpp File Reference . . . 154
9.25.1 Detailed Description . . . 154
9.26 /Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.h File Reference . . . 155
9.26.1 Detailed Description . . . 155
9.26.2 Macro Definition Documentation . . . 155
9.26.2.1 SKIPLIST_TRACE_CMPPYOBJECT_REF_COUNTS . . . 155
9.27 /Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.cpp File Reference . . . 156
9.27.1 Macro Definition Documentation . . . 156
9.27.1.1 GETSTATE . . . 156
9.27.2 Function Documentation . . . 157
9.27.2.1 initorderedstructs() . . . 157
9.27.2.2 long_max_value() . . . 157
9.27.2.3 long_min_value() . . . 157
9.27.2.4 seed_rand() . . . 157
9.27.2.5 toss_coin() . . . 157
9.27.3 Variable Documentation . . . 158
9.27.3.1 _state . . . 158
9.27.3.2 c_skip_list_docs . . . 158
9.27.3.3 MODULE_BUILD_DOCS . . . 158
9.27.3.4 MODULE_VERSION . . . 158
9.27.3.5 orderedstructsmodule_methods . . . 159
9.27.3.6 seed_rand_docs . . . 159
9.27.3.7 toss_coin_docs . . . 159
9.28 /Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.h File Reference . . . 159
9.28.1 Macro Definition Documentation . . . 160
9.28.1.1 ORDERED_STRUCTS_MODULE_NAME . . . 160
9.29 /Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.cpp File Reference . . . 160
9.29.1 Detailed Description . . . 161
9.29.2 Note about Python thread safety . . . 162
9.29.2.1 Non-PyObjects's . . . 162
9.29.2.2 PyObjects's . . . 162
9.29.3 Macro Definition Documentation . . . 162
9.29.3.1 INCLUDE_METHODS_THAT_USE_STREAMS . . . 162
9.29.4 Function Documentation . . . 162
9.29.4.1 at_sequence_bytes() . . . 163
9.29.4.2 at_sequence_double() . . . 163
9.29.4.3 at_sequence_long() . . . 163
9.29.4.4 at_sequence_object() . . . 163
9.29.4.5 check_index_against_size() . . . 163

9.29.4.6 decref_all_contents() . . . 164
9.29.4.7 get_size() . . . 164
9.29.4.8 has_object() . . . 164
9.29.4.9 index_object() . . . 164
9.29.4.10 remove_bytes() . . . 165
9.29.4.11 remove_double() . . . 165
9.29.4.12 remove_long() . . . 165
9.29.4.13 remove_object() . . . 165
9.29.4.14 SkipList_at() . . . 165
9.29.4.15 SkipList_at_sequence() . . . 166
9.29.4.16 SkipList_dealloc() . . . 166
9.29.4.17 SkipList_dot_file() . . . 166
9.29.4.18 SkipList_has() . . . 166
9.29.4.19 SkipList_height() . . . 166
9.29.4.20 SkipList_index() . . . 167
9.29.4.21 SkipList_init() . . . 167
9.29.4.22 SkipList_insert() . . . 167
9.29.4.23 SkipList_lacks_integrity() . . . 167
9.29.4.24 SkipList_length() . . . 168
9.29.4.25 SkipList_new() . . . 168
9.29.4.26 SkipList_node_height() . . . 168
9.29.4.27 SkipList_node_width() . . . 168
9.29.4.28 SkipList_remove() . . . 169
9.29.4.29 SkipList_size() . . . 169
9.29.4.30 SkipList_size_of() . . . 169
9.29.5 Variable Documentation . . . 169
9.29.5.1 py_skip_list_docs . . . 169
9.29.5.2 SkipList_members . . . 169
9.29.5.3 SkipList_methods . . . 170
9.29.5.4 SkipList_SequenceMethods . . . 170
9.29.5.5 SkipListType . . . 170
9.30 /Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.h File Reference . . . 170
9.30.1 Variable Documentation . . . 171
9.30.1.1 SkipListType . . . 171
9.31 /Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.cpp File Reference . . . 171
9.31.1 Function Documentation . . . 171
9.31.1.1 bytes_as_std_string() . . . 171
9.31.1.2 std_string_as_bytes() . . . 171
9.32 /Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.h File Reference . . . 172
9.32.1 Macro Definition Documentation . . . 172
9.32.1.1 ASSERT_TYPE_IN_RANGE . . . 172
9.32.2 Typedef Documentation . . . 172

9.32.2.1 TYPE_TYPE_BYTES 173
9.32.2.2 TYPE_TYPE_DOUBLE 173
9.32.2.3 TYPE_TYPE_LONG 173
9.32.2.4 TYPE_TYPE_OBJECT 173
9.32.3 Enumeration Type Documentation 173
9.32.3.1 KeyDataType 173
9.32.4 Function Documentation 174
9.32.4.1 bytes_as_std_string() 174
9.32.4.2 std_string_as_bytes() 174

Index **175**

Chapter 1

Main Page

1.1 General

This is a generic skip list implementation for any type T. There only restriction on the size of this skip list is the available memory.

A skip list is a singly linked list of ordered nodes with a series of other, coarser, lists that reference a subset of nodes in order. 'Level' is an size_t that specifies the coarseness of the linked list, level 0 is the linked list to every node.

Typically:

- The list at level 1 links (ideally) to every other node.
- The list at level 2 links (ideally) to every fourth node and so on.

In general the list at level n links (ideally) to every 2∗∗n node.

These additional lists allow rapid location, insertion and removal of nodes. These lists are created and updated in a probabilistic manner and this is achieved at node creation time by tossing a virtual coin. These lists are not explicit, they are implied by the references between Nodes at a particular level.

Skip lists are alternatives to balanced trees for operations such as a rolling median. The disadvantages of skip lists are:

- Less space efficient than balanced trees (see 'Space Complexity' below).
- performance is similar to balanced trees except finding the mid-point which is `O(log(N))` for a skip list compared with `O(1)` for a balanced tree.

The advantages claimed for skip lists are:

- The insert() and remove() logic is simpler (I do not subscribe to this).

1.2 Examples of Usage

1.2.1 C++

```
#include "SkipList.h"
OrderedStructs::SkipList::HeadNode<double> sl;
sl.insert(42.0);
sl.insert(21.0);
sl.insert(84.0);
sl.has(42.0) // true
sl.size() // 3
sl.at(1) // 42.0
```

1.2.2 Python

```
import orderedstructs
sl = orderedstructs.SkipList(float)
sl.insert(42.0)
sl.insert(21.0)
sl.insert(84.0)
sl.has(42.0) # True
sl.size() # 3
sl.at(1) # 42.0
```

1.3 Design

This skip list design has the coarser lists implemented as optional additional links between the nodes themselves. The drawing below shows a well formed skip list with a head node ('HED') linked to the ordered nodes A to H.

```
| 5 E |------------------------------------->| 4 0 |---------------------------->| NULL |
| 1 A |->| 2 C |---------->| 2 E |---------->| 2 G |---------->| 2 0 |---------->| NULL |
| 1 A |->| 1 B |->| 1 C |->| 1 D |->| 1 E |->| 1 F |->| 1 G |->| 1 H |->| 1 0 |->| NULL |
| HED |  |  A  |  |  B  |  |  C  |  |  D  |  |  E  |  |  F  |  |  G  |  |  H  |
```

Each node has a stack of values that consist of a 'width' and a reference to another node (or NULL). At the lowest level is a singly linked list and all widths are 1. At level 1 the links are (ideally) to every other node and at level 2 the links are (ideally) to every fourth node. The 'widths' at each node/level specify how many level 0 nodes the node reference skips over. The widths are used to rapidly index into the skip list starting from the highest level and working down.

To understand how the skip list is maintained, consider insertion; before inserting node 'E' the skip list would look like this:

```
| 1 A |->| 2 C |---------->| 3 G |------------------->| 2 0 |---------->| NULL |
| 1 A |->| 1 B |->| 1 C |->| 1 D |->| 1 F |->| 1 G |->| 1 H |->| 1 0 |->| NULL |
| HED |  |  A  |  |  B  |  |  C  |  |  D  |  |  F  |  |  G  |  |  H  |
```

Inserting 'E' means:

- Finding where 'E' should be inserted (after 'D').
- Creating node 'E' with a random height (heads/heads/tails so 3 high).
- Updating 'D' to refer to 'E' at level 0.
- Updating 'C' to refer to 'E' at level 1 and decreasing C's width to 2, increasing 'E' width at level 1 to 2.
- Expanding HED to level 2 with a reference to 'E' and a width of 5.
- Updating 'E' with a reference to NULL and a width of 4.

1.3.1 Recursive Search for the Node Position

The first two operations are done by a recursive search. This creates the chain HED[1], A[1], C[1], C[0], D[0] thus E will be created at level 0 and inserted after D.

1.3.2 Node Creation

Node E is created with a stack containing a single pointer to the next node F. Then a virtual coin is tossed, for each 'head' and extra NULL reference is added to the stack. If a 'tail' is thrown the stack is complete. In the example above when creating Node E we have encountered tosses of 'head', 'head', 'tail'.

1.3.3 Recursive Unwinding

The remaining operations are done as recursion unwinds:

- D[0] and C[0] update E[1] with their cumulative width (2).
- C[1] adds 1 to width (a new node is inserted) then subtracts E[1].
- Then C[1]/E[1] are swapped so that the pointers and widths are correct.
- And so on until HED is reached, in this case a new level is added and HED[2] swapped with E[2].

A similar procedure will be followed, in reverse, when removing E to restore the state of the skip list to the picture above.

1.4 Algorithms

There doesn't seem to be much literature that I could find about the algorithms used for a skip list so these have all been invented here.

In these descriptions:

- 'right' is used to mean move to a higher ordinal node.
- 'left' means to move to a lower ordinal node.
- 'up' means to move to a coarser grained list, 'top' is the highest.
- 'down' means to move to a finer grained list, 'bottom' is the level 0.

1.4.1 has(T &val) const;

This returns true/false is the skip list has the value val. Starting at the highest possible level search rightwards until a larger value is encountered, then drop down. At level 0 return true if the Node value is the supplied value. This is `O(log(N))` for well formed skip lists.

1.4.2 at(size_t index) const;

This returns the value of type T at the given index. The algorithm is similar to has(T &val) but the search moves rightwards if the width is less than the index and decrementing the index by the width.

If progress can not be made to the right, drop down a level. If the index is 0 return the node value. This is `O(log(N))` for well formed skip lists.

1.4.3 insert(T &val)

Finding the place to insert a node follows the has(T &val) algorithm to find the place in the skip list to create a new node. A duplicate value is inserted after any existing duplicate values.

- All nodes are inserted at level 0 even if the insertion point can be seen at a higher level.
- The search for an insertion location creates a recursion stack that, when unwound, updates the traversed nodes `{width, Node<T>*}` data.
- Once an insert position is found a Node is created whose height is determined by repeatedly tossing a virtual coin until a 'tails' is thrown.
- This node initially has all node references to be to itself (this), and the widths set to 1 for level 0 and 0 for the remaining levels, they will be used to sum the widths at one level down.
- On recursion ('left') each node adds its width to the new node at the level above the current level.
- On moving up a level the current node swaps its width and node pointer with the new node at that new level.

1.4.4 remove(T &val)

If there are duplicate values the last one is removed first, this is for symmetry with insert(). Essentially this is the same as insert() but once the node is found the insert() updating algorithm is reversed and the node deleted.

1.5 Code Layout

There are three classes defined in their own .h files and these are all included into the SkipList.h file.

The classes are:

`SwappableNodeRefStack`

This is simple bookkeeping class that has a vector of `[{skip_width, Node<T>*}, ...]`. This vector can be expanded or contracted at will. Both HeadNode and Node classes have one of these to manage their references.

`Node`

This represents a single value in the skip list. The height of a Node is determined at construction by tossing a virtual coin, this determines how many coarser lists this node participates in. A Node has a SwappableNodeRefStack object and a value of type T.

`HeadNode`

There is one of these per skip list and this provides the API to the entire skip list. The height of the HeadNode expands and contracts as required when Nodes are inserted or removed (it is the height of the highest Node). A HeadNode has a SwappableNodeRefStack object and an independently maintained count of the number of Node objects in the skip list.

A Node and HeadNode have specialised methods such as has(), at(), insert(), remove() that traverse the skip lis recursively.

1.5.1 Other Files of Significance

SkipList.cpp exposes the random number generator (rand()) and seeder (srand()) so that they can be accessed CPython for deterministic testing.

cSkipList.h and cSkipList.cpp contains a CPython module with a SkipList implementation for a number of builtin Python types.

IntegrityEnums.h has definitions of error codes that can be created by the skip list integrity checking functions.

1.6 Code Idioms

1.6.1 Prevent Copying

Copying operations are (mostly) prohibited for performance reasons. The only class that allows copying is struct NodeRef that contains fundamental types. All other classes declare their copying operation private and unimplemented (rather than using C++11 delete) for compatibility with older compilers.

1.6.2 Reverse Loop of Unsigned int

In a lot of the code we have to count down from some value to 0 with a size_t (an unsigned integer type) The idiom used is this:

```
for (size_t l = height(); l-- > 0;) {
    // ...
}
```

The "l-- > 0" means test l against 0 then decrement it. l will thus start at the value height() - 1 down to 0 then exit the loop.

Note

If l is declared before the loop it will have the maximum value of a size_t unless a break statement is encountered.

1.7 Roads not Travelled

Certain designs were not explored, here they are and why.

1.7.1 Key/Value Implementation

Skip lists are commonly used for key/value dictionaries. Given things like map<T> or unorderedmap<T> I see no reason why a SkipList should be used as an alternative.

1.7.2 Adversarial Users

If the user knows the behaviour of the random number generator it is possible that they can change the order of insertion to create a poor distribution of nodes which will make operations tend to O(N) rather than O(log(N)).

1.7.3 Probability != 0.5

This implementation uses a fair coin to decide the height of the node.

Some literature suggests other values such as p = 0.25 might be more efficient. Some experiments seem to show that this is the case with this implementation. Here are some results when using a vector of 1 million doubles and a sliding window of 101 where each value is inserted and removed and the cental value recovered:

```
Probability calculation        p    Time compared to p = 0.5
 rand() < RAND_MAX / 16;    0.0625   90%
 rand() < RAND_MAX / 8;     0.125    83%
 rand() < RAND_MAX / 4;     0.25     80%
 rand() < RAND_MAX / 2;     0.5     100%
 rand() > RAND_MAX / 4;     0.75    143%
 rand() > RAND_MAX / 8;     0.875   201%
```

1.7.4 Optimisation: Re-index Nodes on Complete Traversal

Todo Re-index Nodes on Complete Traversal ???

1.7.5 Optimisation: Reuse removed nodes for insert()

Todo Reuse removed nodes for insert() ???

1.7.6 Reference Counting

Some time (and particularly space) improvement could be obtained by reference counting nodes so that duplicate values could be eliminated. Since the primary use case for this skip list is for computing the rolling median of doubles the chances of duplicates are slim. For int, long and string there is a higher probability so reference counting might be implemented in the future if these types become commonly used.

1.7.7 Use and Array of <tt>{skip_width, Node<T>*}</tt> rather than a vector

Less space would be used for each Node if the SwappableNodeRefStack used a dynamically allocated array of `[{skip_width, Node<T>*}, ...]` rather than a vector.

1.8 Performance

Reference platform: Macbook Pro, 13" running OS X 10.9.5. LLVM version 6.0 targeting x86_64-apple-darwin13.↩ 4.0 Compiled with -Os (small fast).

1.8.1 Performance of at() and has()

Performance is O(log(N)) where N is the position in the skip list.

On the reference platform this tests as t = 200 log2(N) in nanoseconds for skip lists of doubles. This factor of 200 can be between 70 and 500 for the same data but different indices because of the probabilistic nature of a skip list. For example finding the mid value of 1M doubles takes 3 to 4 microseconds.

Note

On Linux RHEL5 with -O3 this is much faster with t = 12 log2(N) [main.cpp perf_at_in_one_million(), main.cpp perf_has_in_one_million()]

1.8.2 Performance of insert() and remove()

A test that inserts then removes a single value in an empty list takes 440 nanoseconds (around 2.3 million per second). This should be fast as the search space is small.

Note

Linux RHEL5 with -O3 this is 4.2 million per second. [main.cpp perf_single_insert_remove()]

A test that inserts 1M doubles into a skip list (no removal) takes 0.9 seconds (around 1.1 million per second).

Note

Linux RHEL5 with -O3 this is similar. [main.cpp perf_large_skiplist_ins_only()]

A test that inserts 1M doubles into a skip list then removes all of them takes 1.0 seconds (around 1 million per second).

Note

Linux RHEL5 with -O3 this is similar. [main.cpp perf_large_skiplist_ins_rem()]

A test that creates a skip list of 1M doubles then times how long it takes to insert and remove a value at the mid-point takes 1.03 microseconds per item (around 1 million per second).

Note

Linux RHEL5 with -O3 this is around 0.8 million per second. [main.cpp perf_single_ins_rem_middle()]

A test that creates a skip list of 1M doubles then times how long it takes to insert a value, find the value at the mid point then remove that value (using insert()/at()/remove()) takes 1.2 microseconds per item (around 0.84 million per second).

Note

Linux RHEL5 with -O3 this is around 0.7 million per second. [main.cpp perf_single_ins_at_rem_middle()]

1.8.3 Performance of a rolling median

On the reference platform a rolling median (using insert()/at()/remove()) on 1M random values takes about 0.93 seconds.

Note

Linux RHEL5 with -O3 this is about 0.7 seconds. [main.cpp perf_1m_median_values(), main.cpp perf_1m_medians_1000_vectors(), main.cpp perf_simulate_real_use()]

The window size makes little difference, a rolling median on 1m items with a window size of 1 takes 0.491 seconds, with a window size of 524288 it takes 1.03 seconds.

Note

Linux RHEL5 with -O3 this is about 0.5 seconds. [main.cpp perf_roll_med_odd_index_wins()]

1.8.4 Space Complexity

Given:

- t = sizeof(T) $\sim$ typ. 8 bytes for a double
- v = sizeof(std::vector<struct NodeRef<T>>) $\sim$ typ. 32 bytes
- p = sizeof(Node<T>$*$) $\sim$ typ. 8 bytes
- e = sizeof(struct NodeRef<T>) $\sim$ typ. 8 + p = 16 bytes

Then each node: is t + v bytes.

Linked list at level 0 is e bytes per node.

Linked list at level 1 is, typically, e / 2 bytes per node and so on.

So the totality of linked lists is about 2e bytes per node.

The total is N $*$ (t + v + 2 $*$ e) which for T as a double is typically 72 bytes per item.

In practice this has been measured on the reference platform as a bit larger at 86.0 Mb for 1024$*$1024 doubles.

```
END: SkipList Documentation
```

Chapter 2

Todo List

page Main Page

Re-index Nodes on Complete Traversal ???

Reuse removed nodes for insert() ???

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

OrderedStructs
Namespace for all the C++ ordered structures 19
OrderedStructs::RollingMedian
Namespace for the C++ Rolling Median 19
OrderedStructs::SkipList
Namespace for the C++ Slip List 25

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AcquireLock 29
cmpPyObject 30
std::exception
  OrderedStructs::SkipList::Exception 33
    OrderedStructs::SkipList::FailedComparison 36
    OrderedStructs::SkipList::IndexError 52
    OrderedStructs::SkipList::ValueError 91
ExecClock 35
OrderedStructs::SkipList::HeadNode< T, Compare > 37
OrderedStructs::SkipList::HeadNode< TYPE_TYPE_BYTES > 37
OrderedStructs::SkipList::HeadNode< TYPE_TYPE_DOUBLE > 37
OrderedStructs::SkipList::HeadNode< TYPE_TYPE_LONG > 37
OrderedStructs::SkipList::HeadNode< TYPE_TYPE_OBJECT, cmpPyObject > 37
OrderedStructs::SkipList::HeadNode< void > 37
module_state 53
OrderedStructs::SkipList::Node< T, Compare > 53
OrderedStructs::SkipList::Node< T, std::less< T > > 53
OrderedStructs::SkipList::NodeRef< T, Compare > 64
reversed< T > 65
SkipList 66
StreamFormatState 68
SubTestCount 70
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare > 72
OrderedStructs::SkipList::SwappableNodeRefStack< T, std::less< T > > 72
TestResult::test_result 80
TestResult 81
TestResultS 88

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AcquireLock
A CPython RAII wrapper around the PyThread_type_lock in a SkipList 29
cmpPyObject
A wrapper around a CPython comparison function 30
OrderedStructs::SkipList::Exception
Base exception class for all exceptions in the OrderedStructs::SkipList namespace 33
ExecClock 35
OrderedStructs::SkipList::FailedComparison
Specialised exception used for NaN detection where value != value (example NaNs) 36
OrderedStructs::SkipList::HeadNode< T, Compare >
A HeadNode is a skip list. This is the single node leading to all other content Nodes 37
OrderedStructs::SkipList::IndexError
Specialised exception case for an index out of range error 52
module_state
Wrapper of module state for Python 2 and 3 53
OrderedStructs::SkipList::Node< T, Compare >
A single node in a Skip List containing a value and references to other downstream Node objects 53
OrderedStructs::SkipList::NodeRef< T, Compare >
A PoD struct that contains a pointer to a Node and a width that represents the coarser linked list span to the next Node 64
reversed< T >
Creates a comparison function that return the inverse of `std::less` to create a decreasing Skip List 65
SkipList
Contains a CPython Skip List that is capable of holding int/float/bytes/object types 66
StreamFormatState 68
SubTestCount 70
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >
Class that represents a stack of references to other nodes 72
TestResult::test_result 80
TestResult 81
TestResultS 88
OrderedStructs::SkipList::ValueError
Specialised exception for an value error where the given value does not exist in the Skip List . 91

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

/Users/paulross/CLionProjects/skiplist/src/cpp/HeadNode.h . . . 93
/Users/paulross/CLionProjects/skiplist/src/cpp/IntegrityEnums.h . . . 94
/Users/paulross/CLionProjects/skiplist/src/cpp/main.cpp . . . 95
/Users/paulross/CLionProjects/skiplist/src/cpp/Node.h . . . 96
/Users/paulross/CLionProjects/skiplist/src/cpp/NodeRefs.h . . . 97
/Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.cpp . . . 98
/Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.h . . . 98
/Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.cpp . . . 100
/Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h . . . 101
/Users/paulross/CLionProjects/skiplist/src/cpp/test/save_stream_state.h . . . 102
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.cpp . . . 103
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.h . . . 110
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.cpp . . . 111
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.h . . . 114
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.cpp . . . 114
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.h . . . 123
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.cpp . . . 124
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.h . . . 139
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.cpp . . . 140
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.h . . . 141
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.cpp . . . 142
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.h . . . 144
/Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.cpp . . . 145
/Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h . . . 150
/Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.cpp . . . 154
/Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.h . . . 155
/Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.cpp . . . 156
/Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.h . . . 159
/Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.cpp . . . 160
/Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.h . . . 170
/Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.cpp . . . 171
/Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.h . . . 172

Chapter 7

Namespace Documentation

7.1 OrderedStructs Namespace Reference

Namespace for all the C++ ordered structures.

Namespaces

- RollingMedian

  Namespace for the C++ Rolling Median.
- SkipList

  Namespace for the C++ Slip List.

7.1.1 Detailed Description

Namespace for all the C++ ordered structures.

7.2 OrderedStructs::RollingMedian Namespace Reference

Namespace for the C++ Rolling Median.

Enumerations

- enum RollingMedianResult { ROLLING_MEDIAN_SUCCESS = 0 , ROLLING_MEDIAN_SOURCE_STRIDE , ROLLING_MEDIAN_DESTINATION_STRIDE , ROLLING_MEDIAN_WIN_LENGTH }

Functions

- size_t dest_count (size_t count, size_t win_length)
- size_t dest_size (size_t count, size_t win_length, size_t dest_stride)
- template<typename T >
  RollingMedianResult rolling_median (const std::vector< T > data, size_t win_length, std::vector< T > &result)
- template<typename T >
  RollingMedianResult rolling_median_lower_bound (const std::vector< T > data, size_t win_length, std::vector< T > &result)
- template<typename T >
  RollingMedianResult odd_index (const T *src, size_t src_stride, size_t count, size_t win_length, T *dest, size_t dest_stride)
- template<typename T >
  RollingMedianResult even_index (const T *src, size_t src_stride, size_t count, size_t win_length, T *dest, size_t dest_stride)
- template<typename T >
  RollingMedianResult even_odd_index (const T *src, size_t src_stride, size_t count, size_t win_length, T *dest, size_t dest_stride)

7.2.1 Detailed Description

Namespace for the C++ Rolling Median.

7.2.2 Enumeration Type Documentation

7.2.2.1 RollingMedianResult

`enum OrderedStructs::RollingMedian::RollingMedianResult`

Error codes.

Enumerator

ROLLING_MEDIAN_SUCCESS	
ROLLING_MEDIAN_SOURCE_STRIDE	
ROLLING_MEDIAN_DESTINATION_STRIDE	
ROLLING_MEDIAN_WIN_LENGTH	

Definition at line 53 of file RollingMedian.h.

7.2.3 Function Documentation

7.2.3.1 dest_count()

```
size_t OrderedStructs::RollingMedian::dest_count (
            size_t count,
            size_t win_length )
```

Returns the size of the destination array for a rolling median on an array of count values with a window of win_length and a destination stride.

Parameters

count	Number of input values.
win_length	Window length.

Returns

Number of destination values.

Definition at line 12 of file RollingMedian.cpp.

7.2.3.2 dest_size()

```
size_t OrderedStructs::RollingMedian::dest_size (
            size_t count,
            size_t win_length,
            size_t dest_stride )
```

Returns the size of the destination array for a rolling median on an array of count values with a window of win_length and a destination stride.

Parameters

count	Number of input values.
win_length	Window length.
dest_stride	The destination stride given a 2D array.

Returns

Size of destination array.

Definition at line 16 of file RollingMedian.cpp.

7.2.3.3 even_index()

```
template<typename T >
RollingMedianResult OrderedStructs::RollingMedian::even_index (
```

```
            const T * src,
            size_t src_stride,
            size_t count,
            size_t win_length,
            T * dest,
            size_t dest_stride )
```

Rolling median where only the even mid-index is considered. Then value is: the mean of the two centre values SkipList[(win_length - 1) / 2] / 2 + SkipList[(win_length - 1) / 2] / 2 See odd_index() for a different treatment of odd lengths.

This is valid only for types T that can be divided by two. It is up to the caller to ensure that there is enough space in dest for the results, use dest_size() for this.

The number of valid values in the result is count - win_length

Template Parameters

T	Type of the value(s).

Parameters

src	Source array of values.
src_stride	Source stride for 2D arrays.
count	Number of input values.
win_length	Window length.
dest	The destination array.
dest_stride	The destination stride given a 2D array.

Returns

The result of the Rolling Median operation as a RollingMedianResult enum.

Definition at line 247 of file RollingMedian.h.

7.2.3.4 even_odd_index()

```
template<typename T >
RollingMedianResult OrderedStructs::RollingMedian::even_odd_index (
            const T * src,
            size_t src_stride,
            size_t count,
            size_t win_length,
            T * dest,
            size_t dest_stride )
```

Rolling median where the mean of adjacent values is used when the window size is even length. This requires T / 2 to be meaningful. It is up to the caller to ensure that there is enough space in dest for the results, use dest_size() for this.

Template Parameters

T	Type of the value(s).

Parameters

src	Source array of values.
src_stride	Source stride for 2D arrays.
count	Number of input values.
win_length	Window length.
dest	The destination array.
dest_stride	The destination stride given a 2D array.

Returns

The result of the Rolling Median operation as a RollingMedianResult enum.

Definition at line 289 of file RollingMedian.h.

7.2.3.5 odd_index()

```
template<typename T >
RollingMedianResult OrderedStructs::RollingMedian::odd_index (
            const T * src,
            size_t src_stride,
            size_t count,
            size_t win_length,
            T * dest,
            size_t dest_stride )
```

Rolling median where only the odd mid-index is considered. The (win_length - 1) / 2 value is used. See even_index() for a different treatment of even lengths.

This is valid for all types T. It is up to the caller to ensure that there is enough space in dest for the results, use dest_size() for this.

The number of valid values in the result is count - win_length

Template Parameters

T	Type of the value(s).

Parameters

src	Source array of values.
src_stride	Source stride for 2D arrays.
count	Number of input values.
win_length	Window length.

Parameters

dest	The destination array.
dest_stride	The destination stride given a 2D array.

Returns

The result of the Rolling Median operation as a RollingMedianResult enum.

Definition at line 204 of file RollingMedian.h.

7.2.3.6 rolling_median()

```
template<typename T >
RollingMedianResult OrderedStructs::RollingMedian::rolling_median (
            const std::vector< T > data,
            size_t win_length,
            std::vector< T > & result )
```

Vector based rolling median. This handles both even and odd window lengths.

The length of the result is data.size() - win_length

Template Parameters

T	Data type.

Parameters

data	Data vector.
win_length	Window length.
result	Result vector.

Returns

ROLLING_MEDIAN_SUCCESS on success, non-zero on failure.

Definition at line 73 of file RollingMedian.h.

7.2.3.7 rolling_median_lower_bound()

```
template<typename T >
RollingMedianResult OrderedStructs::RollingMedian::rolling_median_lower_bound (
            const std::vector< T > data,
```

```
size_t win_length,
std::vector< T > & result )
```

Vector based rolling median. This always uses the lower bound so works correctly for odd sized window lengths. For even sized window lengths this chooses the lower value rather than averaging two values. This is useful for, say, strings that can not be averaged.

The length of the result is data.size() - win_length

Template Parameters

T	Data type.

Parameters

data	Data vector.
win_length	Window length.
result	Result vector.

Returns

ROLLING_MEDIAN_SUCCESS on success, non-zero on failure.

Definition at line 115 of file RollingMedian.h.

7.3 OrderedStructs::SkipList Namespace Reference

Namespace for the C++ Slip List.

Classes

- class HeadNode

  A HeadNode is a skip list. This is the single node leading to all other content Nodes.
- class Node

  A single node in a Skip List containing a value and references to other downstream Node objects.
- struct NodeRef

  A PoD struct that contains a pointer to a Node and a width that represents the coarser linked list span to the next Node.
- class SwappableNodeRefStack

  Class that represents a stack of references to other nodes.
- class Exception

  Base exception class for all exceptions in the OrderedStructs::SkipList namespace.
- class IndexError

  Specialised exception case for an index out of range error.
- class ValueError

  Specialised exception for an value error where the given value does not exist in the Skip List.
- class FailedComparison

  Specialised exception used for NaN detection where value != value (example NaNs).

Enumerations

- enum IntegrityCheck {
  INTEGRITY_SUCCESS = 0 , NODEREFS_WIDTH_ZERO_NOT_UNITY = 100 , NODEREFS_WIDTH_DECREASING , NODE_HEIGHT_ZERO = 200 ,
  NODE_HEIGHT_EXCEEDS_HEADNODE , NODE_NON_NULL_AFTER_NULL , NODE_SELF_REFERENCE , NODE_REFERENCES_NOT_IN_GLOBAL_SET ,
  HEADNODE_CONTAINS_NULL = 300 , HEADNODE_COUNT_MISMATCH , HEADNODE_LEVEL_WIDTHS_MISMATCH , HEADNODE_DETECTS_CYCLIC_REFERENCE ,
  HEADNODE_DETECTS_OUT_OF_ORDER }

Functions

- bool tossCoin ()
- void seedRand (unsigned seed)
- void _throw_exceeds_size (size_t index)

Variables

- std::mutex gSkipListMutex

7.3.1 Detailed Description

Namespace for the C++ Slip List.

7.3.2 Enumeration Type Documentation

7.3.2.1 IntegrityCheck

`enum OrderedStructs::SkipList::IntegrityCheck`

Various integrity codes for structures in this code.

Enumerator

INTEGRITY_SUCCESS	
NODEREFS_WIDTH_ZERO_NOT_UNITY	
NODEREFS_WIDTH_DECREASING	
NODE_HEIGHT_ZERO	
NODE_HEIGHT_EXCEEDS_HEADNODE	
NODE_NON_NULL_AFTER_NULL	
NODE_SELF_REFERENCE	
NODE_REFERENCES_NOT_IN_GLOBAL_SET	
HEADNODE_CONTAINS_NULL	
HEADNODE_COUNT_MISMATCH	
HEADNODE_LEVEL_WIDTHS_MISMATCH	
HEADNODE_DETECTS_CYCLIC_REFERENCE	
HEADNODE_DETECTS_OUT_OF_ORDER	

Definition at line 46 of file IntegrityEnums.h.

7.3.3 Function Documentation

7.3.3.1 _throw_exceeds_size()

```
void OrderedStructs::SkipList::_throw_exceeds_size (
            size_t index )
```

This throws an IndexError when the index value >= the size of Skip List. If INCLUDE_METHODS_THAT_USE_STREAMS is defined then the error will have an informative message.

Parameters

index	The out of range index.

Definition at line 73 of file SkipList.cpp.

7.3.3.2 seedRand()

```
void OrderedStructs::SkipList::seedRand (
            unsigned seed )
```

Seed the random number generator for coin tosses.

Definition at line 66 of file SkipList.cpp.

7.3.3.3 tossCoin()

```
bool OrderedStructs::SkipList::tossCoin ( )
```

Tosses a virtual coin, returns true if 'heads'.

No heads, ever:
```
return false;
```

6.25% heads:
```
return rand() < RAND_MAX / 16;
```

12.5% heads:
```
return rand() < RAND_MAX / 8;
```

25% heads:
```
return rand() < RAND_MAX / 4;
```

Fair coin:
```
return rand() < RAND_MAX / 2;
```

75% heads:
```
return rand() < RAND_MAX - RAND_MAX / 4;
```

87.5% heads:
```
return rand() < RAND_MAX - RAND_MAX / 8;
```

93.75% heads:
```
@return rand() < RAND_MAX - RAND_MAX / 16;
```

Definition at line 62 of file SkipList.cpp.

7.3.4 Variable Documentation

7.3.4.1 gSkipListMutex

```
std::mutex OrderedStructs::SkipList::gSkipListMutex
```

Mutex used in a multi-threaded environment.

Definition at line 85 of file SkipList.cpp.

Chapter 8

Class Documentation

8.1 AcquireLock Class Reference

A CPython RAII wrapper around the PyThread_type_lock in a SkipList.

Public Member Functions

- AcquireLock (SkipList *pSL)
- ~AcquireLock ()

Private Attributes

- SkipList * _pSL

8.1.1 Detailed Description

A CPython RAII wrapper around the PyThread_type_lock in a SkipList.

Definition at line 122 of file cSkipList.cpp.

8.1.2 Constructor & Destructor Documentation

8.1.2.1 AcquireLock()

```
AcquireLock::AcquireLock (
            SkipList * pSL )  [inline]
```

Constructor. Acquire the Lock.

Parameters

pSL	The Python SkipList.

Definition at line 130 of file cSkipList.cpp.

8.1.2.2 ~AcquireLock()

```
AcquireLock::~AcquireLock ( ) [inline]
```

Destructor. Release the Lock.

Definition at line 141 of file cSkipList.cpp.

8.1.3 Member Data Documentation

8.1.3.1 _pSL

```
SkipList* AcquireLock::_pSL [private]
```

Definition at line 148 of file cSkipList.cpp.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.cpp

8.2 cmpPyObject Class Reference

A wrapper around a CPython comparison function.

```
#include <cmpPyObject.h>
```

Public Member Functions

- cmpPyObject (PyObject *cmp=NULL)
- bool operator() (PyObject *a, PyObject *b) const
- ~cmpPyObject ()

  Destructor.
- cmpPyObject (const cmpPyObject &rhs)

  Copying needs to manage the refcount of the cmp_func.

Private Member Functions

- cmpPyObject & operator= (const cmpPyObject &rhs)
- void copy_cmpPyObject (const cmpPyObject &rhs)

Private Attributes

- PyObject * cmp_func

  The CPython comparison function.

8.2.1 Detailed Description

A wrapper around a CPython comparison function.

Definition at line 61 of file cmpPyObject.h.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 cmpPyObject() [1/2]

```
cmpPyObject::cmpPyObject (
            PyObject * cmp = NULL )  [inline]
```

Constructor.

Parameters

cmp	The CPython comparison function.

Definition at line 68 of file cmpPyObject.h.

8.2.2.2 ~cmpPyObject()

```
cmpPyObject::~cmpPyObject ( )  [inline]
```

Destructor.

Definition at line 87 of file cmpPyObject.h.

8.2.2.3 cmpPyObject() [2/2]

```
cmpPyObject::cmpPyObject (
            const cmpPyObject & rhs )  [inline]
```

Copying needs to manage the refcount of the cmp_func.

Definition at line 104 of file cmpPyObject.h.

8.2.3 Member Function Documentation

8.2.3.1 copy_cmpPyObject()

```
void cmpPyObject::copy_cmpPyObject (
            const cmpPyObject & rhs )  [inline], [private]
```

Make a copy managing the reference count.

Parameters

rhs	

Definition at line 122 of file cmpPyObject.h.

8.2.3.2 operator()()

```
bool cmpPyObject::operator() (
            PyObject * a,
            PyObject * b ) const
```

Compare two CPython objects.

This calls PyObject_CallFunctionObjArgs() if the comparison function is non-NULL.

This calls PyObject_RichCompareBool() if the comparison function is NULL.

Parameters

a	
b	

Returns

true if the objects compare equal.

Definition at line 52 of file cmpPyObject.cpp.

8.2.3.3 operator=()

```
cmpPyObject& cmpPyObject::operator= (
            const cmpPyObject & rhs )  [private]
```

Not implemented.

8.2.4 Member Data Documentation

8.2.4.1 cmp_func

```
PyObject* cmpPyObject::cmp_func  [private]
```

The CPython comparison function.

Definition at line 152 of file cmpPyObject.h.

The documentation for this class was generated from the following files:

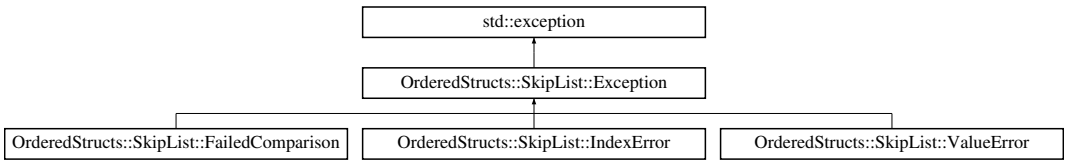
- /Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.h
- /Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.cpp

8.3 OrderedStructs::SkipList::Exception Class Reference

Base exception class for all exceptions in the OrderedStructs::SkipList namespace.

```
#include <SkipList.h>
```

Inheritance diagram for OrderedStructs::SkipList::Exception:



Public Member Functions

- Exception (const std::string &in_msg)
- const std::string & message () const
- virtual ~Exception () throw ()

Protected Attributes

- std::string msg

8.3.1 Detailed Description

Base exception class for all exceptions in the OrderedStructs::SkipList namespace.

Definition at line 489 of file SkipList.h.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 Exception()

```
OrderedStructs::SkipList::Exception::Exception (
            const std::string & in_msg )  [inline], [explicit]
```

Definition at line 491 of file SkipList.h.

8.3.2.2 ~Exception()

```
virtual OrderedStructs::SkipList::Exception::~Exception ( ) throw ( )   [inline], [virtual]
```

Definition at line 495 of file SkipList.h.

8.3.3 Member Function Documentation

8.3.3.1 message()

```
const std::string& OrderedStructs::SkipList::Exception::message ( ) const  [inline]
```

Definition at line 493 of file SkipList.h.

8.3.4 Member Data Documentation

8.3.4.1 msg

```
std::string OrderedStructs::SkipList::Exception::msg  [protected]
```

Definition at line 498 of file SkipList.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h

8.4 ExecClock Class Reference

```
#include <TestFramework.h>
```

Public Types

- typedef std::chrono::duration< double > tHiResDouble

Public Member Functions

- ExecClock ()
- double seconds ()

Private Attributes

- std::chrono::time_point< std::chrono::high_resolution_clock > _start

8.4.1 Detailed Description

Definition at line 15 of file TestFramework.h.

8.4.2 Member Typedef Documentation

8.4.2.1 tHiResDouble

```
typedef std::chrono::duration<double> ExecClock::tHiResDouble
```

Definition at line 17 of file TestFramework.h.

8.4.3 Constructor & Destructor Documentation

8.4.3.1 ExecClock()

```
ExecClock::ExecClock ( ) [inline]
```

Definition at line 18 of file TestFramework.h.

8.4.4 Member Function Documentation

8.4.4.1 seconds()

```
double ExecClock::seconds ( ) [inline]
```

Definition at line 20 of file TestFramework.h.

8.4.5 Member Data Documentation

8.4.5.1 _start

```
std::chrono::time_point<std::chrono::high_resolution_clock> ExecClock::_start [private]
```

Definition at line 26 of file TestFramework.h.

The documentation for this class was generated from the following file:

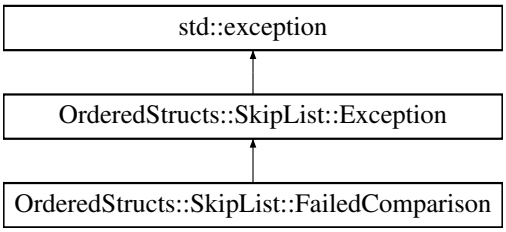
- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h

8.5 OrderedStructs::SkipList::FailedComparison Class Reference

Specialised exception used for NaN detection where value != value (example NaNs).

```
#include <SkipList.h>
```

Inheritance diagram for OrderedStructs::SkipList::FailedComparison:



Public Member Functions

- FailedComparison (const std::string &in_msg)

Additional Inherited Members

8.5.1 Detailed Description

Specialised exception used for NaN detection where value != value (example NaNs).

Definition at line 518 of file SkipList.h.

8.5.2 Constructor & Destructor Documentation

8.5.2.1 FailedComparison()

```
OrderedStructs::SkipList::FailedComparison::FailedComparison (
            const std::string & in_msg )  [inline], [explicit]
```

Definition at line 520 of file SkipList.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h

8.6 OrderedStructs::SkipList::HeadNode< T, Compare > Class Template Reference

A HeadNode is a skip list. This is the single node leading to all other content Nodes.

```
#include <HeadNode.h>
```

Public Member Functions

- HeadNode (Compare cmp=Compare())
- bool has (const T &value) const
- const T & at (size_t index) const
- void at (size_t index, size_t count, std::vector< T > &dest) const
- size_t index (const T &value) const
- size_t size () const
- void insert (const T &value)
- T remove (const T &value)
- size_t height () const
- size_t height (size_t idx) const
- size_t width (size_t idx, size_t level) const
- void dotFile (std::ostream &os) const
- void dotFileFinalise (std::ostream &os) const
- IntegrityCheck lacksIntegrity () const
- size_t size_of () const
- virtual ~HeadNode ()

Protected Member Functions

- void _adjRemoveRefs (size_t level, Node< T, Compare > *pNode)
- const Node< T, Compare > * _nodeAt (size_t idx) const
- void _throwValueErrorNotFound (const T &value) const
- void _throwIfValueDoesNotCompare (const T &value) const
- IntegrityCheck _lacksIntegrityCyclicReferences () const
- IntegrityCheck _lacksIntegrityWidthAccumulation () const
- IntegrityCheck _lacksIntegrityNodeReferencesNotInList () const
- IntegrityCheck _lacksIntegrityOrder () const

Protected Attributes

- size_t _count

  Number of nodes in the list.
- SwappableNodeRefStack< T, Compare > _nodeRefs

  My node references, the size of this is the largest height in the list.
- Compare _compare

  Comparison function.
- size_t _dot_file_subgraph

  Used to count how many sub-graphs have been plotted.

Private Member Functions

- HeadNode (const HeadNode &that)

  Prevent cctor and operator=.
- HeadNode & operator= (const HeadNode &that) const

8.6.1 Detailed Description

template<typename T, typename Compare = std::less<T>>
class OrderedStructs::SkipList::HeadNode< T, Compare >

A HeadNode is a skip list. This is the single node leading to all other content Nodes.

HeadNode

Example:

```
OrderedStructs::SkipList::HeadNode<double> sl;
for (int i = 0; i < 100; ++i) {
    sl.insert(i * 22.0 / 7.0);
}
sl.size(); // 100
sl.at(50); // Value of 50 pi
sl.remove(sl.at(50)); // Remove 50 pi
```

Created by Paul Ross on 03/12/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Definition at line 80 of file HeadNode.h.

8.6.2 Constructor & Destructor Documentation

8.6.2.1 HeadNode() [1/2]

```
template<typename T , typename Compare = std::less<T>>
OrderedStructs::SkipList::HeadNode< T, Compare >::HeadNode (
            Compare cmp = Compare() )  [inline]
```

Constructor for and Empty Skip List.

Parameters

cmp	The comparison function for comparing Node values.

Definition at line 87 of file HeadNode.h.

8.6.2.2 ~HeadNode()

```
template<typename T , typename Compare >
OrderedStructs::SkipList::HeadNode< T, Compare >::~HeadNode  [virtual]
```

Destructor. This deletes all Nodes.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Definition at line 809 of file HeadNode.h.

8.6.2.3 HeadNode() [2/2]

```
template<typename T , typename Compare = std::less<T>>
OrderedStructs::SkipList::HeadNode< T, Compare >::HeadNode (
            const HeadNode< T, Compare > & that )  [private]
```

Prevent cctor and operator=.

8.6.3 Member Function Documentation

8.6.3.1 _adjRemoveRefs()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::_adjRemoveRefs (
            size_t level,
            Node< T, Compare > * pNode )  [protected]
```

Adjust references >= level for removal of the node pNode.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

level	Current level.
pNode	Node to swap references with.

Definition at line 474 of file HeadNode.h.

8.6.3.2 _lacksIntegrityCyclicReferences()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::HeadNode< T, Compare >::_lacksIntegrityCyclicReferences
[protected]
```

This tests that at every level >= 0 the sequence of node pointers at that level does not contain a cyclic reference.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

An IntegrityCheck enum.

Definition at line 594 of file HeadNode.h.

8.6.3.3 _lacksIntegrityNodeReferencesNotInList()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::HeadNode< T, Compare >::_lacksIntegrityNodeReferences↩
NotInList [protected]
```

This tests the integrity of each Node.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

An IntegrityCheck enum.

Definition at line 661 of file HeadNode.h.

8.6.3.4 _lacksIntegrityOrder()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::HeadNode< T, Compare >::_lacksIntegrityOrder [protected]
```

Integrity check. Traverse the lowest level and check that the ordering is correct according to the compare function. The HeadNode checks that the Node(s) have correctly applied the compare function.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

An IntegrityCheck enum.

Definition at line 699 of file HeadNode.h.

8.6.3.5 _lacksIntegrityWidthAccumulation()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::HeadNode< T, Compare >::_lacksIntegrityWidthAccumulation
[protected]
```

This tests that at every level > 0 the node to node width is the same as the accumulated node to node widths at level - 1.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

An IntegrityCheck enum.

Definition at line 624 of file HeadNode.h.

8.6.3.6 _nodeAt()

```
template<typename T , typename Compare >
const Node< T, Compare > * OrderedStructs::SkipList::HeadNode< T, Compare >::_nodeAt (
            size_t idx ) const  [protected]
```

Find the Node at the given index. Will throw an IndexError if the index is out of range.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

idx	The index.

Returns

The Node.

Definition at line 362 of file HeadNode.h.

8.6.3.7 _throwIfValueDoesNotCompare()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::_throwIfValueDoesNotCompare (
            const T & value ) const  [protected]
```

Checks that the value == value. This will throw a FailedComparison if that is not the case, for example NaN.

Note

The Node class is (should be) not directly accessible by the user so we can just assert(value == value) in Node.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

value	

Definition at line 578 of file HeadNode.h.

8.6.3.8 _throwValueErrorNotFound()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::_throwValueErrorNotFound (
            const T & value ) const  [protected]
```

Throw a ValueError in a consistent fashion.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

value	The value to put into the ValueError.

Definition at line 555 of file HeadNode.h.

8.6.3.9 at() [1/2]

```
template<typename T , typename Compare >
const T & OrderedStructs::SkipList::HeadNode< T, Compare >::at (
            size_t index ) const
```

Returns the value at a particular index. Will throw an OrderedStructs::SkipList::IndexError if index out of range.

If SKIPLIST_THREAD_SUPPORT is defined this will block.

See _throw_exceeds_size() that does the throw.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

index	The index.

Returns

The value at that index.

Definition at line 213 of file HeadNode.h.

8.6.3.10 at() [2/2]

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::at (
            size_t index,
            size_t count,
            std::vector< T > & dest ) const
```

Find the count number of value starting at index and write them to dest.

Will throw a OrderedStructs::SkipList::IndexError if any index out of range.

This is useful for rolling median on even length lists where the caller might want to implement the mean of two values.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

index	The index.
count	The number of values to retrieve.
dest	The vector of values

Definition at line 237 of file HeadNode.h.

8.6.3.11 dotFile()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::dotFile (
            std::ostream & os ) const
```

Create a DOT file of the internal representation.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

os	Where to write the DOT file.

Definition at line 839 of file HeadNode.h.

8.6.3.12 dotFileFinalise()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::dotFileFinalise (
            std::ostream & os ) const
```

Finalise the DOT file of the internal representation.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

os	Where to write the DOT file.

Definition at line 917 of file HeadNode.h.

8.6.3.13 has()

```
template<typename T , typename Compare >
bool OrderedStructs::SkipList::HeadNode< T, Compare >::has (
            const T & value ) const
```

Returns true if the value is present in the skip list.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

value	Value to check if it is in the Skip List.

Returns

true if in the Skip List.

Definition at line 185 of file HeadNode.h.

8.6.3.14 height() [1/2]

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::height
```

Definition at line 300 of file HeadNode.h.

8.6.3.15 height() [2/2]

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::height (
            size_t idx ) const
```

Return the number of linked lists that the node at index has.

Will throw a OrderedStructs::SkipList::IndexError if the index out of range.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

idx	The index of the Skip List node.

Returns

The number of linked lists that the node at the index has.

Definition at line 319 of file HeadNode.h.

8.6.3.16 index()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::index (
            const T & value ) const
```

Computes index of the first occurrence of a value Will throw a OrderedStructs::SkipList::ValueError if the value does not exist in the skip list Will throw a OrderedStructs::SkipList::FailedComparison if the value is not comparable.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

value	The value to search for.

Returns

Definition at line 268 of file HeadNode.h.

8.6.3.17 insert()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::HeadNode< T, Compare >::insert (
            const T & value )
```

Insert a value.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

value	

Definition at line 390 of file HeadNode.h.

8.6.3.18 lacksIntegrity()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::HeadNode< T, Compare >::lacksIntegrity
```

Full integrity check. This calls the other integrity check functions.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

An IntegrityCheck enum.

Definition at line 725 of file HeadNode.h.

8.6.3.19 operator=()

```
template<typename T , typename Compare = std::less<T>>
HeadNode& OrderedStructs::SkipList::HeadNode< T, Compare >::operator= (
            const HeadNode< T, Compare > & that ) const  [private]
```

8.6.3.20 remove()

```
template<typename T , typename Compare >
T OrderedStructs::SkipList::HeadNode< T, Compare >::remove (
            const T & value )
```

Remove a Node with a value. May throw a ValueError if the value is not found.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

value	The value in the Node to remove.

Returns

The value removed.

Definition at line 515 of file HeadNode.h.

8.6.3.21 size()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::size
```

Return the number of values in the Skip List.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

The number of values in the Skip List.

Definition at line 295 of file HeadNode.h.

8.6.3.22 size_of()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::size_of
```

Returns an estimate of the memory usage of an instance.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Returns

The size of the memory estimate.

Definition at line 784 of file HeadNode.h.

8.6.3.23 width()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::width (
```

```
size_t idx,
size_t level ) const
```

The skip width of the Node at index has at the given level. Will throw an IndexError if the index is out of range.

Template Parameters

T	Type of the values in the Skip List.
Compare	Compare function.

Parameters

idx	The index.
level	The level.

Returns

Width of Node.

Definition at line 339 of file HeadNode.h.

8.6.4 Member Data Documentation

8.6.4.1 _compare

```
template<typename T , typename Compare = std::less<T>>
Compare OrderedStructs::SkipList::HeadNode< T, Compare >::_compare  [protected]
```

Comparison function.

Definition at line 159 of file HeadNode.h.

8.6.4.2 _count

```
template<typename T , typename Compare = std::less<T>>
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::_count  [protected]
```

Number of nodes in the list.

Definition at line 155 of file HeadNode.h.

8.6.4.3 _dot_file_subgraph

```
template<typename T , typename Compare = std::less<T>>
size_t OrderedStructs::SkipList::HeadNode< T, Compare >::_dot_file_subgraph  [mutable], [protected]
```

Used to count how many sub-graphs have been plotted.

Definition at line 162 of file HeadNode.h.

8.6.4.4 _nodeRefs

```
template<typename T , typename Compare = std::less<T>>
SwappableNodeRefStack<T, Compare> OrderedStructs::SkipList::HeadNode< T, Compare >::_nodeRefs
[protected]
```

My node references, the size of this is the largest height in the list.

Definition at line 157 of file HeadNode.h.

The documentation for this class was generated from the following file:

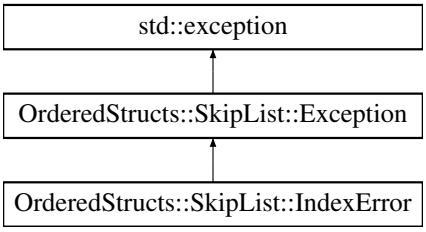
- /Users/paulross/CLionProjects/skiplist/src/cpp/HeadNode.h

8.7 OrderedStructs::SkipList::IndexError Class Reference

Specialised exception case for an index out of range error.

```
#include <SkipList.h>
```

Inheritance diagram for OrderedStructs::SkipList::IndexError:



Public Member Functions

- IndexError (const std::string &in_msg)

Additional Inherited Members

8.7.1 Detailed Description

Specialised exception case for an index out of range error.

Definition at line 504 of file SkipList.h.

8.7.2 Constructor & Destructor Documentation

8.7.2.1 IndexError()

```
OrderedStructs::SkipList::IndexError::IndexError (
            const std::string & in_msg )  [inline], [explicit]
```

Definition at line 506 of file SkipList.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h

8.8 module_state Struct Reference

Wrapper of module state for Python 2 and 3.

Public Attributes

- PyObject * error

8.8.1 Detailed Description

Wrapper of module state for Python 2 and 3.

Definition at line 94 of file cOrderedStructs.cpp.

8.8.2 Member Data Documentation

8.8.2.1 error

```
PyObject* module_state::error
```

Definition at line 95 of file cOrderedStructs.cpp.

The documentation for this struct was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.cpp

8.9 OrderedStructs::SkipList::Node< T, Compare > Class Template Reference

A single node in a Skip List containing a value and references to other downstream Node objects.

```
#include <Node.h>
```

Public Member Functions

- Node (const T &value, Compare _cmp)
- const T & value () const

  Returns the node value.
- bool has (const T &value) const
- const Node< T, Compare > * at (size_t idx) const
- bool index (const T &value, size_t &idx, size_t level) const
- size_t height () const

  Number of linked lists that this node engages in, minimum 1.
- const Node< T, Compare > * next () const
- size_t width (size_t level) const
- const Node< T, Compare > * pNode (size_t level) const
- SwappableNodeRefStack< T, Compare > & nodeRefs ()

  Get a reference to the node references.
- const SwappableNodeRefStack< T, Compare > & nodeRefs () const

  Get a reference to the node references.
- Node< T, Compare > * insert (const T &value)
- Node< T, Compare > * remove (size_t call_level, const T &value)
- size_t size_of () const
- void dotFile (std::ostream &os, size_t suffix=0) const
- void writeNode (std::ostream &os, size_t suffix=0) const
- IntegrityCheck lacksIntegrity (size_t headnode_height) const
- IntegrityCheck lacksIntegrityRefsInSet (const std::set< const Node< T, Compare > * > &nodeSet) const

Protected Member Functions

- Node< T, Compare > * _adjRemoveRefs (size_t level, Node< T, Compare > *pNode)

Protected Attributes

- T _value
- SwappableNodeRefStack< T, Compare > _nodeRefs
- Compare _compare

Private Member Functions

- Node (const Node &that)
- Node & operator= (const Node &that) const

8.9.1 Detailed Description

template<typename T, typename Compare>
class OrderedStructs::SkipList::Node< T, Compare >

A single node in a Skip List containing a value and references to other downstream Node objects.

Forward reference.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Definition at line 60 of file Node.h.

8.9.2 Constructor & Destructor Documentation

8.9.2.1 Node() [1/2]

```
template<typename T , typename Compare >
OrderedStructs::SkipList::Node< T, Compare >::Node (
            const T & value,
            Compare _cmp )
```

Constructor. This also creates a SwappableNodeRefStack of random height by tossing a virtual coin.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

value	The value of the Node.
_cmp	The comparison function.

Definition at line 132 of file Node.h.

8.9.2.2 Node() [2/2]

```
template<typename T , typename Compare >
OrderedStructs::SkipList::Node< T, Compare >::Node (
            const Node< T, Compare > & that )  [private]
```

8.9.3 Member Function Documentation

8.9.3.1 _adjRemoveRefs()

```
template<typename T , typename Compare >
Node< T, Compare > * OrderedStructs::SkipList::Node< T, Compare >::_adjRemoveRefs (
            size_t level,
            Node< T, Compare > * pNode )  [protected]
```

Adjust the Node references.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

level	The level of the caller's node.
pNode	The Node to swap references with.

Returns

The Node with swapped references.

Definition at line 387 of file Node.h.

8.9.3.2 at()

```
template<typename T , typename Compare >
const Node< T, Compare > * OrderedStructs::SkipList::Node< T, Compare >::at (
            size_t idx ) const
```

Return a pointer to the n'th node. Start (or continue) from the highest level, drop down a level if not found. Return nullptr if not found at level 0.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

idx	The index from hereon. If zero return this.

Returns

Pointer to the Node or nullptr.

Definition at line 175 of file Node.h.

8.9.3.3 dotFile()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::Node< T, Compare >::dotFile (
            std::ostream & os,
            size_t suffix = 0 ) const
```

Writes out a fragment of a DOT file representing this Node.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

os	Wheere to write.
suffix	The node number.

Definition at line 585 of file Node.h.

8.9.3.4 has()

```
template<typename T , typename Compare >
bool OrderedStructs::SkipList::Node< T, Compare >::has (
            const T & value ) const
```

Returns true if the value is present in the skip list from this node onwards.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

value	The value to look for.

Returns

true if the value is present in the skip list from this node onwards.

Definition at line 148 of file Node.h.

8.9.3.5 height()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::Node< T, Compare >::height ( ) const  [inline]
```

Number of linked lists that this node engages in, minimum 1.

Definition at line 75 of file Node.h.

8.9.3.6 index()

```
template<typename T , typename Compare >
bool OrderedStructs::SkipList::Node< T, Compare >::index (
            const T & value,
            size_t & idx,
            size_t level ) const
```

Computes index of the first occurrence of a value.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

value	The value to find.
idx	The current index, this will be updated.
level	The current level to search from.

Returns

true if found, false otherwise.

Definition at line 199 of file Node.h.

8.9.3.7 insert()

```
template<typename T , typename Compare >
Node< T, Compare > * OrderedStructs::SkipList::Node< T, Compare >::insert (
            const T & value )
```

Insert a new node with a value. A duplicate value is inserted *after* the last same value. This ensures order stability as it mirrors remove().

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

value	The value of the Node to insert.

Returns

Pointer to the new Node or nullptr on failure.

Definition at line 286 of file Node.h.

8.9.3.8 lacksIntegrity()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::Node< T, Compare >::lacksIntegrity (
            size_t headnode_height ) const
```

This checks the internal concistency of a Node. It returns 0 if succesful, non-zero on error. The tests are:

- Height must be >= 1
- Height must not exceed HeadNode height.
- NULL pointer must not have a non-NULL above them.
- Node pointers must not be self-referential.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

headnode_height	Height of HeadNode.

Returns

An IntegrityCheck enum.

Definition at line 490 of file Node.h.

8.9.3.9 lacksIntegrityRefsInSet()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::Node< T, Compare >::lacksIntegrityRefsInSet (
            const std::set< const Node< T, Compare > * > & nodeSet ) const
```

Checks that this Node is in the set held by the HeadNode.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

nodeSet	Set of Nodes held by the HeadNode.

Returns

An IntegrityCheck enum.

Definition at line 531 of file Node.h.

8.9.3.10 next()

```
template<typename T , typename Compare >
const Node< T, Compare > * OrderedStructs::SkipList::Node< T, Compare >::next
```

Return the pointer to the next node at level 0.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Returns

The next node at level 0.

Definition at line 242 of file Node.h.

8.9.3.11 nodeRefs() [1/2]

```
template<typename T , typename Compare >
SwappableNodeRefStack<T, Compare>& OrderedStructs::SkipList::Node< T, Compare >::nodeRefs ( )
[inline]
```

Get a reference to the node references.

Definition at line 86 of file Node.h.

8.9.3.12 nodeRefs() [2/2]

```
template<typename T , typename Compare >
const SwappableNodeRefStack<T, Compare>& OrderedStructs::SkipList::Node< T, Compare >::node↩
Refs ( ) const  [inline]
```

Get a reference to the node references.

Definition at line 88 of file Node.h.

8.9.3.13 operator=()

```
template<typename T , typename Compare >
Node& OrderedStructs::SkipList::Node< T, Compare >::operator= (
            const Node< T, Compare > & that ) const  [private]
```

8.9.3.14 pNode()

```
template<typename T , typename Compare >
const Node< T, Compare > * OrderedStructs::SkipList::Node< T, Compare >::pNode (
            size_t level ) const
```

Return the node pointer at given level, only used for HeadNode integrity checks.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

level	The requested level.

Returns

The Node.

Definition at line 270 of file Node.h.

8.9.3.15 remove()

```
template<typename T , typename Compare >
Node< T, Compare > * OrderedStructs::SkipList::Node< T, Compare >::remove (
            size_t call_level,
            const T & value )
```

Remove a Node with the given value to be removed. The return value must be deleted, the other Nodes have been adjusted as required. Where there are duplicate values then the last one is removed. This ensures order stability as it mirrors insert().

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

call_level	Level the caller Node is at.
value	Value of the detached Node to remove.

Returns

A pointer to the Node to be free'd or nullptr on failure.

Definition at line 430 of file Node.h.

8.9.3.16 size_of()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::Node< T, Compare >::size_of
```

Returns an estimate of the memory usage of an instance. This does not include the size of any dynamically allocated content in the value<T> which can be significant in the case of, say, strings.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Returns

The memory estimate of this Node.

Definition at line 552 of file Node.h.

8.9.3.17 value()

```
template<typename T , typename Compare >
const T& OrderedStructs::SkipList::Node< T, Compare >::value ( ) const  [inline]
```

Returns the node value.

Definition at line 66 of file Node.h.

8.9.3.18 width()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::Node< T, Compare >::width (
            size_t level ) const
```

Return the width at given level.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

level	The requested level.

Returns

The width.

Definition at line 256 of file Node.h.

8.9.3.19 writeNode()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::Node< T, Compare >::writeNode (
            std::ostream & os,
            size_t suffix = 0 ) const
```

Writes out this Node address.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

os	Where to write.
suffix	The suffix (node number).

Definition at line 570 of file Node.h.

8.9.4 Member Data Documentation

8.9.4.1 _compare

```
template<typename T , typename Compare >
Compare OrderedStructs::SkipList::Node< T, Compare >::_compare  [protected]
```

Definition at line 112 of file Node.h.

8.9.4.2 _nodeRefs

```
template<typename T , typename Compare >
SwappableNodeRefStack<T, Compare> OrderedStructs::SkipList::Node< T, Compare >::_nodeRefs
[protected]
```

Definition at line 110 of file Node.h.

8.9.4.3 _value

```
template<typename T , typename Compare >
T OrderedStructs::SkipList::Node< T, Compare >::_value  [protected]
```

Definition at line 109 of file Node.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/Node.h

8.10 OrderedStructs::SkipList::NodeRef< T, Compare > Struct Template Reference

A PoD struct that contains a pointer to a Node and a width that represents the coarser linked list span to the next Node.

```
#include <NodeRefs.h>
```

Public Attributes

- Node< T, Compare > * pNode
- size_t width

8.10.1 Detailed Description

template<typename T, typename Compare = std::less<T>>
struct OrderedStructs::SkipList::NodeRef< T, Compare >

A PoD struct that contains a pointer to a Node and a width that represents the coarser linked list span to the next Node.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Definition at line 29 of file NodeRefs.h.

8.10.2 Member Data Documentation

8.10.2.1 pNode

```
template<typename T , typename Compare = std::less<T>>
Node<T, Compare>* OrderedStructs::SkipList::NodeRef< T, Compare >::pNode
```

Definition at line 30 of file NodeRefs.h.

8.10.2.2 width

```
template<typename T , typename Compare = std::less<T>>
size_t OrderedStructs::SkipList::NodeRef< T, Compare >::width
```

Definition at line 31 of file NodeRefs.h.

The documentation for this struct was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/NodeRefs.h

8.11 reversed< T > Struct Template Reference

Creates a comparison function that return the inverse of `std::less` to create a decreasing Skip List.

Public Member Functions

- bool operator() (const T &a, const T &b) const

8.11.1 Detailed Description

template<typename T>
struct reversed< T >

Creates a comparison function that return the inverse of `std::less` to create a decreasing Skip List.

Definition at line 676 of file test_functional.cpp.

8.11.2 Member Function Documentation

8.11.2.1 operator()()

```
template<typename T >
bool reversed< T >::operator() (
            const T & a,
            const T & b ) const  [inline]
```

Definition at line 677 of file test_functional.cpp.

The documentation for this struct was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.cpp

8.12 SkipList Struct Reference

Contains a CPython Skip List that is capable of holding int/float/bytes/object types.

Public Attributes

- PyObject_HEAD enum KeyDataType _data_type
- union {
  OrderedStructs::SkipList::HeadNode< void > * pSl_void
  OrderedStructs::SkipList::HeadNode< TYPE_TYPE_LONG > * pSl_long
  OrderedStructs::SkipList::HeadNode< TYPE_TYPE_DOUBLE > * pSl_double
  OrderedStructs::SkipList::HeadNode< TYPE_TYPE_BYTES > * pSl_bytes
  OrderedStructs::SkipList::HeadNode< TYPE_TYPE_OBJECT, cmpPyObject > * pSl_object
  };
- PyThread_type_lock lock

8.12.1 Detailed Description

Contains a CPython Skip List that is capable of holding int/float/bytes/object types.

This uses an internal RTTI to administer the type system.

Definition at line 95 of file cSkipList.cpp.

8.12.2 Member Data Documentation

8.12.2.1

```
union { ...  }
```

8.12.2.2 _data_type

```
PyObject_HEAD enum KeyDataType SkipList::_data_type
```

The data type of this Skip List. Set on construction of this struct.

Definition at line 96 of file cSkipList.cpp.

8.12.2.3 lock

```
PyThread_type_lock SkipList::lock
```

Using Python thread locking.

Definition at line 115 of file cSkipList.cpp.

8.12.2.4 pSl_bytes

```
OrderedStructs::SkipList::HeadNode<TYPE_TYPE_BYTES>* SkipList::pSl_bytes
```

This Skip List is handling C++ std::string (Python bytes type)

Definition at line 109 of file cSkipList.cpp.

8.12.2.5 pSl_double

```
OrderedStructs::SkipList::HeadNode<TYPE_TYPE_DOUBLE>* SkipList::pSl_double
```

This Skip List is handling C++ doubles (Python float type)

Definition at line 107 of file cSkipList.cpp.

8.12.2.6 pSl_long

```
OrderedStructs::SkipList::HeadNode<TYPE_TYPE_LONG>* SkipList::pSl_long
```

This Skip List is handling C++ long integers (Python int type)

Definition at line 105 of file cSkipList.cpp.

8.12.2.7 pSl_object

```
OrderedStructs::SkipList::HeadNode<TYPE_TYPE_OBJECT, cmpPyObject>* SkipList::pSl_object
```

This Skip List is handling C++ PyObject* (Python object type)

Definition at line 111 of file cSkipList.cpp.

8.12.2.8 pSl_void

```
OrderedStructs::SkipList::HeadNode<void>* SkipList::pSl_void
```

NULL/non-NULL pointer, not to be de-referrenced.

Definition at line 103 of file cSkipList.cpp.

The documentation for this struct was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.cpp

8.13 StreamFormatState Class Reference

```
#include <save_stream_state.h>
```

Public Member Functions

- StreamFormatState (std::basic_ios< char > &stream)
- ~StreamFormatState ()

Private Attributes

- std::basic_ios< char > * _pStream
- std::ios_base::fmtflags _ff
- char _fill

8.13.1 Detailed Description

A class that saves the formatting state of a stream and restores it on destruction.

Usage:

```
{
    StreamFormatState stream_state(os); // Stream state captured.
    // ...
} // Stream state restored.
```

The state that is saved is the .flags() and the .fill() character.

Definition at line 25 of file save_stream_state.h.

8.13.2 Constructor & Destructor Documentation

8.13.2.1 StreamFormatState()

```
StreamFormatState::StreamFormatState (
            std::basic_ios< char > & stream )  [inline]
```

Takes a stream and records its formatting state.

Parameters

stream	The stream.

Definition at line 31 of file save_stream_state.h.

8.13.2.2 ~StreamFormatState()

```
StreamFormatState::~StreamFormatState ( )  [inline]
```

Destruction. This restores the formatting state of the stream.

Definition at line 35 of file save_stream_state.h.

8.13.3 Member Data Documentation

8.13.3.1 _ff

```
std::ios_base::fmtflags StreamFormatState::_ff [private]
```

Definition at line 43 of file save_stream_state.h.

8.13.3.2 _fill

```
char StreamFormatState::_fill [private]
```

Definition at line 44 of file save_stream_state.h.

8.13.3.3 _pStream

```
std::basic_ios<char>* StreamFormatState::_pStream [private]
```

Definition at line 42 of file save_stream_state.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/test/save_stream_state.h

8.14 SubTestCount Class Reference

```
#include <TestFramework.h>
```

Public Member Functions

- SubTestCount ()
- int failure () const
- size_t test_count () const
- void test (bool condition)
- std::vector< size_t > test_failures () const

Protected Attributes

- int m_failure
- size_t m_test_count

8.14.1 Detailed Description

In a single test function there might be many tests, this class keeps track of which sub-test is being evaluated and uniquely sets the failure flag by bit twiddling.

Definition at line 280 of file TestFramework.h.

8.14.2 Constructor & Destructor Documentation

8.14.2.1 SubTestCount()

```
SubTestCount::SubTestCount ( ) [inline]
```

Definition at line 282 of file TestFramework.h.

8.14.3 Member Function Documentation

8.14.3.1 failure()

```
int SubTestCount::failure ( ) const [inline]
```

Definition at line 283 of file TestFramework.h.

8.14.3.2 test()

```
void SubTestCount::test (
            bool condition )
```

Definition at line 318 of file TestFramework.cpp.

8.14.3.3 test_count()

```
size_t SubTestCount::test_count ( ) const [inline]
```

Definition at line 284 of file TestFramework.h.

8.14.3.4 test_failures()

```
std::vector< size_t > SubTestCount::test_failures ( ) const
```

Definition at line 328 of file TestFramework.cpp.

8.14.4 Member Data Documentation

8.14.4.1 m_failure

```
int SubTestCount::m_failure [protected]
```

Definition at line 290 of file TestFramework.h.

8.14.4.2 m_test_count

```
size_t SubTestCount::m_test_count [protected]
```

Definition at line 291 of file TestFramework.h.

The documentation for this class was generated from the following files:

- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h
- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.cpp

8.15 OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare > Class Template Reference

Class that represents a stack of references to other nodes.

```
#include <NodeRefs.h>
```

Public Member Functions

- SwappableNodeRefStack ()
- const NodeRef< T, Compare > & operator[ ] (size_t level) const
- NodeRef< T, Compare > & operator[ ] (size_t level)
- size_t height () const

  Number of nodes referenced.
- size_t swapLevel () const

  The current swap level.
- bool canSwap () const

  true if a swap can take place `_swapLevel < height()`
- bool noNodePointerMatches (const Node< T, Compare > *p) const
- bool allNodePointerMatch (const Node< T, Compare > *p) const
- void push_back (Node< T, Compare > *p, size_t w)

  Add a new reference.
- void pop_back ()

  Remove top reference.
- void swap (SwappableNodeRefStack< T, Compare > &val)
- void resetSwapLevel ()

  Reset the swap level (for example before starting a remove).
- void incSwapLevel ()
- IntegrityCheck lacksIntegrity () const
- size_t size_of () const

Protected Attributes

- std::vector< struct NodeRef< T, Compare > > _nodes

  Stack of NodeRef node references.
- size_t _swapLevel

  The current swap level.

Private Member Functions

- SwappableNodeRefStack (const SwappableNodeRefStack &that)

  Prevent cctor.
- SwappableNodeRefStack & operator= (const SwappableNodeRefStack &that) const

  Prevent operator=.

8.15.1 Detailed Description

template<typename T, typename Compare>
class OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >

Class that represents a stack of references to other nodes.

Each reference is a NodeRef so a pointer to a Node and a width. This just does simple bookkeeping on this stack.

It also facilitates swapping references with another SwappableNodeRefStack when inserting or removing a Node.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Definition at line 48 of file NodeRefs.h.

8.15.2 Constructor & Destructor Documentation

8.15.2.1 SwappableNodeRefStack() [1/2]

```
template<typename T , typename Compare >
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::SwappableNodeRefStack ( )
[inline]
```

Constructor. Initialises the swap level to 0.

Definition at line 53 of file NodeRefs.h.

8.15.2.2 SwappableNodeRefStack() [2/2]

```
template<typename T , typename Compare >
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::SwappableNodeRefStack (
            const SwappableNodeRefStack< T, Compare > & that )  [private]
```

Prevent cctor.

8.15.3 Member Function Documentation

8.15.3.1 allNodePointerMatch()

```
template<typename T , typename Compare >
bool OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::allNodePointerMatch (
            const Node< T, Compare > * p ) const
```

Returns true if all pointers in my data are equal to p. Should be true after a remove operation.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

p	The Node.

Returns

true if all the Node references are un-swapped (all are referring to the given Node).

Definition at line 184 of file NodeRefs.h.

8.15.3.2 canSwap()

```
template<typename T , typename Compare >
bool OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::canSwap ( ) const  [inline]
```

true if a swap can take place `_swapLevel < height()`

Definition at line 71 of file NodeRefs.h.

8.15.3.3 height()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::height ( ) const  [inline]
```

Number of nodes referenced.

Definition at line 63 of file NodeRefs.h.

8.15.3.4 incSwapLevel()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::incSwapLevel ( )  [inline]
```

Increment the swap level. This is used when removing nodes where the parent node can record to what level it has made its adjustments so the grand parent knows where to start.

For this reason the _swapLevel can easily be >= `_nodes.size()`.

Definition at line 104 of file NodeRefs.h.

8.15.3.5 lacksIntegrity()

```
template<typename T , typename Compare >
IntegrityCheck OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::lacksIntegrity
```

This checks the internal consistency of the object. It returns INTEGRITY_SUCCESS [0] if successful or non-zero on error. The tests are:

- Widths must all be >= 1
- Widths must be weakly increasing with increasing level

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Returns

An IntegrityCheck enum.

Definition at line 223 of file NodeRefs.h.

8.15.3.6 noNodePointerMatches()

```
template<typename T , typename Compare >
bool OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::noNodePointerMatches (
            const Node< T, Compare > * p ) const
```

Whether all node references are swapped. Should be true after an insert operation.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

p	The Node.

Returns

true if all the Node references are swapped (none are referring to the given Node).

Definition at line 165 of file NodeRefs.h.

8.15.3.7 operator=()

```
template<typename T , typename Compare >
SwappableNodeRefStack& OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::operator=
(
            const SwappableNodeRefStack< T, Compare > & that ) const  [private]
```

Prevent operator=.

8.15.3.8 operator[]() [1/2]

```
template<typename T , typename Compare >
NodeRef< T, Compare > & OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::operator[]
(
            size_t level )
```

The writeable NodeRef at the given level.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

level	The level.

Returns

A reference to the Node.

Definition at line 149 of file NodeRefs.h.

8.15.3.9 operator[]() [2/2]

```
template<typename T , typename Compare >
const NodeRef< T, Compare > & OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >↩
::operator[] (
            size_t level ) const
```

The readable NodeRef at the given level.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

level	The level.

Returns

A reference to the Node.

Definition at line 134 of file NodeRefs.h.

8.15.3.10 pop_back()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::pop_back ( )  [inline]
```

Remove top reference.

Definition at line 89 of file NodeRefs.h.

8.15.3.11 push_back()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::push_back (
            Node< T, Compare > * p,
            size_t w )  [inline]
```

Add a new reference.

Definition at line 83 of file NodeRefs.h.

8.15.3.12 resetSwapLevel()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::resetSwapLevel ( )  [inline]
```

Reset the swap level (for example before starting a remove).

Definition at line 97 of file NodeRefs.h.

8.15.3.13 size_of()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::size_of
```

Returns an estimate of the memory usage of an instance

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Returns

The memory estimate.

Definition at line 245 of file NodeRefs.h.

8.15.3.14 swap()

```
template<typename T , typename Compare >
void OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::swap (
            SwappableNodeRefStack< T, Compare > & val )
```

Swap references with another SwappableNodeRefStack at the current swap level. This also increments the swap level.

Template Parameters

T	The type of the Skip List Node values.
Compare	A comparison function for type T.

Parameters

val	The SwappableNodeRefStack.

Definition at line 202 of file NodeRefs.h.

8.15.3.15 swapLevel()

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::swapLevel ( ) const
[inline]
```

The current swap level.

Definition at line 68 of file NodeRefs.h.

8.15.4 Member Data Documentation

8.15.4.1 _nodes

```
template<typename T , typename Compare >
std::vector<struct NodeRef<T, Compare> > OrderedStructs::SkipList::SwappableNodeRefStack< T,
Compare >::_nodes  [protected]
```

Stack of NodeRef node references.

Definition at line 113 of file NodeRefs.h.

8.15.4.2 _swapLevel

```
template<typename T , typename Compare >
size_t OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >::_swapLevel [protected]
```

The current swap level.

Definition at line 115 of file NodeRefs.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/NodeRefs.h

8.16 TestResult::test_result Struct Reference

Public Attributes

- size_t count
- int failed
- std::list< double > _execTimeS

8.16.1 Detailed Description

Definition at line 245 of file TestFramework.h.

8.16.2 Member Data Documentation

8.16.2.1 _execTimeS

```
std::list<double> TestResult::test_result::_execTimeS
```

Definition at line 248 of file TestFramework.h.

8.16.2.2 count

```
size_t TestResult::test_result::count
```

Definition at line 246 of file TestFramework.h.

8.16.2.3 failed

```
int TestResult::test_result::failed
```

Definition at line 247 of file TestFramework.h.

The documentation for this struct was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h

8.17 TestResult Class Reference

```
#include <TestFramework.h>
```

Classes

- struct test_result

Public Member Functions

- TestResult (std::string name, int failed, double execTime, size_t testCount=1, size_t scale=1)
- TestResult (std::string name)
- void setFailed (size_t scale=1)

  Setters:
- void execTimeAdd (int failed, double execTime, size_t test_count, size_t scale=1)
- std::string name () const

  Getters.
- int failed () const
- int failed (size_t scale) const
- size_t numTests () const
- size_t numTests (size_t scale) const
- size_t testCount (size_t scale) const
- double totalTime (size_t scale) const
- double execTime (size_t scale) const
- double atomicTestMeanExecTime (size_t scale) const
- double execTimeStdDev (size_t scale) const
- bool hasExecTimeStdDev (size_t scale) const
- double execTimeMin (size_t scale) const
- double execTimeMax (size_t scale) const
- size_t numScaleValues () const
- std::vector< size_t > scaleValues () const
- TestResult (const TestResult &rhs)=default
- TestResult & operator= (const TestResult &rhs)=default
- TestResult (TestResult &&)=default

Private Attributes

- std::string _name
- std::map< size_t, struct test_result > _results

8.17.1 Detailed Description

TestResult is a class that retains correctness and performance information of a particular test.

Terminology: scale - The declared scale of the test for time complexity analysis. repeat - The number of times an identical test is conducted to get a statistical feel for the execution time. count - The number of atomic operations that the execution time has measured.

NOTE: execTime() gives the mean time of all of the executions. This needs to be divided by `count` to get the execution time of a single operation.

For example timing sorting a vector. scale: might have a range 32, 64, 128, 256, 512, 1024 (vector length). repeat might be 10 to get some min/max/mean/std. dev. values for variety. count: might be 100 to get a reasonable (single) execution time for each test.

Typical usage: test_something() does a single test, possibly some fast operation count number of times to get a decent time.

test_multiple() does the same but repeats it 10 times to get some statistics on it.

test_complexity() does the same as test_multiple() but over a range of values of N so that some sense of the time complexity can be gained.

```
void test_something(TestResultS &results) {
   ExecClock clk;
   // Do something count number of times here, set failure to 0 | 1.
   results.push_back(TestResult(__FUNCTION__, failure, clk.execTime(), count));
}
void test_multiple(TestResultS &results) {
   TestResult test_result(__FUNCTION__);
   for (size_t i = 0; i < 10; ++i) {
       ExecClock clk;
       for (size_t j = 0; j < COUNT; ++j) {
           // Do something small here COUNT number of times
       }
       test_result.execTimeAdd(0, clk.execTime(), COUNT);
   }
   results.push_back(test_result);
}
void test_complexity(TestResultS &results) {
   int failure = 0;
   TestResult test_result(__FUNCTION__);
   std::list<size_t> scales = {1024, 2048, 4096, 8192};
   for (auto scale: scales) {
       // Create something appropriate to scale
       vector<char> input;
       for (tTimeVectWord i = 0; i < scale; ++i) {
           input.push_back(i);
       }
       for (size_t repeat = 0; repeat < 10; ++repeat) {
           ExecClock clk;
           // Do something small here COUNT number of times, set failure.
           test_result.execTimeAdd(failure, clk.execTime(), COUNT, scale);
       }
    }
    results.push_back(test_result);
}
int main() {
   TestResultS results;
   test_something(results);
   test_multiple(results);
   test_complexity(results);
   std::cout « results;
}
```

8.17.1.1 Note on gnuplot

See also: `std::ostream &operator<<(std::ostream &os, const TestResult &result)`

This example is if the following tests are reported:

```
// Add 4 (mock_repeat) tests results
mock_test_result.execTimeAdd(0, 1.0, 100, 99);
```

```
mock_test_result.execTimeAdd(0, 2.0, 100, 99);
mock_test_result.execTimeAdd(0, 3.0, 100, 99);
mock_test_result.execTimeAdd(0, 4.0, 100, 99);
```

If the test output is put into individual .dat files such as:

```
#HEAD: Fail   Scale  Repeat         Mean(s)     Std.Dev.(s)         Min.(s)         Max.(s)     Count
       Rate(/s) Name
#  $1   $2      $3      $4              $5              $6              $7              $8        $9
         $10 $11
TEST:    0      99       4     2.500000000     1.118033989     1.000000000     4.000000000       400
       160.0 test_internal_test_result_string
```

So:

- The total time to run all of these tests is: `($5 * $4)`, in this case 10.0 (s).
- The mean time for each atomic test is: `($5 * $4 / $9)`, in this case 10.0 / 400 = 0.025 (s)
- The standard deviation atomic time is: `($6 / ($9 / $4))` or `($6 * $4 / $9)`, in this case 4 ∗ 1.11 / 400 = +/- 0.0111 (s)
- The minimum atomic time is: `($7 / ($9 / $4))` or `($7 * $4 / $9)`, in this case 4 ∗ 1.0 / 400 = 0.010 (s)
- The maximum atomic time is: `($8 / ($9 / $4))` or `($8 * $4 / $9)`, in this case 4 ∗ 4.0 / 400 = 0.040 (s)

Candlestick plots, from gnuplot: "Both require five columns of data: the x value, followed (in order) by the opening, low, high, and closing prices."

So a suitable gnuplot statement for a candle stick plot with time in µs is:

```
opening  : Mean - Std.Dev    (($5 - $6) * $4 / $9)
low      : Min               ($7 * $4 / $9)
high     : Max               ($8 * $4 / $9)
closing  : Mean + Std.Dev    (($5 + $6) * $4 / $9)
```

And for a suitable gnuplot statement for a candle stick plot with rate in µs / object is:

```
opening  : Mean - Std.Dev    (($5 - $6) * $4 / ($9 * $3))
low      : Min               ($7 * $4 / ($9 * $3))
high     : Max               ($8 * $4 / ($9 * $3))
closing  : Mean + Std.Dev    (($5 + $6) * $4 / ($9 * $3))
```

So for a time plot:

```
plot "dat/test_cpp_vector_char_to_py_bytes.dat" \
     using 3:(1e6 * ($5 - $6) * $4 / $9):(1e6 * $7 * $4 / $9):(1e6 * $8 * $4 / $9):(1e6 * ($5 + $6) * $4 /
       $9) \
     t "Python str 8 bit -> C++" with candlesticks whiskerbars 0.5 linewidth 2,\
     "dat/test_cpp_vector_char_to_py_bytes.dat" using 3:(1e6 * $7 * $4 / $9) \
      t "Minimum Python str 8 bit -> C++" with lines linewidth 2, \
```

And for a rate plot:

```
plot "dat/test_cpp_vector_char_to_py_bytes.dat" \
     using 3:(1e6 * ($5 - $6) * $4 / ($9 * $3)):(1e6 * $7 * $4 / ($9 * $3)):(1e6 * $8 * $4 / ($9 * $3)):(1e6
       * ($5 + $6) * $4 / ($9 * $3)) \
     t "Python str 8 bit -> C++" with candlesticks whiskerbars 0.5 linewidth 2,\
     "dat/test_cpp_vector_char_to_py_bytes.dat" using 3:(1e6 * $7 * $4 / ($9 * $3)) \
      t "Minimum Python str 8 bit -> C++" with lines linewidth 2, \
```

NOTE: Time plot was, in error:

```
plot "dat/test_cpp_vector_char_to_py_bytes.dat" \
     using 3:(1e6 * ($5 - $6) / $4):(1e6 * $7 / $4):(1e6 * $8 / $4):(1e6 * ($5 + $6) / $4) \
     t "Python str 8 bit -> C++" with candlesticks whiskerbars 0.5 linewidth 2,\
     "dat/test_cpp_vector_char_to_py_bytes.dat" using 3:($7 * 1e6 / $4) \
      t "Minimum Python str 8 bit -> C++" with lines linewidth 2, \
```

8.17.1.2 Note on Using the Rate Column

If the mean value is very small it will loose resolution. For example here is a test that takes 1.23456789 ns that only has 1 S.F. resolution. Inverting the rate can recover the time to a much higher resolution.

```
HEAD: Fail   Scale  Repeat         Mean(s)     Std.Dev.(s)         Min.(s)         Max.(s)     Count
      Rate(/s) Name
TEST:    0      99       1     0.000000001             N/A             N/A             N/A         1
      810000007.4 test_internal_test_result_string_using_rate
```

Definition at line 200 of file TestFramework.h.

8.17.2 Constructor & Destructor Documentation

8.17.2.1 TestResult() [1/4]

```
TestResult::TestResult (
            std::string name,
            int failed,
            double execTime,
            size_t testCount = 1,
            size_t scale = 1 )
```

Definition at line 13 of file TestFramework.cpp.

8.17.2.2 TestResult() [2/4]

```
TestResult::TestResult (
            std::string name )  [inline]
```

Definition at line 211 of file TestFramework.h.

8.17.2.3 TestResult() [3/4]

```
TestResult::TestResult (
            const TestResult & rhs )  [default]
```

8.17.2.4 TestResult() [4/4]

```
TestResult::TestResult (
            TestResult &&  )  [default]
```

8.17.3 Member Function Documentation

8.17.3.1 atomicTestMeanExecTime()

```
double TestResult::atomicTestMeanExecTime (
            size_t scale ) const
```

Returns the sum of the execution times divided by the number of atomic tests.

Definition at line 92 of file TestFramework.cpp.

8.17.3.2 execTime()

```
double TestResult::execTime (
            size_t scale ) const
```

Returns the mean of the execution times. This does not take into account the atomic test counts.

Definition at line 78 of file TestFramework.cpp.

8.17.3.3 execTimeAdd()

```
void TestResult::execTimeAdd (
            int failed,
            double execTime,
            size_t test_count,
            size_t scale = 1 )
```

Definition at line 36 of file TestFramework.cpp.

8.17.3.4 execTimeMax()

```
double TestResult::execTimeMax (
            size_t scale ) const
```

Definition at line 131 of file TestFramework.cpp.

8.17.3.5 execTimeMin()

```
double TestResult::execTimeMin (
            size_t scale ) const
```

Definition at line 116 of file TestFramework.cpp.

8.17.3.6 execTimeStdDev()

```
double TestResult::execTimeStdDev (
            size_t scale ) const
```

Definition at line 101 of file TestFramework.cpp.

8.17.3.7 failed() [1/2]

```
int TestResult::failed ( ) const
```

Definition at line 27 of file TestFramework.cpp.

8.17.3.8 failed() [2/2]

```
int TestResult::failed (
            size_t scale ) const
```

Definition at line 22 of file TestFramework.cpp.

8.17.3.9 hasExecTimeStdDev()

```
bool TestResult::hasExecTimeStdDev (
            size_t scale ) const  [inline]
```

Definition at line 234 of file TestFramework.h.

8.17.3.10 name()

```
std::string TestResult::name ( ) const  [inline]
```

Getters.

Definition at line 218 of file TestFramework.h.

8.17.3.11 numScaleValues()

```
size_t TestResult::numScaleValues ( ) const
```

Definition at line 142 of file TestFramework.cpp.

8.17.3.12 numTests() [1/2]

```
size_t TestResult::numTests ( ) const
```

Definition at line 51 of file TestFramework.cpp.

8.17.3.13 numTests() [2/2]

```
size_t TestResult::numTests (
            size_t scale ) const
```

Definition at line 60 of file TestFramework.cpp.

8.17.3.14 operator=()

```
TestResult& TestResult::operator= (
            const TestResult & rhs )  [default]
```

8.17.3.15 scaleValues()

```
std::vector< size_t > TestResult::scaleValues ( ) const
```

Definition at line 146 of file TestFramework.cpp.

8.17.3.16 setFailed()

```
void TestResult::setFailed (
            size_t scale = 1 )  [inline]
```

Setters:

Definition at line 214 of file TestFramework.h.

8.17.3.17 testCount()

```
size_t TestResult::testCount (
            size_t scale ) const  [inline]
```

Definition at line 226 of file TestFramework.h.

8.17.3.18 totalTime()

```
double TestResult::totalTime (
            size_t scale ) const
```

Definition at line 64 of file TestFramework.cpp.

8.17.4 Member Data Documentation

8.17.4.1 _name

```
std::string TestResult::_name  [private]
```

Definition at line 244 of file TestFramework.h.

8.17.4.2 _results

```
std::map<size_t, struct test_result> TestResult::_results  [private]
```

Definition at line 251 of file TestFramework.h.

The documentation for this class was generated from the following files:

- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h
- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.cpp

8.18 TestResultS Class Reference

```
#include <TestFramework.h>
```

Public Types

- typedef std::list< TestResult > tResults

Public Member Functions

- TestResultS ()
- void push_back (const TestResult &result)
- const tResults & results () const
- void dump_header (std::ostream &os) const
- void dump_tests (std::ostream &os) const
- void dump_tail (std::ostream &os) const
- int failed () const

Private Attributes

- int _failed
- tResults _results

8.18.1 Detailed Description

This just accumulates a list of TestResult objects and can print them out in a human readable form.

Definition at line 257 of file TestFramework.h.

8.18.2 Member Typedef Documentation

8.18.2.1 tResults

```
typedef std::list<TestResult> TestResultS::tResults
```

Definition at line 259 of file TestFramework.h.

8.18.3 Constructor & Destructor Documentation

8.18.3.1 TestResultS()

```
TestResultS::TestResultS ( ) [inline]
```

Definition at line 260 of file TestFramework.h.

8.18.4 Member Function Documentation

8.18.4.1 dump_header()

```
void TestResultS::dump_header (
            std::ostream & os ) const
```

Definition at line 236 of file TestFramework.cpp.

8.18.4.2 dump_tail()

```
void TestResultS::dump_tail (
            std::ostream & os ) const
```

Definition at line 282 of file TestFramework.cpp.

8.18.4.3 dump_tests()

```
void TestResultS::dump_tests (
            std::ostream & os ) const
```

Definition at line 276 of file TestFramework.cpp.

8.18.4.4 failed()

```
int TestResultS::failed ( ) const  [inline]
```

Definition at line 266 of file TestFramework.h.

8.18.4.5 push_back()

```
void TestResultS::push_back (
            const TestResult & result )
```

Definition at line 226 of file TestFramework.cpp.

8.18.4.6 results()

```
const tResults& TestResultS::results ( ) const  [inline]
```

Definition at line 262 of file TestFramework.h.

8.18.5 Member Data Documentation

8.18.5.1 _failed

```
int TestResultS::_failed [private]
```

Definition at line 269 of file TestFramework.h.

8.18.5.2 _results

```
tResults TestResultS::_results [private]
```

Definition at line 270 of file TestFramework.h.

The documentation for this class was generated from the following files:

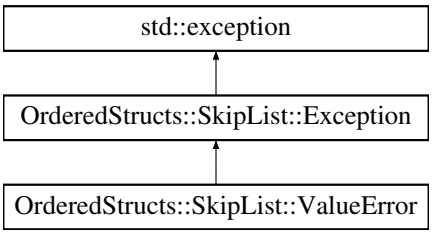
- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h
- /Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.cpp

8.19 OrderedStructs::SkipList::ValueError Class Reference

Specialised exception for an value error where the given value does not exist in the Skip List.

```
#include <SkipList.h>
```

Inheritance diagram for OrderedStructs::SkipList::ValueError:



Public Member Functions

- ValueError (const std::string &in_msg)

Additional Inherited Members

8.19.1 Detailed Description

Specialised exception for an value error where the given value does not exist in the Skip List.

Definition at line 512 of file SkipList.h.

8.19.2 Constructor & Destructor Documentation

8.19.2.1 ValueError()

```
OrderedStructs::SkipList::ValueError::ValueError (
            const std::string & in_msg )  [inline], [explicit]
```

Definition at line 514 of file SkipList.h.

The documentation for this class was generated from the following file:

- /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h

Chapter 9

File Documentation

9.1 /Users/paulross/CLionProjects/skiplist/src/cpp/HeadNode.h File Reference

```
#include <functional>
#include <vector>
#include <sstream>
#include "IntegrityEnums.h"
```

Classes

- class OrderedStructs::SkipList::HeadNode< T, Compare >

  A HeadNode is a skip list. This is the single node leading to all other content Nodes.

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::SkipList

  Namespace for the C++ Slip List.

9.1.1 Detailed Description

Project: skiplist

Created by Paul Ross on 03/12/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2017-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

9.2 /Users/paulross/CLionProjects/skiplist/src/cpp/IntegrityEnums.h File Reference

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::SkipList

  Namespace for the C++ Slip List.

Enumerations

- enum OrderedStructs::SkipList::IntegrityCheck { OrderedStructs::SkipList::INTEGRITY_SUCCESS = 0 , OrderedStructs::SkipList::NODEREFS_WIDTH_ZERO_NOT_UNITY = 100 , OrderedStructs::SkipList::NODEREFS_WIDTH_DECREASING , OrderedStructs::SkipList::NODE_HEIGHT_ZERO = 200 , OrderedStructs::SkipList::NODE_HEIGHT_EXCEEDS_HEADNODE , OrderedStructs::SkipList::NODE_NON_NULL_AFTER_N , OrderedStructs::SkipList::NODE_SELF_REFERENCE , OrderedStructs::SkipList::NODE_REFERENCES_NOT_IN_GLOBAL , OrderedStructs::SkipList::HEADNODE_CONTAINS_NULL = 300 , OrderedStructs::SkipList::HEADNODE_COUNT_MISMATCH , OrderedStructs::SkipList::HEADNODE_LEVEL_WIDTHS_MISMATCH , OrderedStructs::SkipList::HEADNODE_DETECTS_C , OrderedStructs::SkipList::HEADNODE_DETECTS_OUT_OF_ORDER }

9.2.1 Detailed Description

Project: skiplist

Integrity codes for structures in this code.

Created by Paul Ross on 11/12/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.

```
MIT License
Copyright (c) 2015-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

9.3 /Users/paulross/CLionProjects/skiplist/src/cpp/main.cpp File Reference

```
#include <cstdlib>
#include <iostream>
#include <iomanip>
#include <sstream>
#include <ctime>
#include <unistd.h>
#include "SkipList.h"
#include "save_stream_state.h"
#include "test/test_documentation.h"
#include "test/test_functional.h"
#include "test/test_performance.h"
#include "test/test_rolling_median.h"
#include "test/test_concurrent.h"
```

Functions

- void test_clock_resolution ()
- int test_example_introduction_A (void)
- int test_all ()
- int main (int, const char *[ ])

9.3.1 Function Documentation

9.3.1.1 main()

```
int main (
            int ,
            const char * [ ] )
```

Definition at line 107 of file main.cpp.

9.3.1.2 test_all()

```
int test_all ( )
```

Definition at line 78 of file main.cpp.

9.3.1.3 test_clock_resolution()

```
void test_clock_resolution ( )
```

Definition at line 28 of file main.cpp.

9.3.1.4 test_example_introduction_A()

```
int test_example_introduction_A (
            void  )
```

Definition at line 44 of file main.cpp.

9.4 /Users/paulross/CLionProjects/skiplist/src/cpp/Node.h File Reference

```
#include "IntegrityEnums.h"
```

Classes

- class OrderedStructs::SkipList::Node< T, Compare >

  A single node in a Skip List containing a value and references to other downstream Node objects.

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::SkipList

  Namespace for the C++ Slip List.

Macros

- #define nullptr NULL

9.4.1 Detailed Description

Project: skiplist

Concurrency Tests.

Created by Paul Ross on 03/12/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.
```
MIT License
Copyright (c) 2015-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

9.4.2 Macro Definition Documentation

9.4.2.1 nullptr

```
#define nullptr NULL
```

Definition at line 43 of file Node.h.

9.5 /Users/paulross/CLionProjects/skiplist/src/cpp/NodeRefs.h File Reference

```
#include "IntegrityEnums.h"
```

Classes

- struct OrderedStructs::SkipList::NodeRef< T, Compare >

  A PoD struct that contains a pointer to a Node and a width that represents the coarser linked list span to the next Node.
- class OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >

  Class that represents a stack of references to other nodes.

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::SkipList

  Namespace for the C++ Slip List.

9.6 /Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.cpp File Reference

```
#include "RollingMedian.h"
```

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::RollingMedian

  Namespace for the C++ Rolling Median.

Functions

- size_t OrderedStructs::RollingMedian::dest_count (size_t count, size_t win_length)
- size_t OrderedStructs::RollingMedian::dest_size (size_t count, size_t win_length, size_t dest_stride)

9.7 /Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.h File Reference

```
#include <stdlib.h>
#include "SkipList.h"
```

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::RollingMedian

  Namespace for the C++ Rolling Median.

Macros

- #define ROLLING_MEDIAN_ERROR_CHECK

Enumerations

- enum OrderedStructs::RollingMedian::RollingMedianResult { OrderedStructs::RollingMedian::ROLLING_MEDIAN_SUCCESS = 0 , OrderedStructs::RollingMedian::ROLLING_MEDIAN_SOURCE_STRIDE , OrderedStructs::RollingMedian::ROLLING_MEI , OrderedStructs::RollingMedian::ROLLING_MEDIAN_WIN_LENGTH }

Functions

- template<typename T >
  RollingMedianResult OrderedStructs::RollingMedian::rolling_median (const std::vector< T > data, size_↩ t win_length, std::vector< T > &result)
- template<typename T >
  RollingMedianResult OrderedStructs::RollingMedian::rolling_median_lower_bound (const std::vector< T > data, size_t win_length, std::vector< T > &result)
- size_t OrderedStructs::RollingMedian::dest_count (size_t count, size_t win_length)
- size_t OrderedStructs::RollingMedian::dest_size (size_t count, size_t win_length, size_t dest_stride)
- template<typename T >
  RollingMedianResult OrderedStructs::RollingMedian::odd_index (const T *src, size_t src_stride, size_t count, size_t win_length, T *dest, size_t dest_stride)
- template<typename T >
  RollingMedianResult OrderedStructs::RollingMedian::even_index (const T *src, size_t src_stride, size_↩ t count, size_t win_length, T *dest, size_t dest_stride)
- template<typename T >
  RollingMedianResult OrderedStructs::RollingMedian::even_odd_index (const T *src, size_t src_stride, size↩ _t count, size_t win_length, T *dest, size_t dest_stride)

9.7.1 Detailed Description

Project: skiplist

Rolling Median.

Created by Paul Ross on 18/12/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2015-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

9.7.2 Macro Definition Documentation

9.7.2.1 ROLLING_MEDIAN_ERROR_CHECK

```
#define ROLLING_MEDIAN_ERROR_CHECK
```

Value:

```
    do {                                                    \
        if (src_stride == 0) {                              \
            return ROLLING_MEDIAN_SOURCE_STRIDE;            \
        }                                                   \
        if (dest_stride == 0) {                             \
            return ROLLING_MEDIAN_DESTINATION_STRIDE;       \
        }                                                   \
        if (win_length == 0) {                              \
            return ROLLING_MEDIAN_WIN_LENGTH;               \
        }                                                   \
    } while (0)
```

Return an error code.

Definition at line 139 of file RollingMedian.h.

9.8 /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.cpp File Reference

```
#include <cstdlib>
#include <mutex>
#include <string>
#include "SkipList.h"
```

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::SkipList

  Namespace for the C++ Slip List.

Functions

- bool OrderedStructs::SkipList::tossCoin ()
- void OrderedStructs::SkipList::seedRand (unsigned seed)
- void OrderedStructs::SkipList::_throw_exceeds_size (size_t index)

Variables

- std::mutex OrderedStructs::SkipList::gSkipListMutex

9.9 /Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h File Reference

```
#include <functional>
#include <vector>
#include <set>
#include <string>
#include <cassert>
#include <iostream>
#include <sstream>
#include <mutex>
#include "NodeRefs.h"
#include "Node.h"
#include "HeadNode.h"
```

Classes

- class OrderedStructs::SkipList::Exception

  Base exception class for all exceptions in the OrderedStructs::SkipList namespace.
- class OrderedStructs::SkipList::IndexError

  Specialised exception case for an index out of range error.
- class OrderedStructs::SkipList::ValueError

  Specialised exception for an value error where the given value does not exist in the Skip List.
- class OrderedStructs::SkipList::FailedComparison

  Specialised exception used for NaN detection where value != value (example NaNs).

Namespaces

- OrderedStructs

  Namespace for all the C++ ordered structures.
- OrderedStructs::SkipList

  Namespace for the C++ Slip List.

Macros

- #define INCLUDE_METHODS_THAT_USE_STREAMS

Functions

- void OrderedStructs::SkipList::_throw_exceeds_size (size_t index)
- bool OrderedStructs::SkipList::tossCoin ()
- void OrderedStructs::SkipList::seedRand (unsigned seed)

9.9.1 Detailed Description

Project: skiplist

Created by Paul Ross on 15/11/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2017-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

9.9.2 Macro Definition Documentation

9.9.2.1 INCLUDE_METHODS_THAT_USE_STREAMS

```
#define INCLUDE_METHODS_THAT_USE_STREAMS
```

Defined if you want the SkipList to have methods that can output to stream (for debugging for example). Defining this will mean that classes grow methods that use streams. Undef this if you want a smaller binary in production as using streams adds typically around 30kb to the binary. However you may loose useful information such as formatted exception messages with extra data.

Definition at line 442 of file SkipList.h.

9.10 /Users/paulross/CLionProjects/skiplist/src/cpp/test/save_stream_state.h File Reference

```
#include <iostream>
#include <iomanip>
```

Classes

- class StreamFormatState

9.11 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.cpp File Reference

```
#include <iomanip>
#include <thread>
#include "SkipList.h"
#include "TestFramework.h"
#include "test_print.h"
#include "test_concurrent.h"
```

Functions

- template<typename T >
  static void insert_value (OrderedStructs::SkipList::HeadNode< T > *psl, const T &value)
- template<typename T >
  static void insert_has_remove (OrderedStructs::SkipList::HeadNode< T > *psl, const T &value)
- template<typename T >
  static void insert_has_remove_count (OrderedStructs::SkipList::HeadNode< T > *psl, const T &value, const size_t &count)
- template<typename T >
  static void insert_count_has_remove_count (OrderedStructs::SkipList::HeadNode< T > *psl, const T &value, const size_t &count)
- static int test_single_thread_insert ()
- static int test_two_thread_insert_has_remove ()
- static int test_two_thread_insert_count_has_remove_count ()
- static int _test_perf_insert_count_has_remove_count_multi_threads (const char *caller_name, size_t thread_count, size_t count)
- int test_perf_insert_count_has_remove_count_multi_threads_fixed_length ()
- int _test_perf_insert_count_has_remove_count_single_thread (const char *caller_name, size_t repeat_count, size_t count)
- int test_perf_single_thread_fixed_length ()
- int test_perf_sim_rolling_median_single_thread ()
- static int test_perf_sim_rolling_median_multi_thread_multi_no_thread (size_t test_count, size_t repeat, TestResultS &test_results)
- int test_concurrent_all ()

Variables

- const size_t SKIPLIST_FIXED_LENGTH = 1024 * 16

  Size of the test Skip List.
- const size_t SKIPLIST_MAX_THREADS = 128 * 2

  Number of threads < this number, increment is x2.

9.11.1 Detailed Description

Project: skiplist

Concurrency Tests.

Created by Paul Ross on 17/08/2017.

Copyright (c) 2017-2023 Paul Ross. All rights reserved.

```
MIT License
Copyright (c) 2017-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```


9.11.2 Function Documentation

9.11.2.1 _test_perf_insert_count_has_remove_count_multi_threads()

```
static int _test_perf_insert_count_has_remove_count_multi_threads (
            const char * caller_name,
            size_t thread_count,
            size_t count )  [static]
```

Create a `OrderedStructs::SkipList::HeadNode<double>` then create the given number of threads each of which calls insert_count_has_remove_count() with the given count.

This then reports the total time taken in us and the rate which is `thread_count * count / total time`

Example output where the count is fixed at SKIPLIST_FIXED_LENGTH :

```
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:    1 SkiplistSize:    16384
       time:         8673 (us) rate    1.889e+06 /s
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:    2 SkiplistSize:    16384
       time:        71462 (us) rate    4.585e+05 /s
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:    4 SkiplistSize:    16384
       time:       687167 (us) rate    9.537e+04 /s
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:    8 SkiplistSize:    16384
       time:      2328498 (us) rate    5.629e+04 /s
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:   16 SkiplistSize:    16384
       time:      2729010 (us) rate    9.606e+04 /s
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:   32 SkiplistSize:    16384
       time:     10644947 (us) rate    4.925e+04 /s
test_perf_insert_count_has_remove_count_multi_threads_fixed_length(): threads:   64 SkiplistSize:    16384
       time:     15568317 (us) rate    6.735e+04 /s
```

This shows that whilst the total time increases strongly with the number of threads the rate/thread approaches an asymptotic value, although one that is significantly lower than the single threaded value.

Parameters

caller_name	Name of the caller function, the test name.
thread_count	Number of threads to create.
count	Number of insertions into the Skip List.

Returns

-1 if compiled without thread support. Otherwise 0 on success, non-zero on failure.

Definition at line 211 of file test_concurrent.cpp.

9.11.2.2 _test_perf_insert_count_has_remove_count_single_thread()

```
int _test_perf_insert_count_has_remove_count_single_thread (
            const char * caller_name,
            size_t repeat_count,
            size_t count )
```

Test insert/at/remove on a single Skip List in a single-threaded environment.

The repeat count is constant at SKIPLIST_FIXED_LENGTH.

This is comparable with test_perf_single_thread_fixed_length()

Returns

0 on success, non-zero on failure.

Definition at line 308 of file test_concurrent.cpp.

9.11.2.3 insert_count_has_remove_count()

```
template<typename T >
static void insert_count_has_remove_count (
            OrderedStructs::SkipList::HeadNode< T > * psl,
            const T & value,
            const size_t & count )  [static]
```

Insert a value into a Skip List `count` times, check it is there then remove it `count` times.

Template Parameters

T	Type of values in the Skip List.

Parameters

psl	Pointer to the Skip List.
value	Value to insert.
count	Number of times to repeat the insert/remove.

Definition at line 107 of file test_concurrent.cpp.

9.11.2.4 insert_has_remove()

```
template<typename T >
static void insert_has_remove (
            OrderedStructs::SkipList::HeadNode< T > * psl,
            const T & value )  [static]
```

Insert a value into a Skip List, check it is there then remove it.

Template Parameters

T	Type of values in the Skip List.

Parameters

psl	Pointer to the Skip List.
value	Value to insert.

Definition at line 68 of file test_concurrent.cpp.

9.11.2.5 insert_has_remove_count()

```
template<typename T >
static void insert_has_remove_count (
            OrderedStructs::SkipList::HeadNode< T > * psl,
            const T & value,
            const size_t & count )  [static]
```

Insert a value into a Skip List, check it is there then remove it. Repeat this `count` times. This can be used to simulate a rolling median in a single or multi threaded environment.

Template Parameters

T	Type of values in the Skip List.

Parameters

psl	Pointer to the Skip List.
value	Value to insert.
count	Number of times to repeat the insert/add/remove.

Definition at line 86 of file test_concurrent.cpp.

9.11.2.6 insert_value()

```
template<typename T >
static void insert_value (
            OrderedStructs::SkipList::HeadNode< T > * psl,
            const T & value )  [static]
```

Insert a value into a Skip List.

Template Parameters

T	Type of values in the Skip List.

Parameters

psl	Pointer to the Skip List.
value	Value to insert.

Definition at line 55 of file test_concurrent.cpp.

9.11.2.7 test_concurrent_all()

```
int test_concurrent_all ( )
```

Run all concurrency tests.

Returns

0 on success, non-zero on failure.

Definition at line 559 of file test_concurrent.cpp.

9.11.2.8 test_perf_insert_count_has_remove_count_multi_threads_fixed_length()

```
int test_perf_insert_count_has_remove_count_multi_threads_fixed_length ( )
```

Test insert/at/remove on a single Skip List in a multi-threaded environment. Number of threads is from 1 to < SKIPLIST_MAX_THREADS

The repeat count is constant at SKIPLIST_FIXED_LENGTH so, inevitably the more threads the longer the execution time.

This calls _test_perf_insert_count_has_remove_count_multi_threads() that does most of the work.

Returns

0 on success, non-zero on failure.

Definition at line 288 of file test_concurrent.cpp.

9.11.2.9 test_perf_sim_rolling_median_multi_thread_multi_no_thread()

```
static int test_perf_sim_rolling_median_multi_thread_multi_no_thread (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )  [static]
```

Definition at line 448 of file test_concurrent.cpp.

9.11.2.10 test_perf_sim_rolling_median_single_thread()

```
int test_perf_sim_rolling_median_single_thread ( )
```

Definition at line 356 of file test_concurrent.cpp.

9.11.2.11 test_perf_single_thread_fixed_length()

```
int test_perf_single_thread_fixed_length ( )
```

Test insert/at/remove on a single Skip List in a single-threaded environment.

The repeat count is constant at SKIPLIST_FIXED_LENGTH .

This calls _test_perf_insert_count_has_remove_count_single_thread() that does most of the work.

This is comparable with test_perf_insert_count_has_remove_count_multi_threads_fixed_length()

Returns

0 on success, non-zero on failure.

Definition at line 345 of file test_concurrent.cpp.

9.11.2.12 test_single_thread_insert()

```
static int test_single_thread_insert ( ) [static]
```

Functional test of a single insert of a value with a single thread.

Returns

-1 if compiled without thread support. Otherwise 0 on success, non-zero on failure.

Definition at line 124 of file test_concurrent.cpp.

9.11.2.13 test_two_thread_insert_count_has_remove_count()

```
static int test_two_thread_insert_count_has_remove_count ( ) [static]
```

Functional test of a multiple insert/has/remove with a single Skip List with two threads.

Returns

-1 if compiled without thread support. Otherwise 0 on success, non-zero on failure.

Definition at line 164 of file test_concurrent.cpp.

9.11.2.14 test_two_thread_insert_has_remove()

```
static int test_two_thread_insert_has_remove ( ) [static]
```

Functional test of a single insert/has/remove with a single Skip List with two threads.

Returns

-1 if compiled without thread support. Otherwise 0 on success, non-zero on failure.

Definition at line 143 of file test_concurrent.cpp.

9.11.3 Variable Documentation

9.11.3.1 SKIPLIST_FIXED_LENGTH

```
const size_t SKIPLIST_FIXED_LENGTH = 1024 * 16
```

Size of the test Skip List.

Definition at line 181 of file test_concurrent.cpp.

9.11.3.2 SKIPLIST_MAX_THREADS

```
const size_t SKIPLIST_MAX_THREADS = 128 * 2
```

Number of threads < this number, increment is x2.

Definition at line 183 of file test_concurrent.cpp.

9.12 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.h File Reference

Functions

- int test_concurrent_all ()

9.12.1 Detailed Description

Project: skiplist

Concurrency Tests.

Created by Paul Ross on 17/08/2017.

Copyright (c) 2017-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2017-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

9.12.2 Function Documentation

9.12.2.1 test_concurrent_all()

```
int test_concurrent_all ( )
```

Run all concurrency tests.

Returns

0 on success, non-zero on failure.

Definition at line 559 of file test_concurrent.cpp.

9.13 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.cpp File Reference

```
#include <iostream>
#include <iomanip>
#include "test_documentation.h"
#include "test_print.h"
#include "../SkipList.h"
```

Functions

- static int test_doc_height_trend (size_t level)
- static int tests_doc_simple_dot ()
- static int test_doc_insert ()
- static int test_doc_insert_remove ()
- static int test_doc_insert_remove_duplicates ()
- static int test_doc_insert_remove_repeat ()
- int test_documentation_all ()

9.13.1 Detailed Description

Project: skiplist

Documentation Tests.

Created by Paul Ross on 13/07/2016.

Copyright (c) 2016-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2016-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

9.13.2 Function Documentation

9.13.2.1 test_doc_height_trend()

```
static int test_doc_height_trend (
            size_t level )  [static]
```

Print out the height compared with the length of the skip list

Parameters

level	The level to go to.

Returns

Zero.

Definition at line 52 of file test_documentation.cpp.

9.13.2.2 test_doc_insert()

```
static int test_doc_insert ( )  [static]
```

Create a Skip List, populate it and print the DOT file to stdout.

Returns

Zero on success, non-zero on failure.

Definition at line 110 of file test_documentation.cpp.

9.13.2.3 test_doc_insert_remove()

```
static int test_doc_insert_remove ( )  [static]
```

Create a Skip List, populate it, remove items and print the DOT file to stdout.

Returns

Zero on success, non-zero on failure.

Definition at line 134 of file test_documentation.cpp.

9.13.2.4 test_doc_insert_remove_duplicates()

```
static int test_doc_insert_remove_duplicates ( ) [static]
```

Create a Skip List, insert the same value three times then remove it three times and print the DOT file to stdout. This illustrates the stability of the insert/remove methods.

Returns

Zero on success, non-zero on failure.

Definition at line 164 of file test_documentation.cpp.

9.13.2.5 test_doc_insert_remove_repeat()

```
static int test_doc_insert_remove_repeat ( ) [static]
```

Create a Skip List, repeatedly populate it and print the DOT file to stdout.

Returns

Zero on success, non-zero on failure.

Definition at line 194 of file test_documentation.cpp.

9.13.2.6 test_documentation_all()

```
int test_documentation_all ( )
```

Run all the documentation tests.

Returns

Zero on success, non-zero on failure.

Definition at line 230 of file test_documentation.cpp.

9.13.2.7 tests_doc_simple_dot()

```
static int tests_doc_simple_dot ( ) [static]
```

Print out a simple DOT file on stdout.

Returns

Zero.

Definition at line 82 of file test_documentation.cpp.

9.14 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_↩ documentation.h File Reference

Functions

- int test_documentation_all ()

9.14.1 Function Documentation

9.14.1.1 test_documentation_all()

```
int test_documentation_all ( )
```

Run all the documentation tests.

Returns

Zero on success, non-zero on failure.

Definition at line 230 of file test_documentation.cpp.

9.15 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_↩ functional.cpp File Reference

```
#include "test_print.h"
#include "test_functional.h"
#include <functional>
#include "../SkipList.h"
```

Classes

- struct reversed< T >

  Creates a comparison function that return the inverse of `std::less` to create a decreasing Skip List.

Functions

- int test_very_simple_insert ()

  Test the integrity of a Skip List after a single insert.
- int test_simple_insert ()

  Test the integrity of a Skip List after a multiple inserts.
- int test_insert_and_remove_same ()

  Test the integrity of a Skip List after a single insert and remove.
- int test_insert_remove_multiple ()

  Test the integrity of a Skip List after a multiple inserts and removes.
- int test_ins_rem_rand ()

  For many different seeds insert 128 random values then remove them in reverse order.
- int test_insert_n_numbers_same (int n, double value)

  Test the integrity of a Skip List after multiple insert then multiple removes.
- int test_at ()

  Test .at() and the integrity of a Skip List after multiple inserts.
- int test_at_fails ()

  Test .at() ***fails*** *by raising a* `OrderedStructs::SkipList::IndexError`.
- int test_at_dest ()

  Test `at(size_t index, size_t count, T *dest)`.
- int test_at_dest_fails ()

  Test `at(size_t index, size_t count, T *dest)` ***fails*** *throwing a* `OrderedStructs::SkipList::IndexError`.
- int test_has ()

  Test .has() and the integrity of a Skip List after multiple inserts.
- int test_has_not ()

  Test .has() ***fails***.
- int test_remove_fails ()

  Test .remove() ***fails*** *throwing a* `OrderedStructs::SkipList::ValueError`.
- int test_at_large ()

  Test .at() on a larger Skip List with different seeds.
- int test_ins_at_rem_with_srand ()

  Test .insert() .at() .remove() on a larger Skip List with different seeds.
- int test_single_insert_remove ()

  Test .insert() .remove() on a large Skip List.
- int test_single_ins_rem_middle ()

  Test .insert() .remove() in the middle of a medium sized Skip List.
- int test_insert_one_million ()

  Tests insert 1m duplicate doubles for memory measurement.
- int test_insert_nan_throws ()

  Tests inserting a NaN double throws a `OrderedStructs::SkipList::FailedComparison`.
- int test_index_basic_7_node ()

  Tests basic use of .index() on a small Skip List.
- int test_index_throws ()

  Tests .index() throws a `OrderedStructs::SkipList::ValueError` *with a non-existent value.*
- int test_index_large ()

  Tests .index() throws a `OrderedStructs::SkipList::ValueError` *with a non-existent value.*
- int test_reversed_simple_insert ()

  Test a Skip List with a comparison function that gives decreasing order.
- int test_functional_all ()

  All functional tests.

9.15.1 Detailed Description

Project: skiplist

Basic functional tests on a Skip List. Most of these are with a `SkipList<double>`

Created by Paul Ross on 13/07/2016.

Copyright (c) 2016-2023 Paul Ross. All rights reserved.
```
MIT License
Copyright (c) 2016-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

9.15.2 Function Documentation

9.15.2.1 test_at()

```
int test_at ( )
```

Test .at() and the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 214 of file test_functional.cpp.

9.15.2.2 test_at_dest()

```
int test_at_dest ( )
```

Test `at(size_t index, size_t count, T *dest)`.

Also tests the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 286 of file test_functional.cpp.

9.15.2.3 test_at_dest_fails()

```
int test_at_dest_fails ( )
```

Test `at(size_t index, size_t count, T *dest)` **fails** throwing a `OrderedStructs::SkipList::IndexErro`

Also tests the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 313 of file test_functional.cpp.

9.15.2.4 test_at_fails()

```
int test_at_fails ( )
```

Test .at() **fails** by raising a `OrderedStructs::SkipList::IndexError`.

Also test the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 248 of file test_functional.cpp.

9.15.2.5 test_at_large()

```
int test_at_large ( )
```

Test .at() on a larger Skip List with different seeds.

Also tests the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 437 of file test_functional.cpp.

9.15.2.6 test_functional_all()

```
int test_functional_all ( )
```

All functional tests.

Returns

Zero on success, non-zero on failure.

Definition at line 714 of file test_functional.cpp.

9.15.2.7 test_has()

```
int test_has ( )
```

Test .has() and the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 350 of file test_functional.cpp.

9.15.2.8 test_has_not()

```
int test_has_not ( )
```

Test .has() **fails**.

Also tests the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 384 of file test_functional.cpp.

9.15.2.9 test_index_basic_7_node()

```
int test_index_basic_7_node ( )
```

Tests basic use of .index() on a small Skip List.

Returns

Zero on success, non-zero on failure.

Definition at line 598 of file test_functional.cpp.

9.15.2.10 test_index_large()

```
int test_index_large ( )
```

Tests .index() throws a `OrderedStructs::SkipList::ValueError` with a non-existent value.

Medium sized Skip List.

Returns

Zero on success, non-zero on failure.

Definition at line 654 of file test_functional.cpp.

9.15.2.11 test_index_throws()

```
int test_index_throws ( )
```

Tests .index() throws a `OrderedStructs::SkipList::ValueError` with a non-existent value.

Returns

Zero on success, non-zero on failure.

Definition at line 621 of file test_functional.cpp.

9.15.2.12 test_ins_at_rem_with_srand()

```
int test_ins_at_rem_with_srand ( )
```

Test .insert() .at() .remove() on a larger Skip List with different seeds.

Also tests the integrity of a Skip List after each call.

Returns

Zero on success, non-zero on failure.

Definition at line 464 of file test_functional.cpp.

9.15.2.13 test_ins_rem_rand()

```
int test_ins_rem_rand ( )
```

For many different seeds insert 128 random values then remove them in reverse order.

Test the integrity of the Skip List throughout.

Returns

Zero on success, non-zero on failure.

Definition at line 147 of file test_functional.cpp.

9.15.2.14 test_insert_and_remove_same()

```
int test_insert_and_remove_same ( )
```

Test the integrity of a Skip List after a single insert and remove.

Returns

Zero on success, non-zero on failure.

Definition at line 90 of file test_functional.cpp.

9.15.2.15 test_insert_n_numbers_same()

```
int test_insert_n_numbers_same (
            int n,
            double value )
```

Test the integrity of a Skip List after multiple insert then multiple removes.

Returns

Zero on success, non-zero on failure.

Definition at line 193 of file test_functional.cpp.

9.15.2.16 test_insert_nan_throws()

```
int test_insert_nan_throws ( )
```

Tests inserting a NaN double throws a `OrderedStructs::SkipList::FailedComparison`.

Returns

Zero on success, non-zero on failure.

Definition at line 578 of file test_functional.cpp.

9.15.2.17 test_insert_one_million()

```
int test_insert_one_million ( )
```

Tests insert 1m duplicate doubles for memory measurement.

Also tests the integrity of a Skip List after each call.

Returns

Zero on success, non-zero on failure.

Definition at line 553 of file test_functional.cpp.

9.15.2.18 test_insert_remove_multiple()

```
int test_insert_remove_multiple ( )
```

Test the integrity of a Skip List after a multiple inserts and removes.

Returns

Zero on success, non-zero on failure.

Definition at line 109 of file test_functional.cpp.

9.15.2.19 test_remove_fails()

```
int test_remove_fails ( )
```

Test .remove() **fails** throwing a `OrderedStructs::SkipList::ValueError`.

Also tests the integrity of a Skip List after multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 412 of file test_functional.cpp.

9.15.2.20 test_reversed_simple_insert()

```
int test_reversed_simple_insert ( )
```

Test a Skip List with a comparison function that gives decreasing order.

Returns

Zero on success, non-zero on failure.

Definition at line 687 of file test_functional.cpp.

9.15.2.21 test_simple_insert()

```
int test_simple_insert ( )
```

Test the integrity of a Skip List after a multiple inserts.

Returns

Zero on success, non-zero on failure.

Definition at line 65 of file test_functional.cpp.

9.15.2.22 test_single_ins_rem_middle()

```
int test_single_ins_rem_middle ( )
```

Test .insert() .remove() in the middle of a medium sized Skip List.

Also tests the integrity of a Skip List after each call.

Returns

Zero on success, non-zero on failure.

Definition at line 521 of file test_functional.cpp.

9.15.2.23 test_single_insert_remove()

```
int test_single_insert_remove ( )
```

Test .insert() .remove() on a large Skip List.

Also tests the integrity of a Skip List after each call.

Returns

Zero on success, non-zero on failure.

Definition at line 496 of file test_functional.cpp.

9.15.2.24 test_very_simple_insert()

```
int test_very_simple_insert ( )
```

Test the integrity of a Skip List after a single insert.

Returns

Zero on success, non-zero on failure.

Definition at line 50 of file test_functional.cpp.

9.16 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.h File Reference

Functions

- int test_functional_all ()

  All functional tests.

9.16.1 Function Documentation

9.16.1.1 test_functional_all()

```
int test_functional_all ( )
```

All functional tests.

Returns

Zero on success, non-zero on failure.

Definition at line 714 of file test_functional.cpp.

9.17 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.cpp File Reference

```
#include <iostream>
#include <iomanip>
#include "RollingMedian.h"
#include "TestFramework.h"
#include "test_performance.h"
#include "test_print.h"
#include "../SkipList.h"
```

Functions

- int perf_single_insert_remove ()

  Test the performance of inserting and removing a value into a Skip List.
- int perf_large_skiplist_ins_only ()

  Test the performance of `insert()` to create a large Skip List with no deletion.
- int perf_large_skiplist_ins_rem ()

  Test the performance of `insert()` into a large skiplist then `remove()` many times.
- int perf_single_ins_rem_middle ()

  Create a large skip list then time how long it takes to insert and remove a value at the mid point.
- int perf_single_ins_rem_middle_vary_length ()

  Create a large skip list then time how long it takes to insert and remove a value at the mid point for various length skip lists.
- int perf_single_at_middle ()

  Performance of calling `at()` on the middle value of a 1M long skip list.
- int perf_single_has_middle ()

  Performance of calling `has()` on the middle value of a 1M long skip list.
- int perf_single_ins_at_rem_middle ()

  Performance of calling `insert()`, `at()`, `remove()` on the middle value of a 1M long skip list.

- int perf_median_sliding_window ()

  Performance of calling a rolling median of 100 values in a 10,000 long skip list.
- int perf_1m_median_values ()

  Simulates a rolling median on single vector of 1000000 prices, window size of 101.
- int perf_1m_medians_1000_vectors ()

  Simulates a rolling median on 1000 vectors of 1000 prices, window size of 101.
- int perf_simulate_real_use ()

  Simulates a rolling median on 8000 vectors of 8000 prices, window size of 200.
- int perf_at_in_one_million ()

  Performance of `at()` in various positions on a vector 1m long.
- int perf_has_in_one_million_vary_length ()

  Performance of `has()` in various positions on a vector 1m long.
- int perf_size_of_double_vary_length ()

  Tests the performance of `size_of()` function on a skip list of doubles length up to 1M.
- int perf_size_of_char_vary_length ()

  Tests the performance of `size_of()` function on a skip list of chars length up to 1M.
- int perf_size_of_1m ()

  Tests the `size_of()` function on a skip list of length 1M.
- int perf_index ()

  Tests the `index()` function on a skip list of length 1M.
- int perf_index_vary_length ()

  Tests the `size_of()` function on a skip lists of variable length up to 1M doubles.
- int perf_test_double_insert_remove_value (std::string function, size_t test_count, size_t repeat, size_t sl_↩ length, double value, TestResultS &test_results)
- int perf_test_double_insert_remove_value_begin (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_double_insert_remove_value_mid (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_double_insert_remove_value_end (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_double_at_1m (size_t test_count, size_t repeat, size_t index, TestResultS &test_results)
- int perf_test_double_at_1m_all (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_double_has_1m (size_t test_count, size_t repeat, size_t value, TestResultS &test_results)
- int perf_test_double_has_1m_all (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_double_index_1m (size_t test_count, size_t repeat, size_t value, TestResultS &test_results)
- int perf_test_double_index_1m_all (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_node_height_growth (size_t repeat, TestResultS &test_results)
- int perf_roll_med_by_win_size (size_t repeat, TestResultS &test_results)
- int perf_roll_med_odd_index_wins (size_t repeat, TestResultS &test_results)

  Test the performance of a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::de` Odd length window.
- int perf_roll_med_vector_style_even_win_length (size_t repeat, TestResultS &test_results)

  Test the performance of a simple rolling median using std::vector<double>. Even length window.
- int perf_roll_med_vector_style_odd_win_length (size_t repeat, TestResultS &test_results)

  Test the performance of a simple rolling median using std::vector<double>. Odd length window.
- int perf_roll_med_vector_style_even_win_length_string (size_t repeat, TestResultS &test_results)

  Test the performance of a simple rolling median using std::vector<std::string>. Even length window.
- int perf_test_string_insert_remove_value (std::string function, size_t test_count, size_t repeat, size_t sl_↩ length, size_t str_length, size_t index_for_value, TestResultS &test_results)
- int perf_test_string_insert_remove_value_begin (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_string_insert_remove_value_mid (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_test_string_insert_remove_value_end (size_t test_count, size_t repeat, TestResultS &test_results)
- int perf_skiplist ()

  Runs all the basic performance tests.
- int perf_size ()

Runs all the basic size tests.

- int perf_skiplist_unfair_coin ()

  This function call just tests a subset of the performance. This subset is the one we are interested in when exploring the effect of an unfair coin.

- int test_performance_all ()

  Runs all the performance tests.

Variables

- static int GLOBAL_REPEAT_COUNT = 1000 ∗ 1000

  The number of times to repeat a test to get an accurate performance figure.

9.17.1 Detailed Description

Project: skiplist

Performance tests on a Skip List.

These should have a corresponding test_... which checks correctness. The perf_... version should do no checking but aim for maximum performance however assert(! sl.lacksIntegrity()); is encouraged as that will get stripped out in release builds where performance is really tested.

Created by Paul Ross on 13/07/2016.

Copyright (c) 2016-2023 Paul Ross. All rights reserved.

```
MIT License
Copyright (c) 2016-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```


9.17.2 Function Documentation

9.17.2.1 perf_1m_median_values()

```
int perf_1m_median_values ( )
```

Simulates a rolling median on single vector of 1000000 prices, window size of 101.

This requires an `insert()`, `at()` and `remove()` on the middle value.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 436 of file test_performance.cpp.

9.17.2.2 perf_1m_medians_1000_vectors()

```
int perf_1m_medians_1000_vectors ( )
```

Simulates a rolling median on 1000 vectors of 1000 prices, window size of 101.

This requires an `insert()`, `at()` and `remove()` on the middle value.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 479 of file test_performance.cpp.

9.17.2.3 perf_at_in_one_million()

```
int perf_at_in_one_million ( )
```

Performance of `at()` in various positions on a vector 1m long.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 563 of file test_performance.cpp.

9.17.2.4 perf_has_in_one_million_vary_length()

```
int perf_has_in_one_million_vary_length ( )
```

Performance of `has()` in various positions on a vector 1m long.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 594 of file test_performance.cpp.

9.17.2.5 perf_index()

```
int perf_index ( )
```

Tests the `index()` function on a skip list of length 1M.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 721 of file test_performance.cpp.

9.17.2.6 perf_index_vary_length()

```
int perf_index_vary_length ( )
```

Tests the `size_of()` function on a skip lists of variable length up to 1M doubles.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 748 of file test_performance.cpp.

9.17.2.7 perf_large_skiplist_ins_only()

```
int perf_large_skiplist_ins_only ( )
```

Test the performance of `insert()` to create a large Skip List with no deletion.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 93 of file test_performance.cpp.

9.17.2.8 perf_large_skiplist_ins_rem()

```
int perf_large_skiplist_ins_rem ( )
```

Test the performance of `insert()` into a large skiplist then `remove()` many times.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 130 of file test_performance.cpp.

9.17.2.9 perf_median_sliding_window()

```
int perf_median_sliding_window ( )
```

Performance of calling a rolling median of 100 values in a 10,000 long skip list.

This requires an `insert()`, `at()` and `remove()` on the middle value.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 397 of file test_performance.cpp.

9.17.2.10 perf_roll_med_by_win_size()

```
int perf_roll_med_by_win_size (
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 987 of file test_performance.cpp.

9.17.2.11 perf_roll_med_odd_index_wins()

```
int perf_roll_med_odd_index_wins (
            size_t repeat,
            TestResultS & test_results )
```

Test the performance of a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian:` Odd length window.

Returns

Zero on success, non-zero on failure.

Definition at line 1028 of file test_performance.cpp.

9.17.2.12 perf_roll_med_vector_style_even_win_length()

```
int perf_roll_med_vector_style_even_win_length (
            size_t repeat,
            TestResultS & test_results )
```

Test the performance of a simple rolling median using std::vector<double>. Even length window.

Returns

Zero on success, non-zero on failure.

Definition at line 1067 of file test_performance.cpp.

9.17.2.13 perf_roll_med_vector_style_even_win_length_string()

```
int perf_roll_med_vector_style_even_win_length_string (
            size_t repeat,
            TestResultS & test_results )
```

Test the performance of a simple rolling median using std::vector<std::string>. Even length window.

Returns

Zero on success, non-zero on failure.

Definition at line 1135 of file test_performance.cpp.

9.17.2.14 perf_roll_med_vector_style_odd_win_length()

```
int perf_roll_med_vector_style_odd_win_length (
            size_t repeat,
            TestResultS & test_results )
```

Test the performance of a simple rolling median using std::vector<double>. Odd length window.

Returns

Zero on success, non-zero on failure.

Definition at line 1101 of file test_performance.cpp.

9.17.2.15 perf_simulate_real_use()

```
int perf_simulate_real_use ( )
```

Simulates a rolling median on 8000 vectors of 8000 prices, window size of 200.

This requires an `insert()`, `at()` and `remove()` on the middle value.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 522 of file test_performance.cpp.

9.17.2.16 perf_single_at_middle()

```
int perf_single_at_middle ( )
```

Performance of calling `at()` on the middle value of a 1M long skip list.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 262 of file test_performance.cpp.

9.17.2.17 perf_single_has_middle()

```
int perf_single_has_middle ( )
```

Performance of calling `has()` on the middle value of a 1M long skip list.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 306 of file test_performance.cpp.

9.17.2.18 perf_single_ins_at_rem_middle()

```
int perf_single_ins_at_rem_middle ( )
```

Performance of calling `insert()`, `at()`, `remove()` on the middle value of a 1M long skip list.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 350 of file test_performance.cpp.

9.17.2.19 perf_single_ins_rem_middle()

```
int perf_single_ins_rem_middle ( )
```

Create a large skip list then time how long it takes to insert and remove a value at the mid point.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 172 of file test_performance.cpp.

9.17.2.20 perf_single_ins_rem_middle_vary_length()

```
int perf_single_ins_rem_middle_vary_length ( )
```

Create a large skip list then time how long it takes to insert and remove a value at the mid point for various length skip lists.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 216 of file test_performance.cpp.

9.17.2.21 perf_single_insert_remove()

```
int perf_single_insert_remove ( )
```

Test the performance of inserting and removing a value into a Skip List.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 62 of file test_performance.cpp.

9.17.2.22 perf_size()

```
int perf_size ( )
```

Runs all the basic size tests.

Returns

Zero on success, non-zero on failure.

Definition at line 1306 of file test_performance.cpp.

9.17.2.23 perf_size_of_1m()

```
int perf_size_of_1m ( )
```

Tests the `size_of()` function on a skip list of length 1M.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 690 of file test_performance.cpp.

9.17.2.24 perf_size_of_char_vary_length()

```
int perf_size_of_char_vary_length ( )
```

Tests the performance of `size_of()` function on a skip list of chars length up to 1M.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 657 of file test_performance.cpp.

9.17.2.25 perf_size_of_double_vary_length()

```
int perf_size_of_double_vary_length ( )
```

Tests the performance of `size_of()` function on a skip list of doubles length up to 1M.

This writes the performance figures to `stdout`

Returns

Zero.

Definition at line 624 of file test_performance.cpp.

9.17.2.26 perf_skiplist()

```
int perf_skiplist ( )
```

Runs all the basic performance tests.

Returns

Zero on success, non-zero on failure.

Definition at line 1245 of file test_performance.cpp.

9.17.2.27 perf_skiplist_unfair_coin()

```
int perf_skiplist_unfair_coin ( )
```

This function call just tests a subset of the performance. This subset is the one we are interested in when exploring the effect of an unfair coin.

Returns

Zero on success, non-zero on failure.

Definition at line 1323 of file test_performance.cpp.

9.17.2.28 perf_test_double_at_1m()

```
int perf_test_double_at_1m (
            size_t test_count,
            size_t repeat,
            size_t index,
            TestResultS & test_results )
```

Definition at line 838 of file test_performance.cpp.

9.17.2.29 perf_test_double_at_1m_all()

```
int perf_test_double_at_1m_all (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 868 of file test_performance.cpp.

9.17.2.30 perf_test_double_has_1m()

```
int perf_test_double_has_1m (
            size_t test_count,
            size_t repeat,
            size_t value,
            TestResultS & test_results )
```

Definition at line 879 of file test_performance.cpp.

9.17.2.31 perf_test_double_has_1m_all()

```
int perf_test_double_has_1m_all (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 910 of file test_performance.cpp.

9.17.2.32 perf_test_double_index_1m()

```
int perf_test_double_index_1m (
            size_t test_count,
            size_t repeat,
            size_t value,
            TestResultS & test_results )
```

Definition at line 921 of file test_performance.cpp.

9.17.2.33 perf_test_double_index_1m_all()

```
int perf_test_double_index_1m_all (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 952 of file test_performance.cpp.

9.17.2.34 perf_test_double_insert_remove_value()

```
int perf_test_double_insert_remove_value (
            std::string function,
            size_t test_count,
            size_t repeat,
            size_t sl_length,
            double value,
            TestResultS & test_results )
```

Definition at line 771 of file test_performance.cpp.

9.17.2.35 perf_test_double_insert_remove_value_begin()

```
int perf_test_double_insert_remove_value_begin (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 803 of file test_performance.cpp.

9.17.2.36 perf_test_double_insert_remove_value_end()

```
int perf_test_double_insert_remove_value_end (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 826 of file test_performance.cpp.

9.17.2.37 perf_test_double_insert_remove_value_mid()

```
int perf_test_double_insert_remove_value_mid (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 814 of file test_performance.cpp.

9.17.2.38 perf_test_node_height_growth()

```
int perf_test_node_height_growth (
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 963 of file test_performance.cpp.

9.17.2.39 perf_test_string_insert_remove_value()

```
int perf_test_string_insert_remove_value (
            std::string function,
            size_t test_count,
            size_t repeat,
            size_t sl_length,
            size_t str_length,
            size_t index_for_value,
            TestResultS & test_results )
```

Definition at line 1163 of file test_performance.cpp.

9.17.2.40 perf_test_string_insert_remove_value_begin()

```
int perf_test_string_insert_remove_value_begin (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 1198 of file test_performance.cpp.

9.17.2.41 perf_test_string_insert_remove_value_end()

```
int perf_test_string_insert_remove_value_end (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 1224 of file test_performance.cpp.

9.17.2.42 perf_test_string_insert_remove_value_mid()

```
int perf_test_string_insert_remove_value_mid (
            size_t test_count,
            size_t repeat,
            TestResultS & test_results )
```

Definition at line 1211 of file test_performance.cpp.

9.17.2.43 test_performance_all()

```
int test_performance_all ( )
```

Runs all the performance tests.

Returns

Zero on success, non-zero on failure.

Definition at line 1340 of file test_performance.cpp.

9.17.3 Variable Documentation

9.17.3.1 GLOBAL_REPEAT_COUNT

```
int GLOBAL_REPEAT_COUNT = 1000 * 1000  [static]
```

The number of times to repeat a test to get an accurate performance figure.

Definition at line 53 of file test_performance.cpp.

9.18 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.h File Reference

Functions

- int test_performance_all ()

  Runs all the performance tests.

9.18.1 Function Documentation

9.18.1.1 test_performance_all()

```
int test_performance_all ( )
```

Runs all the performance tests.

Returns

Zero on success, non-zero on failure.

Definition at line 1340 of file test_performance.cpp.

9.19 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.cpp File Reference

```
#include <iostream>
#include <iomanip>
#include "test_print.h"
```

Functions

- int print_result (const char *fn_name, int result)

Variables

- const int FUNCTION_WIDTH = 50

9.19.1 Detailed Description

skiplist

Concurrency Tests.

Created by Paul Ross on 13/07/2016. Copyright (c) 2016-2023 Paul Ross. All rights reserved.

MIT License

Copyright (c) 2016-2023 Paul Ross

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

9.19.2 Function Documentation

9.19.2.1 print_result()

```
int print_result (
            const char * fn_name,
            int result )
```

Print the result of a test function.

Parameters

fn_name	Function name.
result	The result of the function. If $<$0 this is "N/A", if 0 success else failure.

Returns

result $>$ 0.

Definition at line 54 of file test_print.cpp.

9.19.3 Variable Documentation

9.19.3.1 FUNCTION_WIDTH

```
const int FUNCTION_WIDTH = 50
```

Definition at line 45 of file test_print.cpp.

9.20 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.h File Reference

Functions

- int print_result (const char $*$fn_name, int result)

Variables

- const int FUNCTION_WIDTH

9.20.1 Function Documentation

9.20.1.1 print_result()

```
int print_result (
            const char * fn_name,
            int result )
```

Print the result of a test function.

Parameters

fn_name	Function name.
result	The result of the function. If <0 this is "N/A", if 0 success else failure.

Returns

result > 0.

Definition at line 54 of file test_print.cpp.

9.20.2 Variable Documentation

9.20.2.1 FUNCTION_WIDTH

```
const int FUNCTION_WIDTH  [extern]
```

Definition at line 45 of file test_print.cpp.

9.21 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.cpp File Reference

```
#include <iostream>
#include <iomanip>
#include "test_rolling_median.h"
#include "test_print.h"
#include "TestFramework.h"
#include "RollingMedian.h"
```

Functions

- int test_roll_med_simple ()

  Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`.
- int test_roll_med_even_win ()

  Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`. *Even length window.*
- int test_roll_med_even_mean ()

  Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`. *Even length window.*
- int perf_roll_med_odd_index ()

  Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`. *Odd length window.*
- int test_rolling_median_all ()

  Test all rolling median tests. Performance tests are in src/cpp/test/test_performance.cpp.

9.21.1 Function Documentation

9.21.1.1 perf_roll_med_odd_index()

```
int perf_roll_med_odd_index ( )
```

Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`. Odd length window.

Returns

Zero on success, non-zero on failure.

Definition at line 124 of file test_rolling_median.cpp.

9.21.1.2 test_roll_med_even_mean()

```
int test_roll_med_even_mean ( )
```

Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`. Even length window.

Returns

Zero on success, non-zero on failure.

Definition at line 92 of file test_rolling_median.cpp.

9.21.1.3 test_roll_med_even_win()

```
int test_roll_med_even_win ( )
```

Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`. Even length window.

Returns

Zero on success, non-zero on failure.

Definition at line 59 of file test_rolling_median.cpp.

9.21.1.4 test_roll_med_simple()

```
int test_roll_med_simple ( )
```

Test a simple rolling median into a dynamic array of doubles given by `OrderedStructs::RollingMedian::dest_size`.

Returns

Zero on success, non-zero on failure.

Definition at line 26 of file test_rolling_median.cpp.

9.21.1.5 test_rolling_median_all()

```
int test_rolling_median_all ( )
```

Test all rolling median tests. Performance tests are in src/cpp/test/test_performance.cpp.

Returns

Zero on success, non-zero on failure.

Definition at line 161 of file test_rolling_median.cpp.

9.22 /Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.h File Reference

Functions

- int test_rolling_median_all ()

  Test all rolling median tests. Performance tests are in src/cpp/test/test_performance.cpp.

9.22.1 Function Documentation

9.22.1.1 test_rolling_median_all()

```
int test_rolling_median_all ( )
```

Test all rolling median tests. Performance tests are in src/cpp/test/test_performance.cpp.

Returns

Zero on success, non-zero on failure.

Definition at line 161 of file test_rolling_median.cpp.

9.23 /Users/paulross/CLionProjects/skiplist/src/cpp/test/Test↩ Framework.cpp File Reference

```
#include "TestFramework.h"
#include <iomanip>
#include <cmath>
#include <cassert>
#include <sstream>
#include "save_stream_state.h"
```

Macros

- #define REGEX_SPACE_INTEGER "\\s+(\\d+)"
- #define REGEX_SPACE_FLOAT "\\s+([0-9+-.]+)"
- #define REGEX_SPACE_STRING_NO_SPACE "\\s+(\\S+)"
- #define REGEX_SPACE_ANYTHING "\\s+(.+)"

Functions

- std::ostream & operator<< (std::ostream &os, const TestResult &result)
- std::ostream & operator<< (std::ostream &os, const TestResultS &results)
- std::string unique_string (int width)
- std::string random_string (ssize_t length)
- std::u16string unique_u16string (int width)
- std::u32string unique_u32string (int width)
- std::vector< char > unique_vector_char (int width)
- size_t count_of_unique_string ()
- void reset_count_of_unique_string ()

Variables

- static const int TIME_PRECISION = 9
- static const int TIME_WIDTH = 16
- static size_t str_count = 0

9.23.1 Macro Definition Documentation

9.23.1.1 REGEX_SPACE_ANYTHING

```
#define REGEX_SPACE_ANYTHING "\\s+(.+)"
```

Definition at line 234 of file TestFramework.cpp.

9.23.1.2 REGEX_SPACE_FLOAT

```
#define REGEX_SPACE_FLOAT "\\s+([0-9+-.]+)"
```

Definition at line 232 of file TestFramework.cpp.

9.23.1.3 REGEX_SPACE_INTEGER

```
#define REGEX_SPACE_INTEGER "\\s+(\\d+)"
```

Definition at line 231 of file TestFramework.cpp.

9.23.1.4 REGEX_SPACE_STRING_NO_SPACE

```
#define REGEX_SPACE_STRING_NO_SPACE "\\s+(\\S+)"
```

Definition at line 233 of file TestFramework.cpp.

9.23.2 Function Documentation

9.23.2.1 count_of_unique_string()

```
size_t count_of_unique_string ( )
```

Returns the number of unique strings created.

Returns

Number of unique strings created.

Definition at line 423 of file TestFramework.cpp.

9.23.2.2 operator<<() [1/2]

```
std::ostream& operator<< (
            std::ostream & os,
            const TestResult & result )
```

9.23.2.3 Note on The Output

Example output (with header and $ gnuplot guide).

```
#HEAD: Fail   Scale  Repeat         Mean(s)     Std.Dev.(s)         Min.(s)         Max.(s)     Count
       Rate(/s) Name
#  $1   $2      $3      $4              $5              $6              $7              $8        $9
         $10 $11
TEST:    0      99       4     2.500000000     1.118033989     1.000000000     4.000000000       400
       160.0 test_internal_test_result_string
```

In this case we have repeated the tests four times, each of these tests has a timed operation (the 'atomic' test) that is repeated 100 times for each individual execution time.

The total time to run all of these tests is: Repeat ∗ Mean in seconds, in this case 10.0 (s). The mean time for each atomic test is: Repeat ∗ Mean / Count, in this case 10.0 / 400 = 0.025 (s) The minimum atomic time is: Repeat ∗ Min.(s) / Count, in this case 4 ∗ 1.0 / 400 = 0.010 (s) The maximum atomic time is: Repeat ∗ Max.(s) / Count, in this case 4 ∗ 4.0 / 400 = 0.040 (s) The standard deviation atomic time is: Repeat ∗ Std.Dev.(s) / Count, in this case 4 ∗ 1.11 / 400 = +/- 0.0111 (s)

Parameters

os	The stream to write to.
result	The tests result.

Returns

The stream for chaining.

Definition at line 181 of file TestFramework.cpp.

9.23.2.4 operator<<() [2/2]

```
std::ostream& operator<< (
            std::ostream & os,
            const TestResultS & results )
```

Definition at line 311 of file TestFramework.cpp.

9.23.2.5 random_string()

```
std::string random_string (
            ssize_t length )
```

Creates a random `std::string` filled with the character set 32...126 (printable characters) inclusive.

Parameters

length	The string width.

Returns

The random `std::string`.

Definition at line 365 of file TestFramework.cpp.

9.23.2.6 reset_count_of_unique_string()

```
void reset_count_of_unique_string ( )
```

Reset the string counter.

Definition at line 430 of file TestFramework.cpp.

9.23.2.7 unique_string()

```
std::string unique_string (
            int width )
```

Creates a unique `std::string`. This starts as "0", "1" ...

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::string`.

Definition at line 348 of file TestFramework.cpp.

9.23.2.8 unique_u16string()

```
std::u16string unique_u16string (
            int width )
```

Creates a unique `std::u16string`. This starts as "0", "1" ...

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::u16string`.

Definition at line 383 of file TestFramework.cpp.

9.23.2.9 unique_u32string()

```
std::u32string unique_u32string (
            int width )
```

Creates a unique `std::u32string`. This starts as "0", "1" ...

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::u32string`.

Definition at line 398 of file TestFramework.cpp.

9.23.2.10 unique_vector_char()

```
std::vector<char> unique_vector_char (
            int width )
```

Creates a unique `std::vector<char>`.

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::vector<char>`.

Definition at line 413 of file TestFramework.cpp.

9.23.3 Variable Documentation

9.23.3.1 str_count

```
size_t str_count = 0  [static]
```

Definition at line 340 of file TestFramework.cpp.

9.23.3.2 TIME_PRECISION

```
const int TIME_PRECISION = 9  [static]
```

Definition at line 154 of file TestFramework.cpp.

9.23.3.3 TIME_WIDTH

```
const int TIME_WIDTH = 16  [static]
```

Definition at line 155 of file TestFramework.cpp.

9.24 /Users/paulross/CLionProjects/skiplist/src/cpp/test/Test↩ Framework.h File Reference

```
#include <list>
#include <map>
#include <string>
#include <vector>
#include <iostream>
#include <chrono>
```

Classes

- class ExecClock
- class TestResult
- struct TestResult::test_result
- class TestResultS
- class SubTestCount

Functions

- std::ostream & operator<< (std::ostream &os, const TestResult &result)
- std::ostream & operator<< (std::ostream &os, const TestResultS &results)
- std::string unique_string (int width=0)
- std::string random_string (ssize_t length)
- std::u16string unique_u16string (int width=0)
- std::u32string unique_u32string (int width=0)
- std::vector< char > unique_vector_char (int width)
- size_t count_of_unique_string ()
- void reset_count_of_unique_string ()

9.24.1 Function Documentation

9.24.1.1 count_of_unique_string()

```
size_t count_of_unique_string ( )
```

Returns the number of unique strings created.

Returns

Number of unique strings created.

Definition at line 423 of file TestFramework.cpp.

9.24.1.2 operator<<() [1/2]

```
std::ostream& operator<< (
            std::ostream & os,
            const TestResult & result )
```

9.24.1.3 Note on The Output

Example output (with header and $ gnuplot guide).

```
#HEAD: Fail   Scale  Repeat         Mean(s)     Std.Dev.(s)         Min.(s)         Max.(s)     Count
       Rate(/s) Name
#  $1   $2      $3      $4              $5              $6              $7              $8        $9
        $10 $11
TEST:    0       99       4     2.500000000     1.118033989     1.000000000     4.000000000       400
       160.0 test_internal_test_result_string
```

In this case we have repeated the tests four times, each of these tests has a timed operation (the 'atomic' test) that is repeated 100 times for each individual execution time.

The total time to run all of these tests is: Repeat ∗ Mean in seconds, in this case 10.0 (s). The mean time for each atomic test is: Repeat ∗ Mean / Count, in this case 10.0 / 400 = 0.025 (s) The minimum atomic time is: Repeat ∗ Min.(s) / Count, in this case 4 ∗ 1.0 / 400 = 0.010 (s) The maximum atomic time is: Repeat ∗ Max.(s) / Count, in this case 4 ∗ 4.0 / 400 = 0.040 (s) The standard deviation atomic time is: Repeat ∗ Std.Dev.(s) / Count, in this case 4 ∗ 1.11 / 400 = +/- 0.0111 (s)

Parameters

os	The stream to write to.
result	The tests result.

Returns

The stream for chaining.

Definition at line 181 of file TestFramework.cpp.

9.24.1.4 operator<<() [2/2]

```
std::ostream& operator<< (
            std::ostream & os,
            const TestResultS & results )
```

Definition at line 311 of file TestFramework.cpp.

9.24.1.5 random_string()

```
std::string random_string (
            ssize_t length )
```

Creates a random `std::string` filled with the character set 32...126 (printable characters) inclusive.

Parameters

length	The string width.

Returns

The random `std::string`.

Definition at line 365 of file TestFramework.cpp.

9.24.1.6 reset_count_of_unique_string()

```
void reset_count_of_unique_string ( )
```

Reset the string counter.

Definition at line 430 of file TestFramework.cpp.

9.24.1.7 unique_string()

```
std::string unique_string (
            int width )
```

Creates a unique `std::string`. This starts as "0", "1" ...

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::string`.

Definition at line 348 of file TestFramework.cpp.

9.24.1.8 unique_u16string()

```
std::u16string unique_u16string (
            int width )
```

Creates a unique `std::u16string`. This starts as "0", "1" ...

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::u16string`.

Definition at line 383 of file TestFramework.cpp.

9.24.1.9 unique_u32string()

```
std::u32string unique_u32string (
            int width )
```

Creates a unique `std::u32string`. This starts as "0", "1" ...

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::u32string`.

Definition at line 398 of file TestFramework.cpp.

9.24.1.10 unique_vector_char()

```
std::vector<char> unique_vector_char (
            int width )
```

Creates a unique `std::vector<char>`.

Parameters

width	If > 0 the string will be at least this width.

Returns

The unique `std::vector<char>`.

Definition at line 413 of file TestFramework.cpp.

9.25 /Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.cpp File Reference

```
#include <assert.h>
#include <sstream>
#include <stdexcept>
#include "cmpPyObject.h"
```

9.25.1 Detailed Description

Project: skiplist

Created by Paul Ross on 04/10/2016.

Copyright (c) 2016-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2016-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

9.26 /Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.h File Reference

```
#include "Python.h"
#include <functional>
```

Classes

- class cmpPyObject

  A wrapper around a CPython comparison function.

Macros

- #define SKIPLIST_TRACE_CMPPYOBJECT_REF_COUNTS 0

9.26.1 Detailed Description

Project: skiplist

Created by Paul Ross on 04/10/2016.

Copyright (c) 2016-2023 Paul Ross. All rights reserved.

```
MIT License
Copyright (c) 2016-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```


9.26.2 Macro Definition Documentation

9.26.2.1 SKIPLIST_TRACE_CMPPYOBJECT_REF_COUNTS

```
#define SKIPLIST_TRACE_CMPPYOBJECT_REF_COUNTS 0
```

Python 3.8 changed how lambda refcounts were managed and our tests with lambdas were segfaulting.

The tests with normal functions worked fine. This was because we were not:

- Incrementing the reference count in the ctor.
- Decrementing the reference count in the dtor (we hadn't one).
- Adjusting the reference counts correctly in the default copy constructor.
- Implementing the default copy constructor that manages reference counts.

We must of got away with it up to now as previous versions of Python treated lambdas differently.

Definition at line 56 of file cmpPyObject.h.

9.27 /Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.cpp File Reference

```
#include <Python.h>
#include <limits.h>
#include "SkipList.h"
#include "cOrderedStructs.h"
#include "cSkipList.h"
```

Classes

- struct module_state

  Wrapper of module state for Python 2 and 3.

Macros

- #define GETSTATE(m) (&_state)

Functions

- static PyObject ∗ toss_coin (PyObject ∗)
- static PyObject ∗ seed_rand (PyObject ∗, PyObject ∗arg)
- static PyObject ∗ long_min_value (PyObject ∗)
- static PyObject ∗ long_max_value (PyObject ∗)
- PyMODINIT_FUNC initorderedstructs (void)

Variables

- static char toss_coin_docs [ ]
- static char seed_rand_docs [ ]
- static PyMethodDef orderedstructsmodule_methods [ ]
- static char c_skip_list_docs [ ]
- static const char ∗ MODULE_VERSION = "0.4.5"
- static const char ∗ MODULE_BUILD_DOCS = "Standard build."
- static struct module_state _state

9.27.1 Macro Definition Documentation

9.27.1.1 GETSTATE

```
#define GETSTATE(
            m ) (&_state)
```

Definition at line 110 of file cOrderedStructs.cpp.

9.27.2 Function Documentation

9.27.2.1 initorderedstructs()

```
PyMODINIT_FUNC initorderedstructs (
            void  )
```

Definition at line 146 of file cOrderedStructs.cpp.

9.27.2.2 long_max_value()

```
static PyObject* long_max_value (
            PyObject *  )  [static]
```

Definition at line 70 of file cOrderedStructs.cpp.

9.27.2.3 long_min_value()

```
static PyObject* long_min_value (
            PyObject *  )  [static]
```

Definition at line 65 of file cOrderedStructs.cpp.

9.27.2.4 seed_rand()

```
static PyObject* seed_rand (
            PyObject * ,
            PyObject * arg )  [static]
```

Seed the random number generator.

Definition at line 37 of file cOrderedStructs.cpp.

9.27.2.5 toss_coin()

```
static PyObject* toss_coin (
            PyObject *  )  [static]
```

Virtual toss of a coin.

Definition at line 25 of file cOrderedStructs.cpp.

9.27.3 Variable Documentation

9.27.3.1 _state

```
struct module_state _state  [static]
```

Start of module initialisation.

Definition at line 103 of file cOrderedStructs.cpp.

9.27.3.2 c_skip_list_docs

```
char c_skip_list_docs[]  [static]
```

Initial value:

```
=
        "orderedstructs is an interface between Python and a C++ skip list implementation. It contains:"
        "\nSkipList - An implementation of a skip list for float/long/bytes or objects."
        "\nseed_rand(int) - Seed the random number generator."
        "\ntoss_coin() - Toss a coin using the random number generator and return True/False."
```

Definition at line 87 of file cOrderedStructs.cpp.

9.27.3.3 MODULE_BUILD_DOCS

```
const char* MODULE_BUILD_DOCS = "Standard build."  [static]
```

Definition at line 103 of file cOrderedStructs.cpp.

9.27.3.4 MODULE_VERSION

```
const char* MODULE_VERSION = "0.4.5"  [static]
```

Definition at line 101 of file cOrderedStructs.cpp.

9.27.3.5 orderedstructsmodule_methods

```
PyMethodDef orderedstructsmodule_methods[]  [static]
```

Initial value:

```
= {
        {"toss_coin", (PyCFunction) toss_coin,      METH_NOARGS, toss_coin_docs},
        {"seed_rand", (PyCFunction) seed_rand,      METH_O,      seed_rand_docs},
        {"min_long",  (PyCFunction) long_min_value, METH_NOARGS,
                                                                 "Minimum value I can handle for an
       integer."},
        {"max_long",  (PyCFunction) long_max_value, METH_NOARGS,
                                                                 "Maximum value I can handle for an
       integer."},
        {NULL, NULL, 0, NULL}
}
```

CPython orderedstructs package methods.

Definition at line 77 of file cOrderedStructs.cpp.

9.27.3.6 seed_rand_docs

```
char seed_rand_docs[]  [static]
```

Initial value:

```
=
"Seed the random number generator."
" This calls OrderedStructs::SkipList::seedRand()."
```

Definition at line 29 of file cOrderedStructs.cpp.

9.27.3.7 toss_coin_docs

```
char toss_coin_docs[]  [static]
```

Initial value:

```
=
"Toss a coin and return True/False."
" This calls OrderedStructs::SkipList::tossCoin()."
```

Definition at line 17 of file cOrderedStructs.cpp.

9.28 /Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.h File Reference

Macros

- #define ORDERED_STRUCTS_MODULE_NAME "orderedstructs"

  CPython package name.

9.28.1 Macro Definition Documentation

9.28.1.1 ORDERED_STRUCTS_MODULE_NAME

```
#define ORDERED_STRUCTS_MODULE_NAME "orderedstructs"
```

CPython package name.

Definition at line 13 of file cOrderedStructs.h.

9.29 /Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.cpp File Reference

```
#include <Python.h>
#include "structmember.h"
#include "pythread.h"
#include "SkipList.h"
#include <string>
#include <iostream>
#include <stdexcept>
#include "OrderedStructs.h"
#include "cOrderedStructs.h"
#include "cmpPyObject.h"
```

Classes

- struct SkipList

  Contains a CPython Skip List that is capable of holding int/float/bytes/object types.
- class AcquireLock

  A CPython RAII wrapper around the PyThread_type_lock in a SkipList.

Macros

- #define INCLUDE_METHODS_THAT_USE_STREAMS

Functions

- static PyObject * SkipList_new (PyTypeObject *type, PyObject *, PyObject *)
- static int SkipList_init (SkipList *self, PyObject *args, PyObject *kwargs)
- static void decref_all_contents (SkipList *self)
- static void SkipList_dealloc (SkipList *self)
- static PyObject * has_object (SkipList *self, PyObject *arg)
- static PyObject * SkipList_has (SkipList *self, PyObject *arg)
- static int get_size (SkipList *self, Py_ssize_t &value)
- static int check_index_against_size (const char *prefix, long index, Py_ssize_t size)
- static PyObject * SkipList_at (SkipList *self, PyObject *arg)
- static PyObject * at_sequence_long (SkipList *self, Py_ssize_t index, Py_ssize_t count)
- static PyObject * at_sequence_double (SkipList *self, Py_ssize_t index, Py_ssize_t count)
- static PyObject * at_sequence_bytes (SkipList *self, Py_ssize_t index, Py_ssize_t count)
- static PyObject * at_sequence_object (SkipList *self, Py_ssize_t index, Py_ssize_t count)
- static PyObject * SkipList_at_sequence (SkipList *self, PyObject *args, PyObject *kwargs)
- static PyObject * index_object (SkipList *self, PyObject *arg)
- static PyObject * SkipList_index (SkipList *self, PyObject *arg)
- Py_ssize_t SkipList_length (PyObject *self)
- static PyObject * SkipList_size (SkipList *self)
- static PyObject * SkipList_height (SkipList *self)
- static PyObject * SkipList_insert (SkipList *self, PyObject *arg)
- static PyObject * remove_long (SkipList *self, PyObject *arg)
- static PyObject * remove_double (SkipList *self, PyObject *arg)
- static PyObject * remove_bytes (SkipList *self, PyObject *arg)
- static PyObject * remove_object (SkipList *self, PyObject *arg)
- static PyObject * SkipList_remove (SkipList *self, PyObject *arg)
- static PyObject * SkipList_dot_file (SkipList *self)
- static PyObject * SkipList_lacks_integrity (SkipList *self)
- static PyObject * SkipList_node_height (SkipList *self, PyObject *args, PyObject *kwargs)
- static PyObject * SkipList_node_width (SkipList *self, PyObject *args, PyObject *kwargs)
- static PyObject * SkipList_size_of (SkipList *self)

Variables

- static PyMemberDef SkipList_members [ ]
- static PyMethodDef SkipList_methods [ ]
- static PySequenceMethods SkipList_SequenceMethods
- static char py_skip_list_docs [ ]
- PyTypeObject SkipListType

9.29.1 Detailed Description

Project: skiplist

Concurrency Tests.

Created by Paul Ross on 20/11/2015.

Copyright (c) 2015-2023 Paul Ross. All rights reserved.
MIT License
Copyright (c) 2015-2023 Paul Ross
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights

to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

9.29.2 Note about Python thread safety

9.29.2.1 Non-PyObjects's

For skip lists storing non-native Python objects, for example those stored as C++ int / double / std::string This is thread safe.

9.29.2.2 PyObjects's

For skip lists that store native Python objects as a PyObject∗ the comparison function will be either using PyObject_RichCompareBool (the default) or some user supplied comparison function.

In either case these will call into arbitrary Python code such as **lt**() and this could be preempted by another Python thread that modifies this skip list whilst this thread is trying to do the same. Even immutable C functions here (like has()) should not be preempted.

To prevent that we must acquire the GIL with any C function here that might cause the Python comparison function to be used. Once the value of the functon is known the GIL must be released.

If Python is compiled with HAS_THREADS then the Skiplist object gains a PyThread_type_lock and we use a class AcquireLock on the Skiplist to mange this lock.

9.29.3 Macro Definition Documentation

9.29.3.1 INCLUDE_METHODS_THAT_USE_STREAMS

```
#define INCLUDE_METHODS_THAT_USE_STREAMS
```

Definition at line 77 of file cSkipList.cpp.

9.29.4 Function Documentation

9.29.4.1 at_sequence_bytes()

```
static PyObject* at_sequence_bytes (
            SkipList * self,
            Py_ssize_t index,
            Py_ssize_t count )  [static]
```

Definition at line 631 of file cSkipList.cpp.

9.29.4.2 at_sequence_double()

```
static PyObject* at_sequence_double (
            SkipList * self,
            Py_ssize_t index,
            Py_ssize_t count )  [static]
```

Definition at line 604 of file cSkipList.cpp.

9.29.4.3 at_sequence_long()

```
static PyObject* at_sequence_long (
            SkipList * self,
            Py_ssize_t index,
            Py_ssize_t count )  [static]
```

Definition at line 577 of file cSkipList.cpp.

9.29.4.4 at_sequence_object()

```
static PyObject* at_sequence_object (
            SkipList * self,
            Py_ssize_t index,
            Py_ssize_t count )  [static]
```

Definition at line 658 of file cSkipList.cpp.

9.29.4.5 check_index_against_size()

```
static int check_index_against_size (
            const char * prefix,
            long index,
            Py_ssize_t size )  [static]
```

Definition at line 470 of file cSkipList.cpp.

9.29.4.6 decref_all_contents()

```
static void decref_all_contents (
            SkipList * self )  [static]
```

Definition at line 285 of file cSkipList.cpp.

9.29.4.7 get_size()

```
static int get_size (
            SkipList * self,
            Py_ssize_t & value )  [static]
```

Definition at line 438 of file cSkipList.cpp.

9.29.4.8 has_object()

```
static PyObject* has_object (
            SkipList * self,
            PyObject * arg )  [static]
```

Returns the result of .has() for native Python objects or NULL on failure (such a failed comparison between types). As this calls into arbitrary Python code such as **lt**() for comparison operations the GIL has to be claimed for the duration of this function and released once the result of the function is known.

Parameters

self	
arg	

Returns

Definition at line 343 of file cSkipList.cpp.

9.29.4.9 index_object()

```
static PyObject* index_object (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 769 of file cSkipList.cpp.

9.29.4.10 remove_bytes()

```
static PyObject* remove_bytes (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 1054 of file cSkipList.cpp.

9.29.4.11 remove_double()

```
static PyObject* remove_double (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 1030 of file cSkipList.cpp.

9.29.4.12 remove_long()

```
static PyObject* remove_long (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 1008 of file cSkipList.cpp.

9.29.4.13 remove_object()

```
static PyObject* remove_object (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 1074 of file cSkipList.cpp.

9.29.4.14 SkipList_at()

```
static PyObject* SkipList_at (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 487 of file cSkipList.cpp.

9.29.4.15 SkipList_at_sequence()

```
static PyObject* SkipList_at_sequence (
            SkipList * self,
            PyObject * args,
            PyObject * kwargs )  [static]
```

Definition at line 687 of file cSkipList.cpp.

9.29.4.16 SkipList_dealloc()

```
static void SkipList_dealloc (
            SkipList * self )  [static]
```

Definition at line 295 of file cSkipList.cpp.

9.29.4.17 SkipList_dot_file()

```
static PyObject* SkipList_dot_file (
            SkipList * self )  [static]
```

Definition at line 1129 of file cSkipList.cpp.

9.29.4.18 SkipList_has()

```
static PyObject* SkipList_has (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 361 of file cSkipList.cpp.

9.29.4.19 SkipList_height()

```
static PyObject* SkipList_height (
            SkipList * self )  [static]
```

Definition at line 912 of file cSkipList.cpp.

9.29.4.20 SkipList_index()

```
static PyObject* SkipList_index (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 788 of file cSkipList.cpp.

9.29.4.21 SkipList_init()

```
static int SkipList_init (
            SkipList * self,
            PyObject * args,
            PyObject * kwargs )  [static]
```

Initialise a CPython Skip List type.

Parameters

self	The CPython Skip List object.
args	The arguments, as a minimum a CPython type, optionally with a comparison function.
kwargs	Keyword arguments: "value_type", "cmp_func".

Returns

0 on success.

Definition at line 189 of file cSkipList.cpp.

9.29.4.22 SkipList_insert()

```
static PyObject* SkipList_insert (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 941 of file cSkipList.cpp.

9.29.4.23 SkipList_lacks_integrity()

```
static PyObject* SkipList_lacks_integrity (
            SkipList * self )  [static]
```

Definition at line 1165 of file cSkipList.cpp.

9.29.4.24 SkipList_length()

```
Py_ssize_t SkipList_length (
            PyObject * self )
```

Definition at line 873 of file cSkipList.cpp.

9.29.4.25 SkipList_new()

```
static PyObject* SkipList_new (
            PyTypeObject * type,
            PyObject * ,
            PyObject *  )  [static]
```

Create a new CPython Skip List type.

Parameters

type	The CPython type.

Returns

A new CPython Skip List type, uninitialised.

Definition at line 166 of file cSkipList.cpp.

9.29.4.26 SkipList_node_height()

```
static PyObject* SkipList_node_height (
            SkipList * self,
            PyObject * args,
            PyObject * kwargs )  [static]
```

Definition at line 1195 of file cSkipList.cpp.

9.29.4.27 SkipList_node_width()

```
static PyObject* SkipList_node_width (
            SkipList * self,
            PyObject * args,
            PyObject * kwargs )  [static]
```

Definition at line 1254 of file cSkipList.cpp.

9.29.4.28 SkipList_remove()

```
static PyObject* SkipList_remove (
            SkipList * self,
            PyObject * arg )  [static]
```

Definition at line 1100 of file cSkipList.cpp.

9.29.4.29 SkipList_size()

```
static PyObject* SkipList_size (
            SkipList * self )  [static]
```

Definition at line 883 of file cSkipList.cpp.

9.29.4.30 SkipList_size_of()

```
static PyObject* SkipList_size_of (
            SkipList * self )  [static]
```

Definition at line 1334 of file cSkipList.cpp.

9.29.5 Variable Documentation

9.29.5.1 py_skip_list_docs

```
char py_skip_list_docs[]  [static]
```

Initial value:

```
=
        "SkipList - An implementation of a skip list for floats, longs, bytes and"
        " Python objects."
```

Definition at line 1435 of file cSkipList.cpp.

9.29.5.2 SkipList_members

```
PyMemberDef SkipList_members[]  [static]
```

Initial value:

```
= {
        {NULL, 0, 0, 0, NULL}
}
```

Definition at line 326 of file cSkipList.cpp.

9.29.5.3 SkipList_methods

```
PyMethodDef SkipList_methods[]  [static]
```

Definition at line 1362 of file cSkipList.cpp.

9.29.5.4 SkipList_SequenceMethods

```
PySequenceMethods SkipList_SequenceMethods  [static]
```

Initial value:

```
= {
        &SkipList_length,
        0,
        0,
        0,
        0,
        0,
        0,
        0,
}
```

Definition at line 1420 of file cSkipList.cpp.

9.29.5.5 SkipListType

```
PyTypeObject SkipListType
```

Initial value:

```
= {
        .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
        .tp_name = ORDERED_STRUCTS_MODULE_NAME ".SkipList",
        .tp_basicsize = sizeof(SkipList),
        .tp_dealloc = (destructor) SkipList_dealloc,

        .tp_as_sequence = &SkipList_SequenceMethods,
        .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
        .tp_doc = py_skip_list_docs,
        .tp_methods = SkipList_methods,
        .tp_members = SkipList_members,
        .tp_init = (initproc) SkipList_init,
        .tp_new = SkipList_new
}
```

Definition at line 1439 of file cSkipList.cpp.

9.30 /Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.h File Reference

Variables

- PyTypeObject SkipListType

9.30.1 Variable Documentation

9.30.1.1 SkipListType

```
PyTypeObject SkipListType  [extern]
```

Definition at line 1439 of file cSkipList.cpp.

9.31 /Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.cpp File Reference

```
#include "OrderedStructs.h"
```

Functions

- std::string bytes_as_std_string (PyObject ∗arg)
- PyObject ∗ std_string_as_bytes (const std::string &str)

9.31.1 Function Documentation

9.31.1.1 bytes_as_std_string()

```
std::string bytes_as_std_string (
            PyObject * arg )
```

Returns a std::string from the contents of a bytes object. This includes any '\0' characters in the content.

Definition at line 14 of file OrderedStructs.cpp.

9.31.1.2 std_string_as_bytes()

```
PyObject* std_string_as_bytes (
            const std::string & str )
```

Returns a new PyBytesObject from a std::string. This includes any '\0' characters in the content.

Definition at line 22 of file OrderedStructs.cpp.

9.32 /Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.h File Reference

```
#include <Python.h>
#include <string>
```

Macros

- #define ASSERT_TYPE_IN_RANGE assert(self->_data_type > TYPE_ZERO && self->_data_type < TYPE_OVERFLOW)

Typedefs

- typedef long long TYPE_TYPE_LONG
- typedef double TYPE_TYPE_DOUBLE
- typedef std::string TYPE_TYPE_BYTES
- typedef PyObject * TYPE_TYPE_OBJECT

Enumerations

- enum KeyDataType {
  TYPE_ZERO = 0 , TYPE_LONG , TYPE_DOUBLE , TYPE_BYTES ,
  TYPE_OBJECT , TYPE_OVERFLOW }

Functions

- std::string bytes_as_std_string (PyObject *arg)
- PyObject * std_string_as_bytes (const std::string &str)

9.32.1 Macro Definition Documentation

9.32.1.1 ASSERT_TYPE_IN_RANGE

```
#define ASSERT_TYPE_IN_RANGE assert(self->_data_type > TYPE_ZERO && self->_data_type < TYPE_OVERFLOW)
```

Range assert macro

Definition at line 41 of file OrderedStructs.h.

9.32.2 Typedef Documentation

9.32.2.1 TYPE_TYPE_BYTES

```
typedef std::string TYPE_TYPE_BYTES
```

Note: We use a std::string but store bytes in it. So we have to be careful not to treat data as a NTS.

Definition at line 36 of file OrderedStructs.h.

9.32.2.2 TYPE_TYPE_DOUBLE

```
typedef double TYPE_TYPE_DOUBLE
```

Define these as an attempt to reduce C&P errors in very similar code.

Definition at line 32 of file OrderedStructs.h.

9.32.2.3 TYPE_TYPE_LONG

```
typedef long long TYPE_TYPE_LONG
```

Define these as an attempt to reduce C&P errors in very similar code.

Definition at line 30 of file OrderedStructs.h.

9.32.2.4 TYPE_TYPE_OBJECT

```
typedef PyObject* TYPE_TYPE_OBJECT
```

A Python object.

Definition at line 38 of file OrderedStructs.h.

9.32.3 Enumeration Type Documentation

9.32.3.1 KeyDataType

```
enum KeyDataType
```

Roll our own RTTI for specific types.

Enumerator

TYPE_ZERO	
TYPE_LONG	
TYPE_DOUBLE	
TYPE_BYTES	
TYPE_OBJECT	
TYPE_OVERFLOW	

Definition at line 19 of file OrderedStructs.h.

9.32.4 Function Documentation

9.32.4.1 bytes_as_std_string()

```
std::string bytes_as_std_string (
            PyObject * arg )
```

Returns a std::string from the contents of a bytes object. This includes any '\0' characters in the content.

Definition at line 14 of file OrderedStructs.cpp.

9.32.4.2 std_string_as_bytes()

```
PyObject* std_string_as_bytes (
            const std::string & str )
```

Returns a new PyBytesObject from a std::string. This includes any '\0' characters in the content.

Definition at line 22 of file OrderedStructs.cpp.

Index

/Users/paulross/CLionProjects/skiplist/src/cpp/HeadNode.h, 93
/Users/paulross/CLionProjects/skiplist/src/cpp/IntegrityEnums.h, 94
/Users/paulross/CLionProjects/skiplist/src/cpp/Node.h, 96
/Users/paulross/CLionProjects/skiplist/src/cpp/NodeRefs.h, 97
/Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.cpp, 98
/Users/paulross/CLionProjects/skiplist/src/cpp/RollingMedian.h, 98
/Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.cpp, 100
/Users/paulross/CLionProjects/skiplist/src/cpp/SkipList.h, 101
/Users/paulross/CLionProjects/skiplist/src/cpp/main.cpp, 95
/Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.h, 145
/Users/paulross/CLionProjects/skiplist/src/cpp/test/TestFramework.cpp, 150
/Users/paulross/CLionProjects/skiplist/src/cpp/test/save_stream_state.h, 102
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.cpp, 103
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_concurrent.h, 110
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.cpp, 111
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_documentation.h, 114
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.cpp, 114
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_functional.h, 123
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.cpp, 124
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_performance.h, 139
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.cpp, 140
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_print.h, 141
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.cpp, 142
/Users/paulross/CLionProjects/skiplist/src/cpp/test/test_rolling_median.h, 144
/Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.cpp, 171
/Users/paulross/CLionProjects/skiplist/src/cpy/OrderedStructs.h, 172
/Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.cpp, 156
/Users/paulross/CLionProjects/skiplist/src/cpy/cOrderedStructs.h, 159
/Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.cpp, 160
/Users/paulross/CLionProjects/skiplist/src/cpy/cSkipList.h, 170
/Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.cpp, 154
/Users/paulross/CLionProjects/skiplist/src/cpy/cmpPyObject.h, 155
_adjRemoveRefs
OrderedStructs::SkipList::HeadNode< T, Compare >, 40
OrderedStructs::SkipList::Node< T, Compare >, 55
_cmp
OrderedStructs::SkipList::HeadNode< T, Compare >, 51
OrderedStructs::SkipList::Node< T, Compare >, 64
_count
OrderedStructs::SkipList::HeadNode< T, Compare >, 51
_data_type.cpp, SkipList, 67
_dot_file_subgraph
OrderedStructs::SkipList::HeadNode< T, Compare >, 51
_execTimeS
TestResult::test_result, 80
_failed
TestResultS, 91
_ff
StreamFormatState, 70
_fill
StreamFormatState, 70
_lacksIntegrityCyclicReferences
OrderedStructs::SkipList::HeadNode< T, Compare >, 40
_lacksIntegrityNodeReferencesNotInList
OrderedStructs::SkipList::HeadNode< T, Compare >, 40
_lacksIntegrityOrder
OrderedStructs::SkipList::HeadNode< T, Compare

>, 41
_lacksIntegrityWidthAccumulation
OrderedStructs::SkipList::HeadNode< T, Compare >, 41
_name
TestResult, 88
_nodeAt
OrderedStructs::SkipList::HeadNode< T, Compare >, 42
_nodeRefs
OrderedStructs::SkipList::HeadNode< T, Compare >, 51
OrderedStructs::SkipList::Node< T, Compare >, 64
_nodes
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 79
_pSL
AcquireLock, 30
_pStream
StreamFormatState, 70
_results
TestResult, 88
TestResultS, 91
_start
ExecClock, 36
_state
cOrderedStructs.cpp, 158
_swapLevel
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 79
_test_perf_insert_count_has_remove_count_multi_threads
test_concurrent.cpp, 104
_test_perf_insert_count_has_remove_count_single_thread
test_concurrent.cpp, 105
_throwIfValueDoesNotCompare
OrderedStructs::SkipList::HeadNode< T, Compare >, 42
_throwValueErrorNotFound
OrderedStructs::SkipList::HeadNode< T, Compare >, 43
_throw_exceeds_size
OrderedStructs::SkipList, 27
_value
OrderedStructs::SkipList::Node< T, Compare >, 64
~AcquireLock
AcquireLock, 30
~Exception
OrderedStructs::SkipList::Exception, 34
~HeadNode
OrderedStructs::SkipList::HeadNode< T, Compare >, 39
~StreamFormatState
StreamFormatState, 69
~cmpPyObject
cmpPyObject, 31

AcquireLock, 29
_pSL, 30
~AcquireLock, 30
AcquireLock, 29
allNodePointerMatch
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 74
ASSERT_TYPE_IN_RANGE
OrderedStructs.h, 172
at
OrderedStructs::SkipList::HeadNode< T, Compare >, 43, 44
OrderedStructs::SkipList::Node< T, Compare >, 56
at_sequence_bytes
cSkipList.cpp, 162
at_sequence_double
cSkipList.cpp, 163
at_sequence_long
cSkipList.cpp, 163
at_sequence_object
cSkipList.cpp, 163
atomicTestMeanExecTime
TestResult, 85

bytes_as_std_string
OrderedStructs.cpp, 171
OrderedStructs.h, 174

c_skip_list_docs
cOrderedStructs.cpp, 158
canSwap
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 75
check_index_against_size
cSkipList.cpp, 163
cmp_func
cmpPyObject, 33
cmpPyObject, 30
~cmpPyObject, 31
cmp_func, 33
cmpPyObject, 31
copy_cmpPyObject, 32
operator(), 32
operator=, 33
cmpPyObject.h
SKIPLIST_TRACE_CMPPYOBJECT_REF_COUNTS, 155
copy_cmpPyObject
cmpPyObject, 32
cOrderedStructs.cpp
_state, 158
c_skip_list_docs, 158
GETSTATE, 156
initorderedstructs, 157
long_max_value, 157
long_min_value, 157
MODULE_BUILD_DOCS, 158
MODULE_VERSION, 158
orderedstructsmodule_methods, 158

seed_rand, 157
seed_rand_docs, 159
toss_coin, 157
toss_coin_docs, 159
cOrderedStructs.h
ORDERED_STRUCTS_MODULE_NAME, 160
count
TestResult::test_result, 80
count_of_unique_string
TestFramework.cpp, 146
TestFramework.h, 151
cSkipList.cpp
at_sequence_bytes, 162
at_sequence_double, 163
at_sequence_long, 163
at_sequence_object, 163
check_index_against_size, 163
decref_all_contents, 163
get_size, 164
has_object, 164
INCLUDE_METHODS_THAT_USE_STREAMS, 162
index_object, 164
py_skip_list_docs, 169
remove_bytes, 164
remove_double, 165
remove_long, 165
remove_object, 165
SkipList_at, 165
SkipList_at_sequence, 165
SkipList_dealloc, 166
SkipList_dot_file, 166
SkipList_has, 166
SkipList_height, 166
SkipList_index, 166
SkipList_init, 167
SkipList_insert, 167
SkipList_lacks_integrity, 167
SkipList_length, 167
SkipList_members, 169
SkipList_methods, 169
SkipList_new, 168
SkipList_node_height, 168
SkipList_node_width, 168
SkipList_remove, 168
SkipList_SequenceMethods, 170
SkipList_size, 169
SkipList_size_of, 169
SkipListType, 170
cSkipList.h
SkipListType, 171

decref_all_contents
cSkipList.cpp, 163
dest_count
OrderedStructs::RollingMedian, 20
dest_size
OrderedStructs::RollingMedian, 21
dotFile
OrderedStructs::SkipList::HeadNode< T, Compare >, 44
OrderedStructs::SkipList::Node< T, Compare >, 57
dotFileFinalise
OrderedStructs::SkipList::HeadNode< T, Compare >, 45
dump_header
TestResultS, 89
dump_tail
TestResultS, 90
dump_tests
TestResultS, 90

error
module_state, 53
even_index
OrderedStructs::RollingMedian, 21
even_odd_index
OrderedStructs::RollingMedian, 22
Exception
OrderedStructs::SkipList::Exception, 34
ExecClock, 35
_start, 36
ExecClock, 36
seconds, 36
tHiResDouble, 35
execTime
TestResult, 85
execTimeAdd
TestResult, 85
execTimeMax
TestResult, 85
execTimeMin
TestResult, 85
execTimeStdDev
TestResult, 86

failed
TestResult, 86
TestResult::test_result, 80
TestResultS, 90
FailedComparison
OrderedStructs::SkipList::FailedComparison, 37
failure
SubTestCount, 71
FUNCTION_WIDTH
test_print.cpp, 141
test_print.h, 142

get_size
cSkipList.cpp, 164
GETSTATE
cOrderedStructs.cpp, 156
GLOBAL_REPEAT_COUNT
test_performance.cpp, 139
gSkipListMutex
OrderedStructs::SkipList, 28

has
OrderedStructs::SkipList::HeadNode< T, Compare >, 45
OrderedStructs::SkipList::Node< T, Compare >, 57
has_object
cSkipList.cpp, 164
hasExecTimeStdDev
TestResult, 86
HeadNode
OrderedStructs::SkipList::HeadNode< T, Compare >, 39
HEADNODE_CONTAINS_NULL
OrderedStructs::SkipList, 26
HEADNODE_COUNT_MISMATCH
OrderedStructs::SkipList, 26
HEADNODE_DETECTS_CYCLIC_REFERENCE
OrderedStructs::SkipList, 26
HEADNODE_DETECTS_OUT_OF_ORDER
OrderedStructs::SkipList, 26
HEADNODE_LEVEL_WIDTHS_MISMATCH
OrderedStructs::SkipList, 26
height
OrderedStructs::SkipList::HeadNode< T, Compare >, 46
OrderedStructs::SkipList::Node< T, Compare >, 58
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 75

INCLUDE_METHODS_THAT_USE_STREAMS
cSkipList.cpp, 162
SkipList.h, 102
incSwapLevel
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 75
index
OrderedStructs::SkipList::HeadNode< T, Compare >, 46
OrderedStructs::SkipList::Node< T, Compare >, 58
index_object
cSkipList.cpp, 164
IndexError
OrderedStructs::SkipList::IndexError, 52
initorderedstructs
cOrderedStructs.cpp, 157
insert
OrderedStructs::SkipList::HeadNode< T, Compare >, 47
OrderedStructs::SkipList::Node< T, Compare >, 58
insert_count_has_remove_count
test_concurrent.cpp, 105
insert_has_remove
test_concurrent.cpp, 106
insert_has_remove_count
test_concurrent.cpp, 106
insert_value
test_concurrent.cpp, 107
INTEGRITY_SUCCESS
OrderedStructs::SkipList, 26
IntegrityCheck
OrderedStructs::SkipList, 26

KeyDataType
OrderedStructs.h, 173

lacksIntegrity
OrderedStructs::SkipList::HeadNode< T, Compare >, 47
OrderedStructs::SkipList::Node< T, Compare >, 59
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 75
lacksIntegrityRefsInSet
OrderedStructs::SkipList::Node< T, Compare >, 59
lock
SkipList, 67
long_max_value
cOrderedStructs.cpp, 157
long_min_value
cOrderedStructs.cpp, 157

m_failure
SubTestCount, 72
m_test_count
SubTestCount, 72
main
main.cpp, 95
main.cpp
main, 95
test_all, 95
test_clock_resolution, 95
test_example_introduction_A, 96
message
OrderedStructs::SkipList::Exception, 34
MODULE_BUILD_DOCS
cOrderedStructs.cpp, 158
module_state, 53
error, 53
MODULE_VERSION
cOrderedStructs.cpp, 158
msg
OrderedStructs::SkipList::Exception, 34

name
TestResult, 86
next
OrderedStructs::SkipList::Node< T, Compare >, 60
Node
OrderedStructs::SkipList::Node< T, Compare >, 55
Node.h
nullptr, 97
NODE_HEIGHT_EXCEEDS_HEADNODE

OrderedStructs::SkipList, 26
NODE_HEIGHT_ZERO
OrderedStructs::SkipList, 26
NODE_NON_NULL_AFTER_NULL
OrderedStructs::SkipList, 26
NODE_REFERENCES_NOT_IN_GLOBAL_SET
OrderedStructs::SkipList, 26
NODE_SELF_REFERENCE
OrderedStructs::SkipList, 26
nodeRefs
OrderedStructs::SkipList::Node< T, Compare >, 60, 61
NODEREFS_WIDTH_DECREASING
OrderedStructs::SkipList, 26
NODEREFS_WIDTH_ZERO_NOT_UNITY
OrderedStructs::SkipList, 26
noNodePointerMatches
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 76
nullptr
Node.h, 97
numScaleValues
TestResult, 86
numTests
TestResult, 87

odd_index
OrderedStructs::RollingMedian, 23
operator<<
TestFramework.cpp, 147
TestFramework.h, 151, 152
operator()
cmpPyObject, 32
reversed< T >, 66
operator=
cmpPyObject, 33
OrderedStructs::SkipList::HeadNode< T, Compare >, 48
OrderedStructs::SkipList::Node< T, Compare >, 61
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 76
TestResult, 87
operator[]
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 76, 77
ORDERED_STRUCTS_MODULE_NAME
cOrderedStructs.h, 160
OrderedStructs, 19
OrderedStructs.cpp
bytes_as_std_string, 171
std_string_as_bytes, 171
OrderedStructs.h
ASSERT_TYPE_IN_RANGE, 172
bytes_as_std_string, 174
KeyDataType, 173
std_string_as_bytes, 174
TYPE_BYTES, 174
TYPE_DOUBLE, 174
TYPE_LONG, 174
TYPE_OBJECT, 174
TYPE_OVERFLOW, 174
TYPE_TYPE_BYTES, 172
TYPE_TYPE_DOUBLE, 173
TYPE_TYPE_LONG, 173
TYPE_TYPE_OBJECT, 173
TYPE_ZERO, 174
OrderedStructs::RollingMedian, 19
dest_count, 20
dest_size, 21
even_index, 21
even_odd_index, 22
odd_index, 23
rolling_median, 24
ROLLING_MEDIAN_DESTINATION_STRIDE, 20
rolling_median_lower_bound, 24
ROLLING_MEDIAN_SOURCE_STRIDE, 20
ROLLING_MEDIAN_SUCCESS, 20
ROLLING_MEDIAN_WIN_LENGTH, 20
RollingMedianResult, 20
OrderedStructs::SkipList, 25
_throw_exceeds_size, 27
gSkipListMutex, 28
HEADNODE_CONTAINS_NULL, 26
HEADNODE_COUNT_MISMATCH, 26
HEADNODE_DETECTS_CYCLIC_REFERENCE, 26
HEADNODE_DETECTS_OUT_OF_ORDER, 26
HEADNODE_LEVEL_WIDTHS_MISMATCH, 26
INTEGRITY_SUCCESS, 26
IntegrityCheck, 26
NODE_HEIGHT_EXCEEDS_HEADNODE, 26
NODE_HEIGHT_ZERO, 26
NODE_NON_NULL_AFTER_NULL, 26
NODE_REFERENCES_NOT_IN_GLOBAL_SET, 26
NODE_SELF_REFERENCE, 26
NODEREFS_WIDTH_DECREASING, 26
NODEREFS_WIDTH_ZERO_NOT_UNITY, 26
seedRand, 27
tossCoin, 27
OrderedStructs::SkipList::Exception, 33
~Exception, 34
Exception, 34
message, 34
msg, 34
OrderedStructs::SkipList::FailedComparison, 36
FailedComparison, 37
OrderedStructs::SkipList::HeadNode< T, Compare >, 37
_adjRemoveRefs, 40
_compare, 51
_count, 51
_dot_file_subgraph, 51
_lacksIntegrityCyclicReferences, 40
_lacksIntegrityNodeReferencesNotInList, 40
_lacksIntegrityOrder, 41

_lacksIntegrityWidthAccumulation, 41
_nodeAt, 42
_nodeRefs, 51
_throwIfValueDoesNotCompare, 42
_throwValueErrorNotFound, 43
~HeadNode, 39
at, 43, 44
dotFile, 44
dotFileFinalise, 45
has, 45
HeadNode, 39
height, 46
index, 46
insert, 47
lacksIntegrity, 47
operator=, 48
remove, 48
size, 49
size_of, 49
width, 49
OrderedStructs::SkipList::IndexError, 52
IndexError, 52
OrderedStructs::SkipList::Node< T, Compare >, 53
_adjRemoveRefs, 55
_compare, 64
_nodeRefs, 64
_value, 64
at, 56
dotFile, 57
has, 57
height, 58
index, 58
insert, 58
lacksIntegrity, 59
lacksIntegrityRefsInSet, 59
next, 60
Node, 55
nodeRefs, 60, 61
operator=, 61
pNode, 61
remove, 61
size_of, 62
value, 62
width, 63
writeNode, 63
OrderedStructs::SkipList::NodeRef< T, Compare >, 64
pNode, 65
width, 65
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 72
_nodes, 79
_swapLevel, 79
allNodePointerMatch, 74
canSwap, 75
height, 75
incSwapLevel, 75
lacksIntegrity, 75
noNodePointerMatches, 76
operator=, 76
operator[], 76, 77
pop_back, 77
push_back, 78
resetSwapLevel, 78
size_of, 78
swap, 79
swapLevel, 79
SwappableNodeRefStack, 74
OrderedStructs::SkipList::ValueError, 91
ValueError, 92
orderedstructsmodule_methods
cOrderedStructs.cpp, 158

perf_1m_median_values
test_performance.cpp, 126
perf_1m_medians_1000_vectors
test_performance.cpp, 126
perf_at_in_one_million
test_performance.cpp, 127
perf_has_in_one_million_vary_length
test_performance.cpp, 127
perf_index
test_performance.cpp, 127
perf_index_vary_length
test_performance.cpp, 128
perf_large_skiplist_ins_only
test_performance.cpp, 128
perf_large_skiplist_ins_rem
test_performance.cpp, 128
perf_median_sliding_window
test_performance.cpp, 129
perf_roll_med_by_win_size
test_performance.cpp, 129
perf_roll_med_odd_index
test_rolling_median.cpp, 143
perf_roll_med_odd_index_wins
test_performance.cpp, 129
perf_roll_med_vector_style_even_win_length
test_performance.cpp, 130
perf_roll_med_vector_style_even_win_length_string
test_performance.cpp, 130
perf_roll_med_vector_style_odd_win_length
test_performance.cpp, 130
perf_simulate_real_use
test_performance.cpp, 131
perf_single_at_middle
test_performance.cpp, 131
perf_single_has_middle
test_performance.cpp, 131
perf_single_ins_at_rem_middle
test_performance.cpp, 132
perf_single_ins_rem_middle
test_performance.cpp, 132
perf_single_ins_rem_middle_vary_length
test_performance.cpp, 132
perf_single_insert_remove
test_performance.cpp, 133
perf_size

test_performance.cpp, 133
perf_size_of_1m
test_performance.cpp, 133
perf_size_of_char_vary_length
test_performance.cpp, 134
perf_size_of_double_vary_length
test_performance.cpp, 134
perf_skiplist
test_performance.cpp, 134
perf_skiplist_unfair_coin
test_performance.cpp, 135
perf_test_double_at_1m
test_performance.cpp, 135
perf_test_double_at_1m_all
test_performance.cpp, 135
perf_test_double_has_1m
test_performance.cpp, 135
perf_test_double_has_1m_all
test_performance.cpp, 136
perf_test_double_index_1m
test_performance.cpp, 136
perf_test_double_index_1m_all
test_performance.cpp, 136
perf_test_double_insert_remove_value
test_performance.cpp, 136
perf_test_double_insert_remove_value_begin
test_performance.cpp, 137
perf_test_double_insert_remove_value_end
test_performance.cpp, 137
perf_test_double_insert_remove_value_mid
test_performance.cpp, 137
perf_test_node_height_growth
test_performance.cpp, 137
perf_test_string_insert_remove_value
test_performance.cpp, 138
perf_test_string_insert_remove_value_begin
test_performance.cpp, 138
perf_test_string_insert_remove_value_end
test_performance.cpp, 138
perf_test_string_insert_remove_value_mid
test_performance.cpp, 138
pNode
OrderedStructs::SkipList::Node< T, Compare >, 61
OrderedStructs::SkipList::NodeRef< T, Compare >, 65
pop_back
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 77
print_result
test_print.cpp, 141
test_print.h, 142
pSl_bytes
SkipList, 67
pSl_double
SkipList, 67
pSl_long
SkipList, 68
pSl_object
SkipList, 68
pSl_void
SkipList, 68
push_back
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 78
TestResultS, 90
py_skip_list_docs
cSkipList.cpp, 169

random_string
TestFramework.cpp, 148
TestFramework.h, 152
REGEX_SPACE_ANYTHING
TestFramework.cpp, 146
REGEX_SPACE_FLOAT
TestFramework.cpp, 146
REGEX_SPACE_INTEGER
TestFramework.cpp, 146
REGEX_SPACE_STRING_NO_SPACE
TestFramework.cpp, 146
remove
OrderedStructs::SkipList::HeadNode< T, Compare >, 48
OrderedStructs::SkipList::Node< T, Compare >, 61
remove_bytes
cSkipList.cpp, 164
remove_double
cSkipList.cpp, 165
remove_long
cSkipList.cpp, 165
remove_object
cSkipList.cpp, 165
reset_count_of_unique_string
TestFramework.cpp, 148
TestFramework.h, 152
resetSwapLevel
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 78
results
TestResultS, 90
reversed< T >, 65
operator(), 66
rolling_median
OrderedStructs::RollingMedian, 24
ROLLING_MEDIAN_DESTINATION_STRIDE
OrderedStructs::RollingMedian, 20
ROLLING_MEDIAN_ERROR_CHECK
RollingMedian.h, 99
rolling_median_lower_bound
OrderedStructs::RollingMedian, 24
ROLLING_MEDIAN_SOURCE_STRIDE
OrderedStructs::RollingMedian, 20
ROLLING_MEDIAN_SUCCESS
OrderedStructs::RollingMedian, 20
ROLLING_MEDIAN_WIN_LENGTH
OrderedStructs::RollingMedian, 20

RollingMedian.h
ROLLING_MEDIAN_ERROR_CHECK, 99
RollingMedianResult
OrderedStructs::RollingMedian, 20

scaleValues
TestResult, 87
seconds
ExecClock, 36
seed_rand
cOrderedStructs.cpp, 157
seed_rand_docs
cOrderedStructs.cpp, 159
seedRand
OrderedStructs::SkipList, 27
setFailed
TestResult, 87
size
OrderedStructs::SkipList::HeadNode< T, Compare >, 49
size_of
OrderedStructs::SkipList::HeadNode< T, Compare >, 49
OrderedStructs::SkipList::Node< T, Compare >, 62
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 78
SkipList, 66
_data_type, 67
lock, 67
pSl_bytes, 67
pSl_double, 67
pSl_long, 68
pSl_object, 68
pSl_void, 68
SkipList.h
INCLUDE_METHODS_THAT_USE_STREAMS, 102
SkipList_at
cSkipList.cpp, 165
SkipList_at_sequence
cSkipList.cpp, 165
SkipList_dealloc
cSkipList.cpp, 166
SkipList_dot_file
cSkipList.cpp, 166
SKIPLIST_FIXED_LENGTH
test_concurrent.cpp, 109
SkipList_has
cSkipList.cpp, 166
SkipList_height
cSkipList.cpp, 166
SkipList_index
cSkipList.cpp, 166
SkipList_init
cSkipList.cpp, 167
SkipList_insert
cSkipList.cpp, 167
SkipList_lacks_integrity
cSkipList.cpp, 167
SkipList_length
cSkipList.cpp, 167
SKIPLIST_MAX_THREADS
test_concurrent.cpp, 110
SkipList_members
cSkipList.cpp, 169
SkipList_methods
cSkipList.cpp, 169
SkipList_new
cSkipList.cpp, 168
SkipList_node_height
cSkipList.cpp, 168
SkipList_node_width
cSkipList.cpp, 168
SkipList_remove
cSkipList.cpp, 168
SkipList_SequenceMethods
cSkipList.cpp, 170
SkipList_size
cSkipList.cpp, 169
SkipList_size_of
cSkipList.cpp, 169
SKIPLIST_TRACE_CMPPYOBJECT_REF_COUNTS
cmpPyObject.h, 155
SkipListType
cSkipList.cpp, 170
cSkipList.h, 171
std_string_as_bytes
OrderedStructs.cpp, 171
OrderedStructs.h, 174
str_count
TestFramework.cpp, 150
StreamFormatState, 68
_ff, 70
_fill, 70
_pStream, 70
~StreamFormatState, 69
StreamFormatState, 69
SubTestCount, 70
failure, 71
m_failure, 72
m_test_count, 72
SubTestCount, 71
test, 71
test_count, 71
test_failures, 71
swap
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 79
swapLevel
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 79
SwappableNodeRefStack
OrderedStructs::SkipList::SwappableNodeRefStack< T, Compare >, 74

test
SubTestCount, 71

test_all
main.cpp, 95
test_at
test_functional.cpp, 116
test_at_dest
test_functional.cpp, 116
test_at_dest_fails
test_functional.cpp, 116
test_at_fails
test_functional.cpp, 117
test_at_large
test_functional.cpp, 117
test_clock_resolution
main.cpp, 95
test_concurrent.cpp
_test_perf_insert_count_has_remove_count_multi_threads, 104
_test_perf_insert_count_has_remove_count_single_thread, 105
insert_count_has_remove_count, 105
insert_has_remove, 106
insert_has_remove_count, 106
insert_value, 107
SKIPLIST_FIXED_LENGTH, 109
SKIPLIST_MAX_THREADS, 110
test_concurrent_all, 107
test_perf_insert_count_has_remove_count_multi_threads_fixed_length, 107
test_perf_sim_rolling_median_multi_thread_multi_no_threads, 108
test_perf_sim_rolling_median_single_thread, 108
test_perf_single_thread_fixed_length, 108
test_single_thread_insert, 108
test_two_thread_insert_count_has_remove_count, 109
test_two_thread_insert_has_remove, 109
test_concurrent.h
test_concurrent_all, 111
test_concurrent_all
test_concurrent.cpp, 107
test_concurrent.h, 111
test_count
SubTestCount, 71
test_doc_height_trend
test_documentation.cpp, 112
test_doc_insert
test_documentation.cpp, 112
test_doc_insert_remove
test_documentation.cpp, 112
test_doc_insert_remove_duplicates
test_documentation.cpp, 112
test_doc_insert_remove_repeat
test_documentation.cpp, 113
test_documentation.cpp
test_doc_height_trend, 112
test_doc_insert, 112
test_doc_insert_remove, 112
test_doc_insert_remove_duplicates, 112
test_doc_insert_remove_repeat, 113
test_documentation_all, 113
tests_doc_simple_dot, 113
test_documentation.h
test_documentation_all, 114
test_documentation_all
test_documentation.cpp, 113
test_documentation.h, 114
test_example_introduction_A
main.cpp, 96
test_failures
SubTestCount, 71
test_functional.cpp
test_at, 116
test_at_dest, 116
test_at_dest_fails, 116
test_at_fails, 117
test_at_large, 117
test_functional_all, 117
test_has, 118
test_has_not, 118
test_index_basic_7_node, 118
test_index_large, 118
test_index_throws, 119
test_ins_at_rem_with_srand, 119
test_ins_rem_rand, 119
test_insert_and_remove_same, 120
test_insert_n_numbers_same, 120
test_insert_nan_throws, 120
test_insert_one_million, 121
test_insert_remove_multiple, 121
test_remove_fails, 121
test_reversed_simple_insert, 122
test_simple_insert, 122
test_single_ins_rem_middle, 122
test_single_insert_remove, 123
test_very_simple_insert, 123
test_functional.h
test_functional_all, 124
test_functional_all
test_functional.cpp, 117
test_functional.h, 124
test_has
test_functional.cpp, 118
test_has_not
test_functional.cpp, 118
test_index_basic_7_node
test_functional.cpp, 118
test_index_large
test_functional.cpp, 118
test_index_throws
test_functional.cpp, 119
test_ins_at_rem_with_srand
test_functional.cpp, 119
test_ins_rem_rand
test_functional.cpp, 119
test_insert_and_remove_same
test_functional.cpp, 120

test_insert_n_numbers_same
test_functional.cpp, 120
test_insert_nan_throws
test_functional.cpp, 120
test_insert_one_million
test_functional.cpp, 121
test_insert_remove_multiple
test_functional.cpp, 121
test_perf_insert_count_has_remove_count_multi_threads_fixed_length
test_concurrent.cpp, 107
test_perf_sim_rolling_median_multi_thread_multi_no_thread
test_concurrent.cpp, 108
test_perf_sim_rolling_median_single_thread
test_concurrent.cpp, 108
test_perf_single_thread_fixed_length
test_concurrent.cpp, 108
test_performance.cpp
GLOBAL_REPEAT_COUNT, 139
perf_1m_median_values, 126
perf_1m_medians_1000_vectors, 126
perf_at_in_one_million, 127
perf_has_in_one_million_vary_length, 127
perf_index, 127
perf_index_vary_length, 128
perf_large_skiplist_ins_only, 128
perf_large_skiplist_ins_rem, 128
perf_median_sliding_window, 129
perf_roll_med_by_win_size, 129
perf_roll_med_odd_index_wins, 129
perf_roll_med_vector_style_even_win_length, 130
perf_roll_med_vector_style_even_win_length_string, 130
perf_roll_med_vector_style_odd_win_length, 130
perf_simulate_real_use, 131
perf_single_at_middle, 131
perf_single_has_middle, 131
perf_single_ins_at_rem_middle, 132
perf_single_ins_rem_middle, 132
perf_single_ins_rem_middle_vary_length, 132
perf_single_insert_remove, 133
perf_size, 133
perf_size_of_1m, 133
perf_size_of_char_vary_length, 134
perf_size_of_double_vary_length, 134
perf_skiplist, 134
perf_skiplist_unfair_coin, 135
perf_test_double_at_1m, 135
perf_test_double_at_1m_all, 135
perf_test_double_has_1m, 135
perf_test_double_has_1m_all, 136
perf_test_double_index_1m, 136
perf_test_double_index_1m_all, 136
perf_test_double_insert_remove_value, 136
perf_test_double_insert_remove_value_begin, 137
perf_test_double_insert_remove_value_end, 137
perf_test_double_insert_remove_value_mid, 137
perf_test_node_height_growth, 137
perf_test_string_insert_remove_value, 138
perf_test_string_insert_remove_value_begin, 138
perf_test_string_insert_remove_value_end, 138
perf_test_string_insert_remove_value_mid, 138
test_performance_all, 139
test_performance.h
test_performance_all, 139
test_performance_all
test_performance.cpp, 139
test_performance.h, 139
test_print.cpp
FUNCTION_WIDTH, 141
print_result, 141
test_print.h
FUNCTION_WIDTH, 142
print_result, 142
test_remove_fails
test_functional.cpp, 121
test_reversed_simple_insert
test_functional.cpp, 122
test_roll_med_even_mean
test_rolling_median.cpp, 143
test_roll_med_even_win
test_rolling_median.cpp, 143
test_roll_med_simple
test_rolling_median.cpp, 144
test_rolling_median.cpp
perf_roll_med_odd_index, 143
test_roll_med_even_mean, 143
test_roll_med_even_win, 143
test_roll_med_simple, 144
test_rolling_median_all, 144
test_rolling_median.h
test_rolling_median_all, 145
test_rolling_median_all
test_rolling_median.cpp, 144
test_rolling_median.h, 145
test_simple_insert
test_functional.cpp, 122
test_single_ins_rem_middle
test_functional.cpp, 122
test_single_insert_remove
test_functional.cpp, 123
test_single_thread_insert
test_concurrent.cpp, 108
test_two_thread_insert_count_has_remove_count
test_concurrent.cpp, 109
test_two_thread_insert_has_remove
test_concurrent.cpp, 109
test_very_simple_insert
test_functional.cpp, 123
testCount
TestResult, 87
TestFramework.cpp
count_of_unique_string, 146
operator<<, 147
random_string, 148
REGEX_SPACE_ANYTHING, 146
REGEX_SPACE_FLOAT, 146

REGEX_SPACE_INTEGER, 146
REGEX_SPACE_STRING_NO_SPACE, 146
reset_count_of_unique_string, 148
str_count, 150
TIME_PRECISION, 150
TIME_WIDTH, 150
unique_string, 148
unique_u16string, 149
unique_u32string, 149
unique_vector_char, 149
TestFramework.h
count_of_unique_string, 151
operator<<, 151, 152
random_string, 152
reset_count_of_unique_string, 152
unique_string, 152
unique_u16string, 153
unique_u32string, 153
unique_vector_char, 154
TestResult, 81
_name, 88
_results, 88
atomicTestMeanExecTime, 85
execTime, 85
execTimeAdd, 85
execTimeMax, 85
execTimeMin, 85
execTimeStdDev, 86
failed, 86
hasExecTimeStdDev, 86
name, 86
numScaleValues, 86
numTests, 87
operator=, 87
scaleValues, 87
setFailed, 87
testCount, 87
TestResult, 84
totalTime, 88
TestResult::test_result, 80
_execTimeS, 80
count, 80
failed, 80
TestResultS, 88
_failed, 91
_results, 91
dump_header, 89
dump_tail, 90
dump_tests, 90
failed, 90
push_back, 90
results, 90
TestResultS, 89
tResults, 89
tests_doc_simple_dot
test_documentation.cpp, 113
tHiResDouble
ExecClock, 35
TIME_PRECISION
TestFramework.cpp, 150
TIME_WIDTH
TestFramework.cpp, 150
toss_coin
cOrderedStructs.cpp, 157
toss_coin_docs
cOrderedStructs.cpp, 159
tossCoin
OrderedStructs::SkipList, 27
totalTime
TestResult, 88
tResults
TestResultS, 89
TYPE_BYTES
OrderedStructs.h, 174
TYPE_DOUBLE
OrderedStructs.h, 174
TYPE_LONG
OrderedStructs.h, 174
TYPE_OBJECT
OrderedStructs.h, 174
TYPE_OVERFLOW
OrderedStructs.h, 174
TYPE_TYPE_BYTES
OrderedStructs.h, 172
TYPE_TYPE_DOUBLE
OrderedStructs.h, 173
TYPE_TYPE_LONG
OrderedStructs.h, 173
TYPE_TYPE_OBJECT
OrderedStructs.h, 173
TYPE_ZERO
OrderedStructs.h, 174

unique_string
TestFramework.cpp, 148
TestFramework.h, 152
unique_u16string
TestFramework.cpp, 149
TestFramework.h, 153
unique_u32string
TestFramework.cpp, 149
TestFramework.h, 153
unique_vector_char
TestFramework.cpp, 149
TestFramework.h, 154

value
OrderedStructs::SkipList::Node< T, Compare >, 62
ValueError
OrderedStructs::SkipList::ValueError, 92

width
OrderedStructs::SkipList::HeadNode< T, Compare >, 49
OrderedStructs::SkipList::Node< T, Compare >, 63

OrderedStructs::SkipList::NodeRef< T, Compare >, 65
writeNode
OrderedStructs::SkipList::Node< T, Compare >, 63